

ABSTRACT

Autonomous vehicles depend on their perception stack not only to detect and track the objects around them, but also to judge whether each of those objects is behaving safely or dangerously. Estimating position and velocity is not sufficient: a car that signals and changes lanes smoothly and a car that swerves aggressively can occupy the same point in space at the same speed, yet demand entirely different responses from the ego vehicle. This project develops and critically evaluates an end-to-end pipeline that classifies the behavior of surrounding road users as either safe or risky from their motion alone, and more importantly measures how well that classifier survives contact with a realistic, imperfect sensing and tracking system.

The pipeline has four stages. First, more than four hundred and fifty driving scenarios were generated programmatically in MATLAB across three tiers of increasing complexity, spanning highways, rural and village roads, signalized crossroads, multi-turn routes, and railway level crossings, and populated with cars, trucks, buses, bicycles, pedestrians, children, and trains. Second, every actor's behavior was drawn from a library of more than fifteen named modules, each carrying a ground truth safe or risky label, and each actor's trajectory was reduced to ten interpretable motion features. Third, a fine decision tree classifier was trained on these features; a data-leakage investigation revealed that an early model reaching almost perfect accuracy was in fact reading behavior names from metadata rather than learning motion, and after removing the leaking columns and adopting a scenario level train/test split the leakage free classifier reached 95.4% cross validated and 96.0% held out accuracy. Fourth, the classifier was confronted with radar detections passed through a multi-object tracker, exposing a sharp fall in accuracy to 53.1% that this report traces, with evidence, to partial observation and track fragmentation rather than to any weakness of the features themselves.

The central contribution of the work is therefore not a single accuracy figure but an honest account of the gap between idealized and realistic performance, together with a live closed-loop demonstration in which the trained classifier actively shapes the ego vehicle's following distance and lane change decisions in real time. The report closes by identifying multi-sensor fusion and duration independent feature design as the two most promising routes to narrowing that gap.

Keywords: autonomous vehicles; behavior classification; radar; multi-object tracking; extended Kalman filter; decision tree; data leakage; sensor robustness; MATLAB Automated Driving Toolbox.

ACKNOWLEDGEMENTS

First and foremost, we express our sincere gratitude to Almighty Allah for granting us the strength, patience, and determination to successfully complete our project titled as **“Intelligent Object Monitoring System for Autonomous Vehicles”**

We would like to convey our deepest appreciation and heartfelt thanks to our respected supervisor, for his continuous guidance, valuable suggestions, constructive feedback, and constant encouragement throughout the entire development process.

We are also sincerely grateful to all the respected faculty members of the **Department of Computer Science and Engineering at Shyamoli Engineering College** for providing us with the academic foundation and technical knowledge that helped us carry out this project effectively.

We would like to express our sincere appreciation to our Honorable Head of the Department of Computer Science and Engineering for his valuable suggestions, guidance and support during the completion of the project. We also extend our appreciation to our classmates and friends for their cooperation, idea sharing, and moral support during different phases of the project development.

Finally, we are deeply thankful to our family members for their continuous encouragement, understanding, and unconditional support, which inspired us to complete this project successfully.

TABLE OF CONTENTS

ABSTRACT	i
ACKNOWLEDGEMENTS	ii
TABLE OF CONTENTS	iii
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF EQUATIONS	ix
LIST OF ABBREVIATIONS	x
CHAPTER 1: INTRODUCTION	1
1.1 Background Study.....	1
1.1.1 The Layered Architecture of Autonomous Driving.....	2
1.1.2 Levels of Driving Automation.....	2
1.1.3 Sensing Modalities and the Case for Radar.....	4
1.2 Problem Statement.....	5
1.3 Objectives.....	6
1.4 Scope of Work.....	6
1.5 Organization of the Report.....	7
CHAPTER 2: LITERATURE REVIEW	9
2.1 Related Work.....	9
2.1.1 Perception and Object Detection in Autonomous Vehicles.....	9
2.1.2 Multi-Object Tracking and State Estimation.....	11
2.1.3 Behavior and Intention Classification.....	12
2.1.4 Data Leakage and Evaluation Rigor.....	13
2.1.5 Surrogate Safety Metrics.....	15
2.1.6 Interpretable Machine Learning and Decision Trees.....	16
2.2 Previous Research Studies.....	18
2.3 Comparison with Existing Systems.....	19
2.4 Research Gap.....	20
CHAPTER 3: METHODOLOGY	21
3.1 System Overview.....	21
3.2 System Architecture.....	23

3.2.1 Scenario Generation and Labelling.....	23
3.2.2 Feature Extraction.....	24
3.2.3 Classifier Training and Evaluation.....	24
3.2.4 Radar and Multi Object Tracking.....	24
3.2.5 Closed Loop Demonstration.....	25
3.3 Algorithm and Flowchart.....	25
3.4 Data Collection, Tools and Technology.....	26
3.4.1 Scenario Generation and Dataset Composition.....	27
3.4.2 The Motion Feature Set.....	30
3.4.3 Tools and Technology.....	33
3.5 Evaluation Protocol and Experimental Controls.....	33
3.5.1 Scenario Level Data Splitting.....	34
3.5.2 Cross Validation on the Training Partition.....	34
3.5.3 Control Experiments by Design.....	35
CHAPTER 4: IMPLEMENTATION	36
4.1 Module Description.....	36
4.2 Software Implementation.....	38
4.3 Database Design (ER Model).....	41
4.4 Core Code Logic.....	42
4.4.1 A Single Feature Extractor for Both Paths.....	42
4.4.2 The Coordinate-Frame Correction.....	44
4.4.3 Matching Tracks Back to Actors.....	45
4.5 Reactive Closed-Loop Control.....	46
4.5.1 The Intelligent Driver Model.....	47
4.5.2 Arbitrating Leader, Signal, and Pedestrian Constraints.....	47
4.6 Closing the Loop: Classification-Driven Control.....	49
4.6.1 The Perception Classification Control Cadence.....	49
4.6.2 Longitudinal and Lateral Response to a Risky Verdict.....	50
4.7 Testing and Verification.....	51
4.7.1 Stage Level Consistency Checks.....	51
4.7.2 Verifying the Coordinate Frame Transform.....	52
4.7.3 Verifying the Tracker and the Track to Actor Join.....	52

4.7.4 Determinism and Repeatability.....	53
CHAPTER 5: RESULTS AND ANALYSIS	54
5.1 Experimental Setup.....	54
5.2 Output Visualizations.....	56
5.3 Performance Analysis.....	61
5.3.1 The Data Leakage Investigation.....	61
5.3.2 Leakage Free Performance.....	63
5.3.3 Robustness Under Realistic Sensing and Tracking.....	66
5.4 Error Analysis.....	70
5.4.1 Abstentions versus Misclassifications.....	70
5.4.2 The Safety Critical Asymmetry.....	71
5.5 Discussion.....	71
CHAPTER 6: CONCLUSION	73
6.1 Summary of Achievements.....	73
6.2 Limitations.....	74
6.3 Future Work.....	75
6.3.1 Multi Sensor Fusion.....	75
6.3.2 Duration Independent Feature Design.....	75
6.3.3 Sequence Models over Partial Tracks.....	76
6.3.4 Track Stitching and Re-Identification.....	76
6.3.5 Real Vehicle Validation.....	76
6.4 Concluding Remarks.....	77
BIBLIOGRAPHY	78
APPENDIX A: MATHEMATICAL FORMULATIONS	81
A.1 The Extended Kalman Filter.....	81
A.2 Time to Collision.....	82
A.3 Risk Determination and the Risky-Share Score.....	82
A.4 The Process Noise and Measurement Models.....	82
A.5 Data Association and Track Management.....	83
APPENDIX B: SYSTEM PARAMETERS	85
APPENDIX C: THE DATASET	86
APPENDIX D: MATLAB SOURCE CODE	89

D.1 Feature Extraction and Dataset Construction.....	89
D.2 Sensing, Tracking, and the Robustness Experiment.....	92
D.3 Baseline Scenario Generation.....	94
D.4 Reactive Car-Following Scenarios.....	96
D.5 Complex Multi-Crossroad Route Scenarios.....	98
D.6 The Live Closed-Loop Demonstration.....	102
D.7 Batch Generation Drivers.....	104
D.8 Visualization and Diagnostics.....	105
APPENDIX E: PERFORMANCE METRICS	107

LIST OF TABLES

Table 1.1	SAE levels of driving automation and the role of behavior understanding	3
Table 1.2	Comparison of the three dominant automotive sensing modalities	4
Table 2.1	Representative public autonomous-driving perception datasets	10
Table 2.2	This system compared with representative families of existing work	19
Table 3.1	Three-tier scenario generation scheme and the complexity added at each tier	27
Table 3.2	The ten motion features extracted from every actor trajectory	30
Table 3.3	MATLAB toolboxes and their role in the pipeline	33
Table 4.1	Representative safe and risky behavior modules	38
Table 4.2	Principal data files produced by the pipeline and their schemas	43
Table 4.3	The build chain by stage, with the principal script and output of each	40
Table 5.1	Radar, tracker, and runtime parameters used in the experiments	55
Table 5.2	Leakage-free classifier performance and configuration	64
Table 5.3	Classifier accuracy underground truth and sensor-tracked conditions	66
Table 6.1	Objectives set in Chapter 1 and the degree to which each was met	73
Table B.1	Dataset and train/test-split parameters	85
Table B.2	Radar and multi-object-tracker parameters	85
Table B.3	Classifier and live-demonstration parameters	85
Table C.1	The dataset at a glance	86
Table C.2	Composition of the dataset by generation batch	86
Table C.3	The complete behavior-module library	87
Table E.1	The confusion matrix for the binary safe/risky problem	108

LIST OF FIGURES

Figure 3.1 End to end pipeline, from scenario generation to the live closed loop demonstration	22
Figure 3.2 Perception to decision block diagram: scenario, radar, tracker, features, classifier	23
Figure 3.3 Algorithm flowchart of the behavior classification pipeline	25
Figure 3.4 Script level build chain for scenario generation and labelling	33
Figure 3.5 Distribution of the 450 scenarios across seven road-network types	28
Figure 3.6 Complexity tier composition and tier-3 ego maneuver mix	28
Figure 3.7 Labelled actor counts by type, split into safe and risky	29
Figure 3.8 Ground truth class balance across the dataset	30
Figure 4.1 Behavior module library, color coded by ground truth label	37
Figure 4.2 Six step software implementation build chain with scripts and outputs	39
Figure 4.3 Entity relationship model of the scenario and feature datastore	42
Figure 4.4 Coordinate frame correction: ego local returns to world frame	46
Figure 5.1 Experimental setup: radar coverage (140° / 150 m) and parameter	table55
Figure 5.2 Highway scenario canvas: tailgating and sudden braking	57
Figure 5.3 Signalized crossroad scenario canvas	57
Figure 5.4 Railway level crossing scenario canvas	58
Figure 5.5 multi-crossroad complex route scenario canvas	58
Figure 5.6 Bird's eye view of representative full ego routes	59
Figure 5.7 Live demo per-actor classification timeline over the full drive	60
Figure 5.8 Live demo: share of risky timesteps and final verdict per actor	61
Figure 5.9 Data leakage diagnostic: leaked, shuffled, and clean accuracy	62
Figure 5.10 Attribution of the leak to individual metadata columns	63
Figure 5.11 Confusion matrix of the leakage free classifier on held-out actors	64
Figure 5.12 Robustness accuracy: ground truth versus sensor tracked	67
Figure 5.13 Observation duration comparison, ground truth versus tracked	67
Figure 5.14 Track fragmentation distribution behind the observation gap	68
Figure 5.15 Which motion features separate safe from risky behavior	65
Figure 5.16 Feature corruption: ground truth versus sensor-tracked feature values	69
Figure C.1 Distribution of scenario durations across the dataset	88
Figure C.2 Village Road scenario canvas	88
Figure E.1 Live demo risky share signal over the full run	109

LIST OF EQUATIONS

Equation 3.1	Constant velocity state prediction for a tracked actor	26
Equation 3.2	Longitudinal acceleration obtained by differencing speed	32
Equation 3.3	Heading and yaw rate obtained from successive positions	32
Equation 3.4	Range from a tracked actor to the ego vehicle	32
Equation 4.1	Ego local to world coordinate transform, x component	45
Equation 4.2	Ego local to world coordinate transform, y component	45
Equation 4.3	Intelligent Driver Model acceleration command	48
Equation 4.4	Intelligent Driver Model desired following gap	48
Equation A.1	Extended Kalman filter state prediction	81
Equation A.2	Extended Kalman filter covariance prediction	81
Equation A.3	State transition matrix for the constant velocity model	81
Equation A.4	Measurement innovation	81
Equation A.5	Innovation covariance	81
Equation A.6	Kalman gain	81
Equation A.7	State update from the weighted innovation	81
Equation A.8	Covariance update after a measurement	81
Equation A.9	Time to collision between ego and a tracked actor	82
Equation A.10	Risky share score over a window of classified samples	82
Equation A.11	Process noise covariance block	83
Equation A.12	Radar measurement function mapping state to observation	83
Equation A.13	Measurement Jacobian of the radar model	83
Equation A.14	Squared Mahalanobis distance used for association gating	84
Equation A.15	Global nearest neighbour assignment of detections to tracks	84
Equation E.1	Accuracy	108
Equation E.2	Precision	108
Equation E.3	Recall	108
Equation E.4	F1 score	108
Equation E.5	Specificity	108
Equation E.6	Balanced accuracy	109

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ADAS	Advanced Driver Assistance System
AV	Autonomous Vehicle
BEV	Bird's-Eye View
CART	Classification and Regression Tree
CSV	Comma Separated Values
CV	Constant Velocity (motion model)
EKF	Extended Kalman Filter
ER	Entity Relationship (data model)
FOV	Field of View
FN / FP	False Negative / False Positive
GT	Ground Truth
IQR	Inter Quartile Range
LSTM	Long Short-Term Memory (network)
MOT	Multi-Object Tracker
RCS	Radar Cross Section
RMSE	Root Mean Square Error
ROC	Receiver Operating Characteristic
TTC	Time To Collision
TN / TP	True Negative / True Positive

CHAPTER 1: INTRODUCTION

The promise of the autonomous vehicle is not merely that a machine can steer, accelerate, and brake without a human hand, but that it can do so at least as safely as an attentive human driver, and eventually more safely [2], [3]. Every claim of that kind ultimately rests on a single, deceptively simple capability: the vehicle must understand what the objects around it are doing and what they are about to do. A pedestrian standing at a verb and a pedestrian stepping off it occupy almost the same position; a car holding its lane and a car beginning to drift into yours may, for a fraction of a second, look identical to a sensor. The difference between them is behavior, and behavior not position alone is what separates a safe maneuver from a collision. This report is about giving an autonomous vehicle the means to make that distinction, and, just as importantly, about measuring honestly how well that means holds up when it is fed the noisy, incomplete data that real sensors actually provide.

The project documented here builds a complete pipeline that classifies the behavior of surrounding road users as either safe or risky using only their motion, trains that classifier on a large body of programmatically generated driving scenarios, subjects it to a deliberately demanding robustness test against a simulated radar and tracker, and finally places it inside a live control loop where its predictions actively influence how the ego vehicle drives. The work was carried out entirely in MATLAB and Simulink, drawing on the Automated Driving Toolbox for scenario and sensor modelling [32], the Statistics and Machine Learning Toolbox for classifier training [34], and the Sensor Fusion and Tracking Toolbox for multi-object tracking [33]. The remainder of this chapter sets out the background that motivates the work, states the specific problem it addresses, enumerates the objectives against which it should be judged, delimits its scope, and describes how the rest of the report is organized.

1.1 Background Study

Driving is, at its core, a continuous exercise in prediction. A competent human driver does not simply observe that a cyclist is three meters to the right; they infer, from the cyclist's speed, wobble, head movement, and proximity to a parked car, whether that cyclist is about to pull into the traffic lane. This inference is so automatic that we rarely notice it, yet it is the single hardest thing to reproduce in software. Decades of research into advanced driver-assistance systems and, more recently, into fully autonomous driving have converged on a layered architecture [29], [45] in which raw sensor data is progressively refined into an actionable understanding of the scene.

1.1.1 The Layered Architecture of Autonomous Driving

That architecture is conventionally divided into perception, which detects and tracks objects [14], [16]; prediction, which anticipates their future motion [29]; planning, which decides what the ego vehicle should do; and control, which executes that decision. The

behavior-classification problem sits precisely at the seam between perception and prediction. It consumes the tracked trajectories that perception produces, and it produces a semantic judgement safe or risky that prediction and planning can act upon. Because it lives at this seam, it inherits every imperfection of the perception layer beneath it: if the tracker loses an object for two seconds, the behavior classifier must cope with a two-second hole in its evidence. Much of this report is concerned with exactly that inheritance.

A useful way to appreciate why this layer is difficult is to notice that it is the first point in the stack at which a purely geometric description of the world stops being sufficient. Everything beneath it can, at least in principle, be checked against geometric ground truth: a detected bounding box either does or does not overlap the true one, and a track either does or does not follow the object it claims to. Behavior has no such direct referent. Riskiness is a latent property that no sensor measures; it is inferred from a pattern of motion unfolding over time, and the same instantaneous position and velocity may be perfectly safe in one context and dangerous in another. This shift from the measurable to the inferable is what makes the behavior layer simultaneously the most valuable it is the first output a human would recognize as a judgement rather than a measurement and the most fragile, because it must construct that judgement from evidence that is always partial and often noisy.

The layered decomposition also fixes the direction in which errors travel. Because each layer consumes the output of the one below it, an error introduced in perception does not stay in perception; it propagates upward, and it can be amplified as it goes. A momentary failure to detect an object is, at the perception layer, a single missed frame. By the time it reaches the behavior layer it may have become a truncated trajectory, and a truncated trajectory can flip a classification from risky to safe at exactly the moment the judgement matters most. This upward propagation of error is not a peripheral concern to be handled once the interesting work is done; in a safety-critical system it is the interesting work, and it is the reason this project devotes an entire experimental chapter to measuring how much behavior-classification accuracy survives the imperfections of the layers beneath it.

1.1.2 Levels of Driving Automation

It is helpful to place the work within the widely used taxonomy of driving automation. The Society of Automotive Engineers defines six levels [1], from no automation to full automation, and the demands placed on behavior understanding grow sharply as one ascends them. Table 1.1 summarizes these levels and indicates where robust behavior classification becomes indispensable rather than merely helpful.

Table 1.1: The SAE levels of driving automation and the role of behavior understanding at each.

Level	Name	Who monitors the road	Behavior understanding demand
0	No automation	Human, always	None required of the system

1	Driver assistance	Human	Low single-function warnings
2	Partial automation	Human, supervising	Moderate lane and vehicle awareness
3	Conditional automation	System, human fallback	High must judge risk to request handover
4	High automation	System, within its domain	Critical no human fallback in-domain
5	Full automation	System, everywhere	Critical all conditions, all actors

The system built in this project is aimed squarely at the upper half of this table, where the vehicle itself not a supervising human must decide whether a nearby actor is dangerous. At those levels a misjudgment is not merely an inconvenience; it is a safety failure. This is the reason the report devotes as much attention to how the classifier fails as to how well it succeeds.

The transition from Level 2 to Level 3 is widely regarded as the most consequential step in the entire scale, because it is the point at which responsibility for monitoring the driving environment passes from the human to the machine. Below it, a distracted or over-trusting driver is still, legally and practically, the fallback; above it, the system must itself recognize when a situation exceeds its competence and either handle it or hand it back with enough warning to matter. Both halves of that duty rest on behavior understanding. To hand control back in time, a Level 3 system must judge that an approaching actor is behaving in a way it cannot safely accommodate which is precisely a risk classification and it must do so early enough that a human can re-engage. A behavior classifier that is accurate only on complete, clean trajectories is of little use here, because the situations that trigger a handover are exactly the fast-developing ones in which observation is most likely to be partial.

Underpinning the upper levels is the notion of an operational design domain [1], [3]: the specific range of conditions road types, speeds, weather, traffic within which an automated system is certified to operate. Every claim of safety is implicitly a claim about behavior within a stated domain, and a system that leaves its domain must detect that it has done so. The scenario dataset built for this project can be read as a deliberately widened operational design domain in miniature: it spans seven road network types, three complexity tiers, and a full mix of road users precisely so that the classifier is exercised across a broad slice of conditions rather than tuned to a single, flattering one. The robustness experiment then asks the harder question of whether performance within that domain survives when the perfect knowledge assumed during training is replaced by realistic sensing.

1.1.3 Sensing Modalities and the Case for Radar

The sensing foundation on which behavior understanding is built varies from platform to platform, and each modality shapes what the layers above it can achieve. Camera, radar, and

lidar differ in resolution, range, robustness to weather, and the physical quantities they measure directly [43]. The choice matters here because this project’s robustness test is grounded specifically in the characteristics of automotive radar, whose direct measurement of range and range rate, wide availability, and resilience to poor lighting make it a mainstay of production driver assistance systems [36], but whose limited angular resolution and field of view impose real constraints on how long any given actor can be observed. Table 1.2 contrasts the three dominant modalities on the axes most relevant to behavior classification.

Table 1.2: A comparison of the three dominant automotive sensing modalities on axes relevant to behavior understanding.

Property	Camera	Radar	Lidar
Directly measures	Color, texture	Range, range-rate	Range, 3-D shape
Angular resolution	High	Low–moderate	High
Velocity (direct)	No	Yes (Doppler)	No
Performance at night	Poor	Excellent	Excellent
Performance in rain/fog	Poor	Good	Moderate
Relative cost	Low	Low–moderate	High
Role in this project	Not modelled	Primary sensor	Not modelled

The decision to model radar rather than a camera or lidar was deliberate. Radar is the sensor most commonly deployed for the kind of forward-looking collision-avoidance function this project targets, and it is precisely its limited coverage a wide but shallow wedge in front of the vehicle that produces the most instructive failure mode in the entire study. A classifier that looks flawless on complete trajectories is shown, later in this report, to falter when it can only see each actor through the narrow, intermittent window that radar affords.

The physical reason for radar’s distinctive profile is worth stating, because it explains why the sensor is at once so well suited to this task and so limiting. Automotive radar operates in the millimeter wave band and measures two things about each reflector almost directly: its range, from the round-trip time of the transmitted signal, and its radial velocity the rate at which it is approaching or receding from the Doppler shift the motion imparts on the returned wave. This direct measurement of closing speed is exactly the quantity on which surrogate safety metrics such as time to collision depend [30], [31], so a radar first perception stack is, in a sense, measuring danger in its native units. Neither a camera nor a lidar recovers velocity so directly; both must infer it by differencing successive positions, which amplifies noise. What radar gives up in return is angular precision: because it resolves direction far more coarsely than it resolves range or speed, and because a single forward unit illuminates only a shallow wedge of the scene, it sees each actor briefly and cannot separate closely spaced objects as sharply as an optical sensor can. The chapters that follow show that it is this angular and geometric limitation, rather than any deficiency in the velocity or range measurement, that ultimately governs how well behavior can be classified from radar alone.

1.2 Problem Statement

Stated plainly, the problem this project addresses are the following. Given the motion of an object tracked in the vicinity of an autonomous vehicle, decide automatically and in real time whether that object is behaving safely or riskily, and determine how dependable that decision remains once the motion is no longer known perfectly but must instead be estimated from a noisy radar feed and an imperfect tracker.

Cast a little more formally, the task is to learn a mapping from a description of an actor’s motion to a binary label, safe or risky. The description is a fixed length vector of motion features summarizing how the actor moved; the label is the ground truth assigned when the actor’s behavior was generated. What makes the problem more than a routine classification exercise is that the same mapping must operate under two very different input conditions. In the first, the feature vector is computed from the actor’s true trajectory, known perfectly because the world was generated rather than observed. In the second, it is computed from whatever a simulated radar and tracker managed to reconstruct of that trajectory. The classifier is trained only under the first condition but must ultimately be judged under the second, and the distance between its performance in the two is the real object of study. Framing the problem as one mapping evaluated under two input regimes is what allows the eventual accuracy loss to be attributed cleanly to the data rather than to any change in the model.

This single sentence conceals three distinct difficulties, each of which the project confronts in turn. The first is data: there exists no convenient public dataset that pairs richly varied driving behaviors with clean safe-or-risky labels and full trajectories [11], [12], [13], so the data had to be generated. The second is honesty: a classifier trained on labelled data can achieve near perfect accuracy for entirely the wrong reasons, silently memorizing a label instead of learning the motion pattern that the label is supposed to represent [21], [22], and detecting this requires deliberate, adversarial self-examination rather than a satisfied glance at a high accuracy number. The third is realism: a model that performs well on perfect ground truth trajectories may collapse when fed the fragmentary observations a real sensor produces, and the only way to know is to build the sensor and tracker stack and measure the fall.

The project brief that seeded this work made the third difficulty explicit, asking the investigator to “assess the robustness of your classifier to errors introduced by the sensing and tracking.” That instruction became the spine of the project. It is easy to report the accuracy of a classifier on the same kind of clean data it was trained on; it is far more useful, and far more honest, to report what happens when that assumption is removed. The problem, then, is not only to build a behavior classifier, but to build one and then prove with evidence, quantitatively where and why it breaks.

1.3 Objectives

The project set out to achieve a small number of concretes, verifiable objectives. They are stated here so that the concluding chapter can return to them and assess, one by one, the extent to which each was met.

1. **Generate a large, diverse, labelled scenario dataset.** Produce several hundred driving scenarios spanning multiple road types and traffic mixes, in which each actor's behavior is drawn from a library of named safe and risky maneuvers carrying ground-truth labels.
2. **Extract genuine motion features and train a behavior classifier.** Reduce each actor's trajectory to a fixed set of interpretable motion features and train a classifier that distinguishes safe from risky behavior from those features alone.
3. **Detect and eliminate data leakage.** Actively test whether the classifier's accuracy reflects genuine learning rather than memorization of metadata, and, if leakage is found, remove its source and retrain.
4. **Evaluate robustness under realistic sensing and tracking.** Pass the scenarios through a simulated radar and multi object tracker, re-derive the same features from the tracked trajectories, and quantify how much accuracy is lost and why.
5. **Demonstrate closed loop operation.** Place the trained classifier inside a live driving loop in which its real time predictions visibly change the ego vehicle's following distance and lane change behavior.

These five objectives are intentionally ordered so that each depends on the one before it: without a labelled dataset there is nothing to train on, without a trustworthy classifier the robustness test would be meaningless, and without the robustness test the live demonstration would rest on an untested assumption. The report follows the same order.

1.4 Scope of Work

It is as important to state what this project does not attempt as what it does. The scope was drawn deliberately so that the work could be carried through end to end, with genuine depth in the robustness question, rather than spread thinly across every aspect of autonomous driving.

Within scope are the programmatic generation of driving scenarios and ground-truth labels; the extraction of motion features from trajectories; the training, leakage analysis, and evaluation of a behavior classifier; the modelling of a single forward-looking automotive radar together with a constant-velocity multi-object tracker [41]; the re-derivation of features from tracked data and the resulting robustness analysis; and a live closed-loop demonstration in which classification informs control. All of this is implemented and evaluated in simulation, which affords perfect ground truth against which sensor-degraded performance can be measured an advantage that is difficult to obtain on real roads.

The decision to work entirely in simulation deserves to be read as a methodological choice rather than a mere convenience [35]. The central question of the project how much accuracy is lost when perfect motion is replaced by sensor-tracked motion can only be answered if the perfect motion is known in the first place. On a real road it never is; the best one can obtain is a second, more expensive sensor treated as a reference, itself imperfect. Simulation dissolves this difficulty by construction: because the world is generated, the true trajectory of every actor is available exactly, and the degraded trajectory can be laid against it feature by feature. The price of that clarity is the reality gap between a synthetic radar and a physical one, which the report acknowledges wherever the results are stated and revisits in Chapter 6. The stance taken here is that, for the specific question of quantifying and explaining degradation, controlled access to ground truth is worth more than the surface realism of on-road data, and the two are in any case complementary steps rather than substitutes.

Outside scope are several things that a production system would require but that would not change the central findings. The project does not fuse multiple sensors; it models radar alone precisely because a single limited sensor makes the observation-coverage problem visible. It does not implement a full motion-planning or trajectory-optimization stack; the ego vehicle’s response to a risky classification is a deliberately simple, legible rule rather than an optimal planner. It does not train deep temporal networks [23], [42]; the classifier is a compact decision tree, chosen for interpretability so that its successes and failures can be explained rather than merely observed [44]. And it is not validated on physical hardware; the conclusions are drawn from high-fidelity simulation. Each of these boundaries is revisited in the future-work discussion of Chapter 6, where the natural extensions beyond the present scope are laid out.

1.5 Organization of the Report

The report is organized into six chapters and a set of appendices, following the natural progression of the work from context to conclusion.

- **Chapter 2, Literature Review**, surveys the prior work on perception, multi-object tracking, state estimation, and behavior classification, compares this system against representative existing approaches, and identifies the research gap this project fills.
- **Chapter 3, Methodology**, presents the four-stage pipeline: the system overview and block diagrams, the classification algorithm and its flowchart, and the data-collection, tooling, and scenario generation scheme.
- **Chapter 4, Implementation**, describes the behavior module library, the software build chain, the entity relationship model of the datastore, and the core code logic, including the coordinate frame correction at the heart of the robustness pipeline.
- **Chapter 5, Results and Analysis**, reports the experimental setup, output visualizations, the data leakage investigation, the leakage-free performance, and the sensor and tracking robustness analysis, with a discussion of what the numbers mean.

- **Chapter 6, Conclusion**, summarizes the achievements against the objectives, states the limitations candidly, and sets out the future work that would carry the system towards deployment.
- **The appendices** collect the Kalman-filter equations, the time to collision and risk-scoring formulations, the full system parameters, a description of the dataset, listings of the core MATLAB function, and the performance evaluation metrics.

Throughout, the report favors explanation over assertion. Where a number is given, the report tries to say how it was obtained and what could undermine it; where a design choice was made, it tries to say what the alternatives were and why this one was preferred. The aim is a document from which the work could be reconstructed and, more importantly, trusted.

CHAPTER 2: LITERATURE REVIEW

The system built in this project draws on four established bodies of work: the perception layer that detects and tracks objects around a vehicle, the estimation theory that turns noisy detections into smooth tracks, the growing literature on classifying and predicting the behavior of road users, and the methodological literature on how machine-learning results can mislead when evaluation is not done carefully. This chapter reviews each in turn, then surveys a representative set of prior studies, compares the present system against existing approaches, and closes by articulating the specific gap this work addresses. The review is written to make one argument in particular: that the honest evaluation of a behavior classifier under realistic sensing is undervalued in much of the literature, and that this is precisely where the present project concentrates its effort.

2.1 Related Work

The related work falls naturally into the layers of the autonomous driving stack that this project touches. The following five subsections move upward through that stack, from raw detection to the semantic judgement of behavior, step sideways to the question of evaluation rigor that cuts across all of them, and finally reach back to the surrogate safety metrics that give the very notion of “risk” its physical meaning.

2.1.1 Perception and Object Detection in Autonomous Vehicles

Perception is the foundation on which every higher-level driving function is built, and the last decade has seen it transformed by deep learning. Image based detectors such as the single stage YOLO family [14] and the two stage Faster R-CNN [15] made it possible to localize and classify vehicles, pedestrians, and cyclists in real time, while point cloud networks such as Point Pillars [16] extended the same idea to lidar, where the three-dimensional geometry of the scene is measured directly. A comprehensive survey by Feng et al. [43] catalogues the many ways these modalities can be combined, and makes clear that no single sensor dominates on every axis of range, resolution, cost, and robustness to weather.

Radar occupies a distinctive place in this landscape. Unlike a camera, it measures range and radial velocity directly through the Doppler effect, and unlike lidar it degrades gracefully in rain, fog, and darkness [43]. These properties have made radar a mainstay of production driver assistance systems, particularly for the forward-looking collision avoidance functions that this project models. The price of those advantages is angular resolution: a radar’s field of view is typically a wide but shallow wedge, and its ability

to separate two nearby objects in azimuth is limited. As later chapters show, it is exactly this shallow wedge geometry not sensor noise per se that most sharply limits how long any given actor can be observed, and therefore how reliably its behavior can be classified.

For all their sophistication, detectors are also evaluated in a way that quietly matters for everything built above them. The standard detection metric, mean average precision, rewards a detector for finding objects and localizing them well on average across a dataset, but it says comparatively little about the temporal continuity of those detections on any single object precisely the property a downstream behavior classifier depends upon. A detector that alternately finds and loses the same pedestrian from one frame to the next can post a respectable average precision while presenting the tracker beneath it with a stream of gaps. This mismatch between what perception is optimized for and what behavior understanding actually needs recurs throughout the stack, and it is one of the reasons a system can appear strong at the detection layer yet still starve the behavior layer of the continuous evidence it requires. The present project makes that starvation its explicit object of measurement, quantifying how a shortfall in continuous observation whatever its origin in the perception layer propagates into a shortfall in behavior-classification accuracy.

The empirical progress of perception has been driven as much by data as by architecture. Public benchmarks such as KITTI [11], nuScenes [12], and the Waymo Open Dataset [13] provided the large, carefully annotated corpora on which modern detectors were trained and compared, and in doing so they set the community’s expectations for how perception should be evaluated. Table 2.1 summarizes the scale and sensing of these benchmarks. Their existence is a double-edged sword for a project such as this one: they are invaluable for detection research, but none of them labels the behavior of each actor as safe or risky in the sense required here, and none provides the perfectly known ground truth against which sensor-degraded tracking can be measured. That absence is one of the reasons this project generates its own scenarios in simulation, where ground truth is available by construction.

Table 2.1: Representative public autonomous driving perception datasets and their principal sensing modalities.

Dataset	Year	Primary sensors	Scale / notes
KITTI [11]	2012	Stereo camera, lidar	15k annotated frames; detection and odometry
nuScenes [12]	2020	Camera, lidar, radar	1000 scenes; full 360° sensor suite
Waymo Open [13]	2020	Camera, lidar	1150 scenes; large-scale, high frame rate
This project	2026	Simulated radar	450+ scenarios; perfect ground truth for robustness

The point of the final row is not to claim comparable scale a simulated single-radar dataset cannot rival a multi city, multi sensor corpus but to highlight a different purpose. Where the public benchmarks optimize for detection accuracy on real imagery, this project optimizes for a controlled, fully observable environment in which the degradation caused by realistic sensing can be isolated and measured. The two purposes are complementary rather than competing.

2.1.2 Multi-Object Tracking and State Estimation

Detection tells a vehicle what is present in a single frame; tracking tells it what persists across frames, assigning a stable identity and a smoothed motion estimate to each object over time. The theoretical bedrock of tracking is the Kalman filter [4], which optimally fuses a motion model with noisy measurements for linear-Gaussian systems, and its extension to nonlinear systems through local linearization, the extended Kalman filter [7]. Bar-Shalom and colleagues [5] and Blackman and Popoli [6] codified the surrounding machinery of data association, gating, track initiation, and track deletion that turns a single-target filter into a practical multi object tracker. Thrun, Burgard, and Fox [45] situate all of this within the broader framework of probabilistic robotics.

The multi-object tracker used in this project is built directly on these ideas: it pairs a constant-velocity extended Kalman filter with a nearest-neighbor association step, and applies confirmation and deletion logic so that spurious detections do not create phantom tracks and lost objects are eventually dropped. The constant-velocity assumption is a deliberate, well understood simplification; Schöller et al. [41] show, perhaps surprisingly, that a constant bvelocity model remains a strong baseline even for pedestrian motion, which cautions against assuming that more complex models are automatically better. Where behavior is genuinely non-linear a hard brake, a sharp turn the filter lags, and that lag is one of several mechanisms by which tracked features diverge from ground truth features in the robustness experiments of Chapter 5.

Data association deciding which detection belongs to which track is the second pillar of tracking. The assignment problem it poses can be solved optimally by the Hungarian algorithm of Munkres [39], and more sophisticated schemes such as multiple-hypothesis tracking [40] defer association decisions until more evidence accrues. In the computer vision community, the SORT tracker [9] demonstrated that a simple Kalman filter combined with Hungarian assignment could achieve competitive real time tracking, and its successor DeepSORT [10] added a learned appearance descriptor to reduce identity switches. These systems are the modern lineage of the classical theory, and their well-documented failure mode fragmentation, in which a single object is tracked as a succession of short-lived tracks whenever association briefly fails is exactly the phenomenon that this project measures and must contend with.

The tracking community measures its systems with metrics that make this failure mode explicit, and they are worth naming because the present work effectively rediscovers them from the top down. Multiple object tracking accuracy folds missed detections, false tracks, and identity switches into a single figure, while identity preservation measures and the explicit count of track fragmentations penalize precisely the act of breaking one physical object into several tracks. A tracker that scores well on these metrics is, almost by definition, one that maintains long and unbroken identities which is the very property on which any duration dependent behavior feature silently relies. What this project adds is a measurement of the downstream consequence: rather than stopping at a fragmentation count, it follows

each fragmentation through to the shortened trajectory it produces and then to the corrupted feature and the misclassification that follow, closing a loop that the tracking literature and the behavior literature usually leave open at opposite ends.

It is also worth being explicit about why the constant-velocity assumption, for all its convenience, is a source of error rather than a neutral modelling choice. Between radar updates the filter propagates each track forward as though its velocity were unchanging; when the underlying actor is in fact braking hard or turning sharply, the prediction and the next measurement disagree, and the filter needs several updates to catch up. During those updates the estimated speed and heading lag their true values, so that the very maneuver's a safety system most wants to catch the sudden stop, the sharp swerve are the ones the tracker represents least faithfully. This is a second, subtler mechanism of feature corruption, distinct from outright fragmentation: even a track that is never broken can carry systematically smoothed estimates of exactly the aggressive motion that defines risky behavior. Chapter 5 observes both mechanisms acting together.

2.1.3 Behavior and Intention Classification

Above tracking sits the layer this project is chiefly concerned with: inferring what tracked objects intend or how they are behaving. Lefèvre, Vasquez, and Laugier [29] provide the canonical survey of motion prediction and risk assessment for intelligent vehicles, organizing the field into physics based, maneuvers-based, and interaction-aware approaches of increasing sophistication. The present work is maneuvers-based: it classifies an observed motion into a discrete behavior category (safe or risky) rather than predicting a continuous future trajectory, which keeps the output interpretable and directly actionable by a simple control rule.

The deep learning wing of this field predicts trajectories rather than labels. Social LSTM [23] and Social GAN [24] model the mutual influence of pedestrians in a crowd, and convolutional social pooling [25] applies analogous ideas to highway vehicles, all built on the long short-term memory network of Hochreiter and Schmidhuber [42]. These models are powerful, but they demand large realworld trajectory corpora and produce outputs that are harder to audit a serious drawback in a safety context, where Rudin [44] argues forcefully for interpretable models over post-hoc explanations of black boxes. A separate strand models behavior through the lens of interaction and game theory: Sadigh et al. [27] plan for autonomous cars that account for their effect on human drivers, and Hubmann et al. [28] treat other vehicles' maneuvers as hidden variables to be inferred during planning. Kuefler et al. [26] use adversarial imitation learning to reproduce human driving styles.

It is worth dwelling on the three-way taxonomy that Lefèvre and colleagues [29] use to organize this field, because it clarifies the deliberate modesty of the present approach. Physics based methods predict future motion by propagating a kinematic or dynamic model forward; they are accurate over very short horizons but essentially blind to intention, since they extrapolate what an actor is doing rather than infer what it means to do. Maneuver-based methods, the family to which this project belongs, instead recognize that an actor is executing

one of a discrete set of named maneuvers and reason at that level, trading the fine temporal detail of a predicted trajectory for a categorical judgement that is directly actionable and easily audited. Interaction aware methods go furthest, modelling how actors influence one another’s decisions, at the cost of considerably greater complexity and a much larger appetite for data. Each rung of this ladder purchases expressiveness with complexity and opacity. The choice to remain on the maneuver based rung is, in effect, a wager that a categorical safe or risky judgement made transparently is worth more in a safety setting than a richer but less scrutable prediction a wager the interpretability argument of Rudin [44] directly supports, and one the robustness results of Chapter 5 vindicate, since it is only because the model is legible that its failure under sensing can be attributed to named features rather than left as an unexplained drop.

The choice of a maneuver-based, interpretable model carries a further, less obvious consequence for how such a system must be validated. A trajectory-prediction network is normally judged by a displacement error averaged over a benchmark, a single aggregate that reveals little about which situations it handles poorly. A discrete behavior classifier, by contrast, invites and this project insists upon a confusion matrix, a feature-importance ranking, and a leakage audit, each of which exposes a specific and interpretable facet of the model’s competence. The interpretability that motivates the model choice thus also shapes the evaluation methodology, steering it away from a single headline number and towards the layered, adversarial scrutiny that the remainder of this review argues the field too often omits. This project positions itself deliberately at the interpretable end of that spectrum. It reduces each trajectory to ten transparent motion features speeds, accelerations, lateral and yaw rates, and distances to the ego vehicle and classifies them with a decision tree whose logic can be read and defended. The cost of that choice is the inability to model long-range interaction or to exploit raw temporal structure; the benefit is that every prediction can be explained, every failure can be diagnosed, and, as Chapter 5 shows, the specific features responsible for the sensor-induced accuracy drop can be named. In a safety-critical setting this transparency is not a minor convenience but a substantive advantage.

2.1.4 Data Leakage and Evaluation Rigor

The fourth body of work is methodological, and it is central to the honesty of this project’s results. Data leakage the inadvertent presence, among a model’s inputs, of information that would not be available at prediction time or that trivially encodes the target is one of the most common and most damaging errors in applied machine learning. Kaufman et al. [21] formalized the problem and catalogued its many guises, and more recently Kapoor and Narayanan [22] documented a reproducibility crisis across dozens of scientific fields traceable directly to leakage, in which impressive published accuracies collapsed once the leak was closed.

The classic symptom of leakage is an accuracy that is suspiciously high, and the classic remedy is a deliberately adversarial test: perturb or remove the suspect information and observe whether performance falls to chance. This project applies exactly that discipline. An

early classifier reached almost perfect accuracy; rather than celebrate, the investigation shuffled the metadata columns while leaving the genuine motion features intact, watched accuracy collapse, and concluded that the model had been reading the behavior name off a label string rather than learning motion. The offending columns were removed and the model retrained. The episode is described in full in Chapter 5, and it exemplifies the standard set by Hastie, Tibshirani, and Friedman [20] and by the leakage literature: a high number means nothing until you have tried and failed to break it.

Leakage is not a single mistake but a family of them, and naming the members that bear on this project sharpens the account of Chapter 5. Feature leakage occurs when a predictor encodes the target more or less directly, as a free text description naming the maneuver would; it is the variety the leakage audit here uncovers and removes. Group, or dependence, leakage occurs when related observations are divided across the training and test sets, so that a model can score well on the test set by having effectively memorized a near identical training example; the scenario level split adopted in this work is a direct guard against it, since actors that share a scenario are never split across the boundary. Temporal leakage occurs when information from the future contaminates a judgement that is nominally about the past, a hazard any sliding-window classifier such as the live demonstration must respect. Kapoor and Narayanan [22] catalogue these and further variants across the reproducibility failures they survey, and the recurring lesson is that leakage is seldom a matter of carelessness with the test set alone; far more often it is built into how the features and the splits were constructed, which is precisely why it must be hunted deliberately rather than presumed absent.

There is a cultural dimension to this as well, and it motivates the stance the project takes. In much applied work a high accuracy is an occasion for celebration and publication, not suspicion, and the incentive structure quietly discourages the very adversarial testing that would expose a leak. The methodology literature [21], [22] can be read as an argument that this incentive is exactly backwards for safety-critical systems, in which an inflated accuracy is not a harmless overstatement but a false assurance that can propagate all the way to a deployed vehicle. This project therefore treats its own most flattering result as a hypothesis to be attacked rather than a finding to be reported, and it is that inversion of the usual reflex attacks the good news first that the leakage investigation of Chapter 5 puts into practice.

Related to leakage is the question of how train and test sets are separated. Because actors in the same driving scenario share a road layout, a traffic mix, and a random seed, splitting them at the level of individual actors would let near identical situations appear in both training and test, inflating accuracy. This project therefore splits at the scenario level, so that every actor from a given scenario lands entirely in one set a precaution analogous to grouping by patient in medical machine learning, and one that the leakage literature [21], [22] identifies as essential. Standard evaluation metrics accuracy, precision, recall, and the confusion matrix are used throughout, following the definitions consolidated by Powers [37].

2.1.5 Surrogate Safety Metrics

Underlying every behavior label in this project is a prior question that the traffic-safety community has studied for half a century: what, measured from motion alone, makes a situation dangerous? The answer that field has converged on is the surrogate safety metric a quantity computed from the relative motion of two road users that stands in for the rare and hard to observe event of an actual collision. The oldest and most influential of these is time to collision, introduced by Hayward [30], who proposed a continuous “scale of danger” defined by the time that would remain before two vehicles on a converging course collided if neither changed its motion. In its simplest longitudinal form, time to collision is the range between two vehicles divided by their closing speed, and it becomes small precisely in the situations a driver would call risky: high speed, short following distance, and rapid closure.

A substantial literature has since refined and compared these indicators. Vogel [31] contrasted headway the time gap to the vehicle ahead with time to collision, showing that the two capture different facets of risk and that neither alone is sufficient, and the survey of Lefèvre, Vasquez, and Laugier [29] situates such metrics within the broader problem of risk assessment for intelligent vehicles. The common thread is that danger can be quantified from kinematics: relative distance, relative speed, and their rates of change. This is the same physical vocabulary from which the present project’s motion features are drawn closest approach and mean distance to the ego vehicle, peak deceleration, and the variability of speed and heading so that the classifier trained in later chapters can be read as learning a data driven, multi feature surrogate safety metric rather than a single hand tuned threshold. The explicit time to collision and risk score formulations that make this connection precise are set out in Appendix A, where they complement, rather than replace, the learned classifier.

Time to collision is only the best-known member of a broader family, and the others are worth naming because each addresses a blind spot of the last. Post-encroachment time measures the interval between one road user leaving a shared conflict point and another arriving at it, capturing the crossing and turning conflicts that a purely longitudinal measure overlook. The deceleration rate required to avoid a crash expresses danger instead as the braking effort a situation demands, so that a conflict resolvable only by very hard braking is flagged even when its nominal time margin still looks comfortable. What unites these indicators is that each condenses relative motion into a single scalar with a transparent physical meaning; what separates them is that each carries a characteristic weakness a scenario it silently rates as safe. That pattern is itself an argument for the multi-feature, learned approach adopted here. Rather than commit to one hand-chosen surrogate and inherit its particular blind spot, the classifier is given several complementary motion features and left to combine them, so that its decision boundary behaves like a data-driven blend of these classical metrics rather than a reproduction of any single one.

Two properties of surrogate metrics matter for the design that follows. First, they are interpretable by construction: a small time-to-collision has an unambiguous physical meaning, which aligns with this project’s preference for a transparent classifier over a black

box. Second, and less comfortably, every surrogate metric depends on an accurate estimate of relative position and velocity exactly the quantities that a radar and tracker pipeline estimates imperfectly. A metric that is well defined on ground truth motion can be badly distorted when its inputs are fragmentary tracks, which foreshadows the robustness problem that occupies Chapter 5 and reinforces why that problem must be measured rather than assumed away.

2.1.6 Interpretable Machine Learning and Decision Trees

The classifier at the center of this project is a decision tree, and the choice rests on a body of work stretching back four decades. Quinlan’s ID3 [18] and the classification and regression tree framework of Breiman, Friedman, Olshen, and Stone [17] established recursive partitioning as a way of learning a predictive rule that is, uniquely among the mainstream methods, directly legible to a human. A decision tree divides the feature space with a sequence of simple threshold tests is the minimum distance to the ego below some value, is the peak deceleration above another and arranges those tests into a branching structure whose every leaf corresponds to a plain conjunction of conditions. To classify an actor is simply to follow the branch its features select and read off the label at the end; to audit the classifier is to read the tree.

The learning procedure that builds such a tree is greedy and local. Beginning with the whole training set at the root, the algorithm considers every feature and every candidate threshold and selects the split that most reduces an impurity measure typically the Gini index or the entropy of the two resulting subsets, so that each child is purer in its class composition than its parent. It then recurses on each child, and continues until the leaves are pure or a stopping rule intervenes. The consolidated treatment of these ideas by Hastie, Tibshirani, and Friedman [20] makes clear both their appeal and their principal hazard: an unconstrained tree will keep splitting until it has memorized its training data, achieving a perfect training accuracy that does not generalize. The remedy is to limit the tree’s growth or to prune it back, and to judge the result by cross validation rather than by training accuracy precisely the discipline this project applies when it reports a cross-validated figure alongside the held out one. The MATLAB “fine tree” used here is a tree permitted many splits, and therefore a detailed decision boundary, whose tendency to overfit is checked by exactly this cross validated evaluation.

A single tree is not, on raw accuracy, the strongest classifier available. Ensemble methods that average many trees the random forest of Breiman [19] foremost among them almost always predict more accurately, because averaging cancels the high variance to which any one greedy tree is prone. But an ensemble buys that accuracy by sacrificing the very property that makes a single tree valuable here: a forest of hundreds of trees is no longer a rule a human can read, only a black box that happens to be made of legible parts. Rudin [44] argues forcefully that in high-stakes settings this trade is usually the wrong one, and that an interpretable model should be preferred to a post-hoc explanation of an opaque one wherever the interpretable model is good enough. This project takes exactly that position. It accepts a

little less accuracy, and a little more variance, in exchange for a classifier whose decisions can be inspected, defended, and crucially for the robustness study diagnosed when they fail.

That interpretability is not merely an aesthetic preference; it is load-bearing for the argument of this report. Because a decision tree splits on one feature at a time, its reliance on each feature can be quantified directly, and the effect of corrupting a feature can be traced to the branches that test it. When Chapter 5 shows that the classifier’s accuracy collapses under realistic sensing, it is able to attribute the collapse to specific, named features the duration-dependent statistics precisely because the model exposes which features its decisions turn on. An equally accurate neural network would have failed in the same way for the same underlying reason, but would have offered no comparable purchase on why. The axis-aligned, threshold-based structure of the tree also sits naturally alongside the surrogate-safety-metric tradition of the previous subsection: a split on the minimum distance to the ego is, in effect, a learned proximity threshold, and the trained tree can be read as a data-driven combination of exactly the kind of hand-set thresholds that traffic engineers have long used to define danger. The methods reviewed here recursive partitioning [17], [18], the bias variance reasoning behind ensembles [19], [20], and the argument for interpretability [44] together justify the modelling choice on which the rest of the report depends. The scikit-learn library [38] provides the same family of estimators in the Python ecosystem, and the MATLAB Statistics and Machine Learning Toolbox [34] the implementation actually used here.

2.2 Previous Research Studies

Beyond the thematic strands above, it is useful to consider how complete systems in the literature have combined perception, tracking, and behavior reasoning, and where their evaluations have stopped short. Three patterns recur.

The first pattern is perception and tracking systems evaluated on detection or tracking metrics alone. Multi-sensor fusion systems such as that of Cho et al. [36] demonstrate robust moving object detection and tracking in urban driving, and are evaluated on how accurately they localize and follow objects. These works are the essential substrate for behavior understanding, but they stop at the tracking layer; whether the resulting tracks are good enough to support a downstream behavior classifier is left as an open question precisely the question this project takes up.

The second pattern is behaviors and trajectory-prediction models evaluated on curated, largely complete trajectories. The social-pooling and generative-adversarial trajectory models [23], [24], [25] report displacement errors on benchmark datasets in which each agent is observed cleanly for a fixed window. Their results are impressive, but the evaluation implicitly assumes that a near-complete observation of each agent is available. In a radar-only setting with a shallow field of view, that assumption fails: many actors are seen only briefly and intermittently. A model’s benchmark accuracy therefore may not transfer to a realistic perception stack, and few studies quantify the size of that gap directly.

The third pattern is the simulation-based study. Simulators such as CARLA [35] and, in the MATLAB ecosystem, the Automated Driving Toolbox [32] have made it possible to generate large, perfectly labelled scenario sets and to attach synthetic sensors with configurable noise. This is the tradition the present project works within. The advantage perfect ground truth is exactly what makes a rigorous robustness comparison possible: one can compute a feature from the true trajectory and from the sensor-tracked trajectory of the same actor and measure the discrepancy directly. The corresponding limitation, namely the reality gap between simulated and physical sensors, is acknowledged in Chapter 6. What distinguishes this project from a typical simulation study is that it does not merely generate data and train a model; it uses the perfect ground truth as a measuring instrument to expose and explain a failure that a real-world study would struggle to isolate.

Within that simulation tradition, a smaller body of work has begun to probe robustness in the sense this project means it the sensitivity of a downstream decision to upstream perception error but characteristically one layer at a time. Studies of tracker robustness perturb detections and report the effect on tracking metrics; studies of planner robustness inject localization error and report the effect on the planned path. What remains comparatively rare is a study that carries a single, well-defined perturbation here, the replacement of ground truth trajectories by radar-tracked ones all the way from the sensor through to a semantic behaviors decision, and measures the end-to-end effect while still holding the upstream ground truth in hand for comparison. The Automated Driving Toolbox [32] makes such an end-to-end study unusually tractable, because the one environment that generates the ground-truth scenario also supplies the synthetic radar and the tracker, so that no data need be exported, re-annotated, or re-aligned as it passes between layers. This project exploits that integration deliberately, and it is the reason the ground truth and tracked feature vectors compared in Chapter 5 can be guaranteed to describe the very same actors in the very same scenarios.

Taken together, these three patterns reveal a consistent blind spot. The tracking literature verifies that objects can be followed; the behaviors literature verifies that clean trajectories can be classified; but comparatively little work closes the loop by asking how much behavior-classification accuracy is lost specifically because the tracker’s output is partial and fragmented rather than clean. This project is organized around that question.

2.3 Comparison with Existing Systems

Table 2.2 places the present system alongside the three families of prior work discussed above, across the dimensions that matter most for a safety critical behavior classifier: the data source and whether ground truth is available, the sensing modelled, the tracker, whether data leakage is explicitly tested, whether robustness to realistic sensing is measured, and whether the model’s decisions are interpretable.

Table 2.2: The present system compared with representative families of existing work. “GT” denotes ground truth.

Dimension	Deep trajectory prediction	Fusion detection & tracking	This project
Data source	Real benchmarks	Real / on-vehicle	Simulation, perfect GT
Behavior label	Trajectory (regression)	None (tracks only)	Safe / risky (binary)
Sensing modelled	Post-perception	Multi-sensor	Single radar (by design)
Tracker	Not central	Advanced fusion	CV-EKF multi-object
Leakage tested	Rarely reported	Not applicable	Yes, explicitly
Robustness to sensing	Seldom isolated	Detection-level only	Measured end-to-end
Interpretability	Low (black box)	Moderate	High (decision tree)

The comparison is not meant to imply superiority on every axis the deep models capture interaction and long horizon structure that a decision tree cannot, and a production fusion stack sees far more of the world than a single radar. The point is that the present system occupies a cell of the design space that the others largely leave empty: an interpretable behavior classifier whose robustness to the partial, fragmented output of a realistic tracker is measured directly against perfect ground truth, with data leakage explicitly ruled out. That combination is the project’s distinctive contribution.

2.4 Research Gap

The literature reviewed in this chapter supports a single, focused observation. Each layer of the autonomous driving stack has been studied deeply in isolation: detection and tracking are mature and well benchmarked [11],[16], [43]; behavior and trajectory prediction are advancing rapidly [23],[29]; and the methodology community has issued clear warnings about leakage and evaluation rigor [21], [22]. What is comparatively scarce is work that joins these threads by asking, quantitatively and with interpretable methods, how much a behavior classifier’s accuracy degrades when its inputs come not from clean ground truth but from a realistic radar-and-tracker pipeline, and why.

This gap has three facets. First, robustness to imperfect perception is usually reported, if at all, at the detection or tracking layer, not propagated up to the semantic behavior decision that actually drives the vehicle’s response. Second, the specific mechanism of degradation partial observation and track fragmentation shortening the window over which duration-dependent features are computed is rarely isolated, because doing so requires the perfect ground truth that only simulation cheaply provides. Third, the evaluation discipline that the methodology

literature demands are unevenly applied; leakage is seldom explicitly tested for, so some reported accuracies may not reflect genuine learning.

This project is designed to fill precisely that gap. It builds an interpretable behavior classifier, subjects it to an explicit leakage audit, and then its central contribution measures end to end how much accuracy is lost when the same classifier is fed radar-tracked rather than ground-truth trajectories, tracing the loss to a named and quantified cause. The chapters that follow describe how this was done and what was found.

CHAPTER 3: METHODOLOGY

This chapter describes how the system was designed and built, moving from a bird’s eye view of the whole pipeline down to the individual algorithms and data sources that compose it. The account is deliberately concrete: wherever a design decision was taken, the reasoning behind it is given, and wherever a component was configured, the actual settings used are reported so that the work could be reproduced. The chapter is organized in four parts. Section 3.1 gives a system overview and states the guiding design principles. Section 3.2 decomposes the system into its architectural blocks and traces the flow of data between them. Section 3.3 presents the classification algorithm and its flowchart. Section 3.4 describes how the data was collected and which tools and technologies were used.

3.1 System Overview

At the highest level the system is a single pipeline that begins with a synthetic driving world and ends with a real-time driving decision. A scenario generator produces hundreds of richly varied driving situations, each populated with actors whose behavior is drawn from a labelled library. From each actor’s trajectory a compact set of motion features is extracted, and a classifier is trained to map those features to a safe or risky label. The same features are then re-derived from the output of a simulated radar and multi-object tracker, so that the classifier can be evaluated not on the perfect trajectories it was trained on but on the degraded trajectories a real vehicle would actually see. Finally, the trained classifier is embedded in a live closed-loop simulation where its predictions influence the ego vehicle’s behavior. Figure 3.1 shows this end-to-end flow.

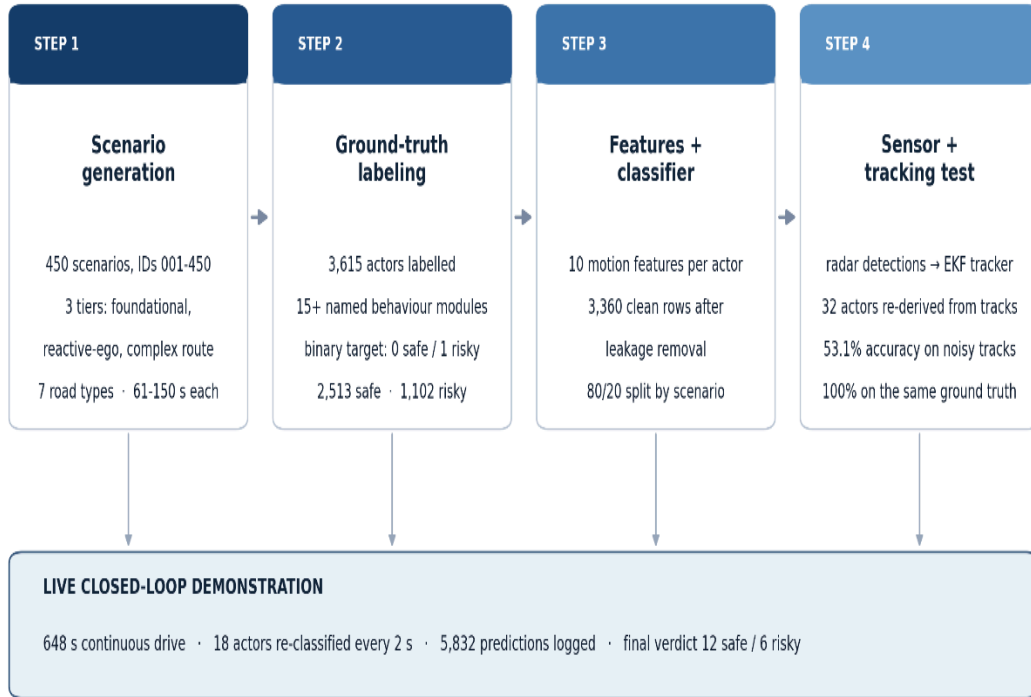


Figure 3.1: End to end system architecture: from programmatic scenario generation and ground truth labelling, through feature extraction and classifier training, to the radar and tracker robustness evaluation and the live closed-loop demonstration.

Three design principles shaped every decision in this pipeline, and they recur throughout the report. The first is interpretability: at each stage a representation was preferred that a human can inspect and reason about, from the ten named motion features to the decision-tree classifier whose branches can be read as rules. The second is honesty of evaluation: the pipeline was built specifically so that the classifier could be confronted with realistic, degraded inputs and its failures measured rather than hidden. The third is reproducibility: the scenarios are generated from seeded random draws, the train and test sets are split deterministically at the scenario level, and every parameter is recorded, so that the same dataset and the same results can be regenerated on demand.

It is worth emphasizing what the overview implies about the relationship between the training and evaluation halves of the system. The classifier never sees radar-tracked data during training; it learns exclusively from clean, ground truth trajectories. The radar-and-tracker stack is introduced only at evaluation time, as an adversary. This separation is intentional. It mirrors the situation of a model developed on idealized data and then deployed onto a physical vehicle, and it is what allows the robustness experiment of Chapter 5 to answer a precise question: holding the classifier fixed, how much accuracy is lost purely because the inputs are now the product of realistic sensing rather than perfect knowledge?

Figure 3.2 zooms into the perception-to-decision portion of the pipeline, showing how a driving scenario is observed by the radar, how the resulting detections are consolidated into tracks by the multi-object tracker, how features are computed from those tracks, and how the classifier turns them into a per actor behavior label that the control logic can act upon.



Figure 3.2: Perception to decision chain: the driving scenario is sensed by a forward-looking radar, detections are fused into tracks by a constant velocity multi object tracker, ten motion features are computed per track, and the classifier emits a safe/risky label that informs the ego vehicle’s response.

3.2 System Architecture

The pipeline decomposes into five architectural blocks, each implemented as one or more MATLAB scripts and functions with well-defined inputs and outputs. Keeping the blocks loosely coupled communicating through files on disk rather than shared state made it possible to develop, test, and re-run each stage independently, and it is what allows the leakage and robustness experiments to re-enter the pipeline midway without rebuilding everything upstream. The five subsections that follow describe each block in turn, in the order data flows through it.

3.2.1 Scenario Generation and Labelling

The first block programmatically constructs driving scenarios using the Automated Driving Toolbox [32]. A road network builder lays down highways, rural and village roads, signalized crossroads, multi turn routes, and railway level crossings; an actor populator places cars, trucks, buses, bicycles, pedestrians, children, and trains onto those roads; and a behavior library assigns each actor a named maneuver smooth car-following, disciplined lane-keeping, a signaled lane change on the safe side, or tailgating, sudden braking, erratic swerving, and red light running on the risky side. Because the behavior is assigned by the generator, its ground truth label is known exactly, without any manual annotation.

That the labels are generated rather than annotated is a quiet but important guarantee. Human annotation of behavior is notoriously subjective reasonable observers disagree on whether a given lane change was aggressive and any such disagreement would enter the dataset as label

noise, placing a ceiling on the accuracy a classifier could honestly reach. By deriving each label from the very module that produced the motion, the pipeline eliminates that noise at the source: a trajectory generated by the tailgating module is risky by definition, not by opinion. The cost of this cleanliness, acknowledged plainly here and again in Chapter 6, is that the behaviors are only as varied and as realistic as the library that scripts them; the dataset cannot contain a form of risky driving that no module encodes.

3.2.2 Feature Extraction

The second block reduces each actor’s trajectory to a fixed-length vector of ten motion features. Operating on the sequence of time-stamped positions and velocities, it computes summary statistics of speed, acceleration, lateral motion, and proximity to the ego vehicle, as detailed in Section 3.4. Trajectories too short to yield meaningful statistics are discarded rather than padded, a decision whose consequences become important in the robustness analysis, where sensor-tracked trajectories are often far shorter than their ground-truth counterparts.

3.2.3 Classifier Training and Evaluation

The third block trains a fine decision tree on the feature vectors and their labels [17], [18], using the Statistics and Machine Learning Toolbox [34]. This block also houses the data-leakage investigation and the leakage free evaluation. Crucially, the train and test sets are separated at the level of whole scenarios, so that no actor from a scenario used in training can reappear in testing; this prevents the near-duplicate situations that share a scenario from inflating the measured accuracy.

3.2.4 Radar and Multi Object Tracking

The fourth block is the simulated perception stack against which robustness is measured. A forward-looking radar observes each scenario and produces noisy detections; a multi-object tracker consolidates those detections into tracks, each carrying an estimated position and velocity [5], [6]. Because the radar reports in the ego vehicle’s local frame while the ground truth trajectories live in the world frame, a coordinate frame correction is applied so that the two are directly comparable a subtlety examined in Chapter 4.

3.2.5 Closed Loop Demonstration

The fifth block places the trained classifier inside a live simulation loop. At each step the radar senses the scene, the tracker updates its tracks, features are computed over a sliding window, the classifier labels each tracked actor, and a simple control rule adjusts the ego vehicle’s following distance and lane change decisions in response to any actor judged risky. This block demonstrates that the classifier is not merely an offline analysis tool but can operate within the real-time budget of a driving loop.

3.3 Algorithm and Flowchart

The behavior classification algorithm can be stated compactly. For each actor observed over a window of time, gather its trajectory as a sequence of time-stamped positions and velocities; if the trajectory is long enough, compute the ten motion features; present the feature vector to the trained decision tree; and record the predicted label. In the offline evaluation this is done once per actor over its whole trajectory; in the live demonstration it is done repeatedly over a sliding window so that a verdict is available at every instant. Figure 3.3 renders this control flow as a flowchart.

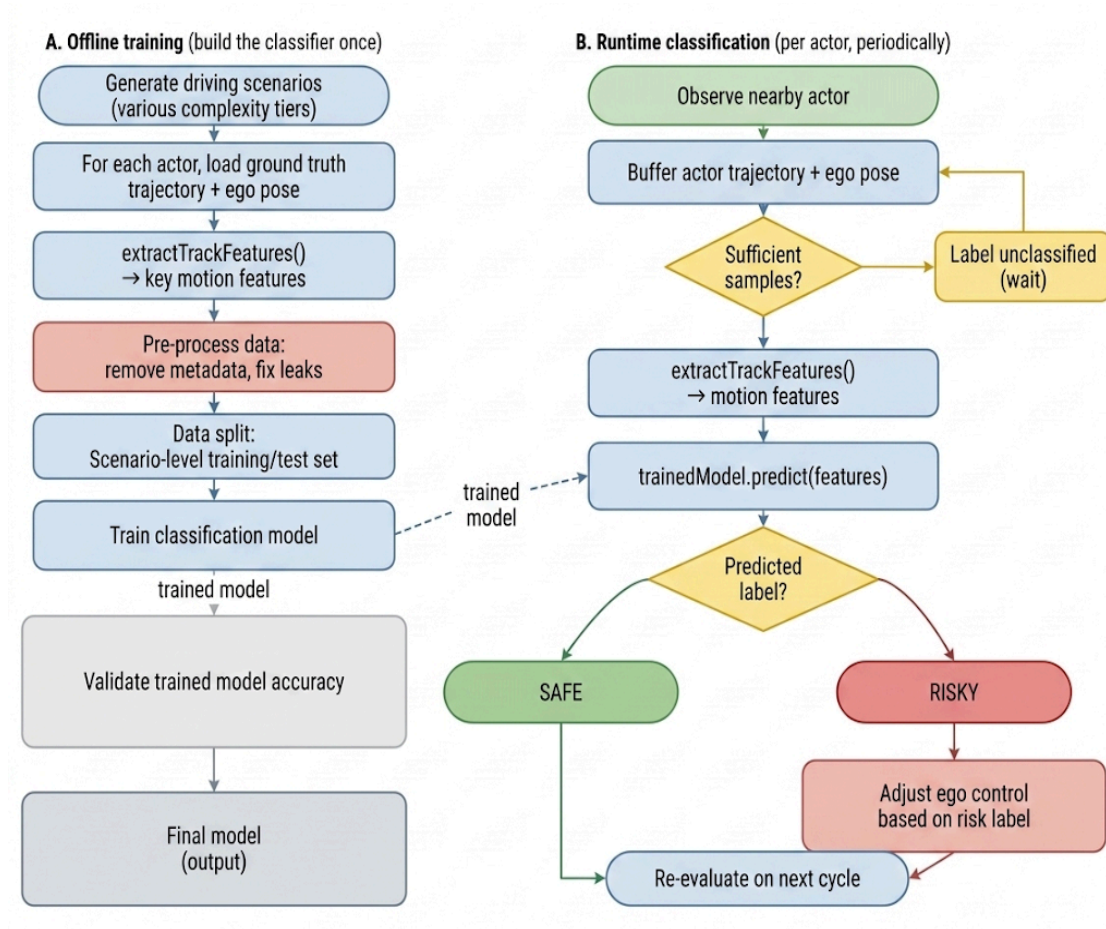


Figure 3.3: Flowchart of the behavior classification algorithm: trajectory acquisition, a minimum length guard, feature computation, decision tree inference, and label output, with the sliding window loop used in the live demonstration.

Two aspects of the algorithm deserve emphasis. The first is the minimum length guard. The feature extractor requires at least four trajectory samples before it will compute features, because statistics such as acceleration and the standard deviation of speed are meaningless on shorter sequences; an actor seen for fewer samples yields no feature vector and is not classified. On ground truth trajectories this guard rarely triggers, but on sensor tracked trajectories it becomes decisive, because the radar’s shallow field of view and the tracker’s fragmentation frequently reduce an actor’s observed trajectory below the threshold.

A third aspect, implicit in the loop drawn on the flowchart, is the distinction between the two modes in which this one algorithm runs. In the offline evaluation each actor is classified exactly once, from features computed over its entire observed trajectory; this is the natural setting for measuring accuracy against a fixed ground-truth label, and it is the setting in which every headline number of Chapter 5 is obtained. In the live demonstration the identical feature and classify step is instead applied repeatedly, over a sliding window that advances with simulated time, so that a provisional verdict exists at every instant and is revised as more of an actor’s motion is seen. The two modes share every line of feature and inference code and differ only in the span of trajectory presented to it. That shared implementation is deliberate: it means the live system inherits exactly the behavior characterized offline, so any discrepancy between them can be ascribed to the windowing itself rather than to a divergence in logic, and the offline accuracy can be read as an upper bound on what the windowed live system could achieve given the same observations.

The second is the motion model underlying the tracker that feeds the live and robustness paths. The tracker assumes constant velocity between updates, so that a track’s state evolves according to a straightforward linear prediction before each new measurement is fused in. Writing the planar state as position and velocity in each axis, the prediction step advances the state over a time step

$$x(t + \Delta t) = x(t) + vx(t) \cdot \Delta t, y(t + \Delta t) = y(t) + vy(t) \cdot \Delta t \quad (3.1)$$

with the velocity components held constant across the step and corrected only when a measurement arrives. This is the well-known constant velocity model [41]; it is a deliberate simplification whose consequences for genuinely non-linear maneuver’s hard braking, sharp turns are examined in the robustness analysis. The full extended Kalman filter equations that surround this prediction, including the measurement update, are collected in Appendix A [4], [7].

3.4 Data Collection, Tools and Technology

Because no existing public dataset pairs the specific ingredients this project needs richly varied driving behaviors, clean safe or risky labels, and full trajectories with perfect ground truth [11], [12], [13] the data was generated programmatically. This subsection describes the scenario generation scheme, the motion features extracted from it, and the software tools used throughout.

3.4.1 Scenario Generation and Dataset Composition

Scenarios were generated in three tiers of increasing complexity, so that the dataset would contain both simple, easily separable situations and dense, ambiguous ones. The tiers, and the complexity each adds, are summarized in Table 3.1. In total more than four hundred and fifty scenarios were produced.

Table 3.1: The three-tier scenario generation scheme and the complexity introduced at each tier.

Tier	Character	Roads and actors	Complexity added
Tier 1	Basic	Single straight roads; a few vehicles	One dominant maneuver per scene; sparse traffic
Tier 2	Intermediate	Signalized crossroads; mixed vehicles and cyclists	Signals, cross-traffic, denser and more varied actors
Tier 3	Complex	Multi-turn routes, village roads, rail crossings; full actor mix incl. pedestrians, children, trains	Ego maneuver's (turns, lane changes); rare high-risk events

The distribution of scenarios across the seven road network types is shown in Figure 3.5, and the composition of the complexity tiers, together with the mix of ego maneuvers exercised in the most complex tier, is shown in Figure 3.6. Together these confirm that the dataset is neither dominated by a single road type nor by a single behavior, which matters because a classifier trained on a lopsided dataset can achieve a high accuracy simply by always predicting the majority class.

The three-tier design is more than an organizational convenience; it is a deliberate spread across difficulty that guards against a subtle form of self-deception. A dataset composed only of easy, unambiguous scenes would let almost any classifier post a high accuracy, flattering the method without ever testing it, while one composed only of dense, adversarial scenes would depress accuracy for reasons unrelated to the sensing question this project exists to answer. By spanning both extremes, the dataset ensures that the leakage free accuracy reported in Chapter 5 reflects performance across a genuine range of difficulty, and that the robustness experiment degrades a classifier that was competent to begin with rather than one that was already struggling.

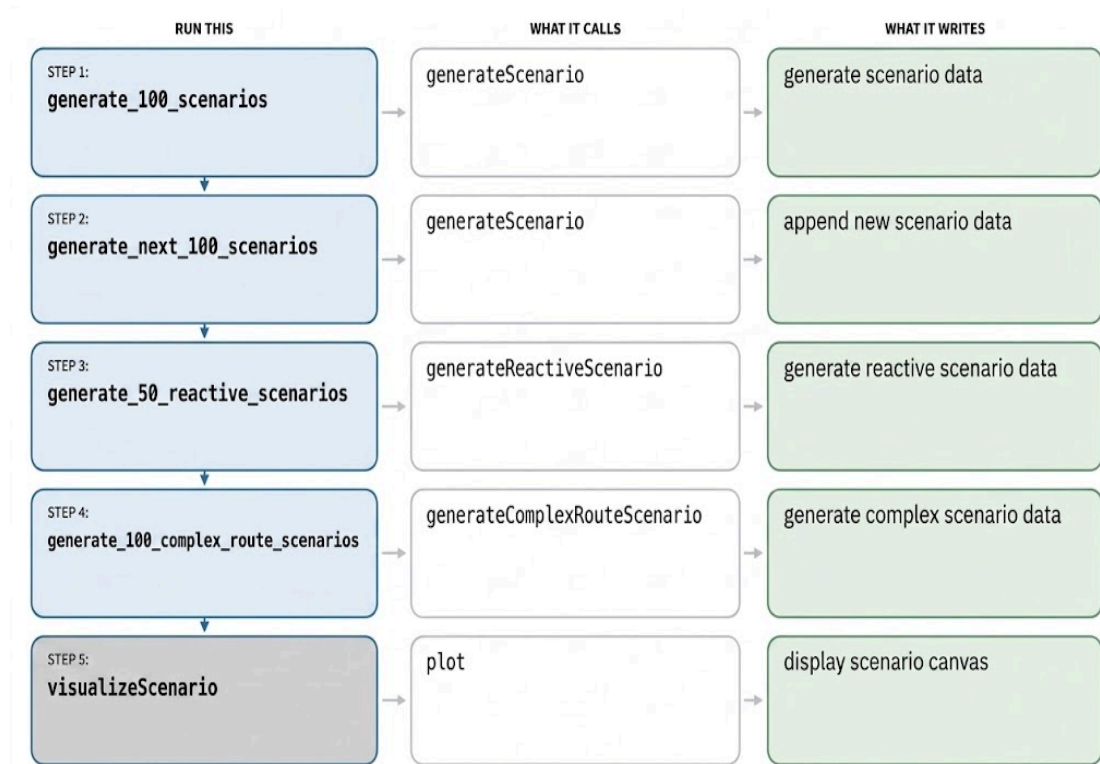


Figure 3.5: Distribution of the generated scenarios across the seven road network types, confirming coverage of highways, rural and village roads, crossroads, multi turn routes, and railway crossings.

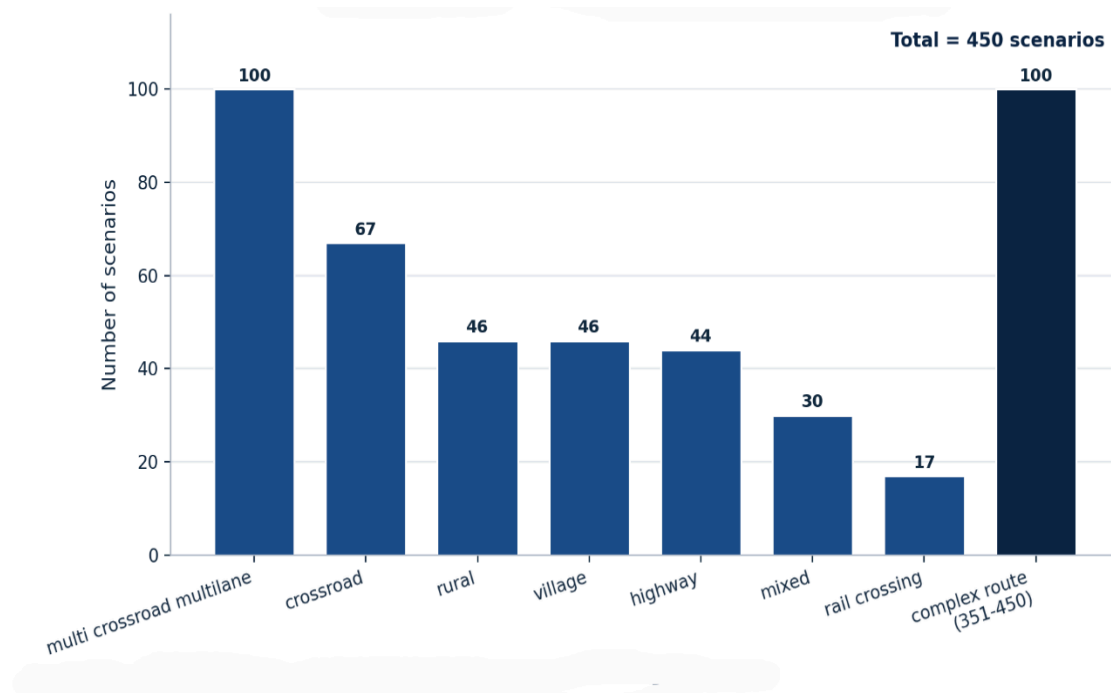


Figure 3.6: Composition of the three complexity tiers and, for the most complex tier, the mix of ego vehicle maneuvers exercised during scenario generation.

Each actor within a scenario was assigned a behavior from the labelled library and therefore carries a ground truth safe or risky label. Figure 3.7 breaks down the labelled actors by type

cars, trucks, buses, bicycles, pedestrians, children, and trains and by label, and Figure 3.8 shows the overall class balance between safe and risky behaviors across the whole dataset.

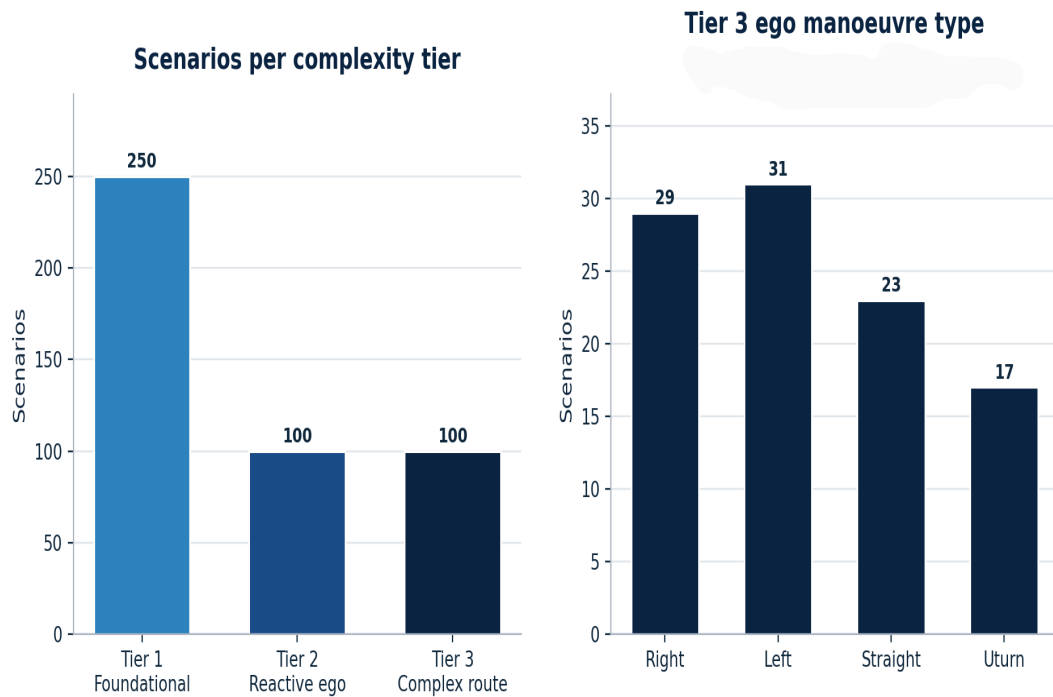


Figure 3.7: Counts of labelled actors by type, each split into its safe and risky portions, showing that both labels are represented across every actor category.

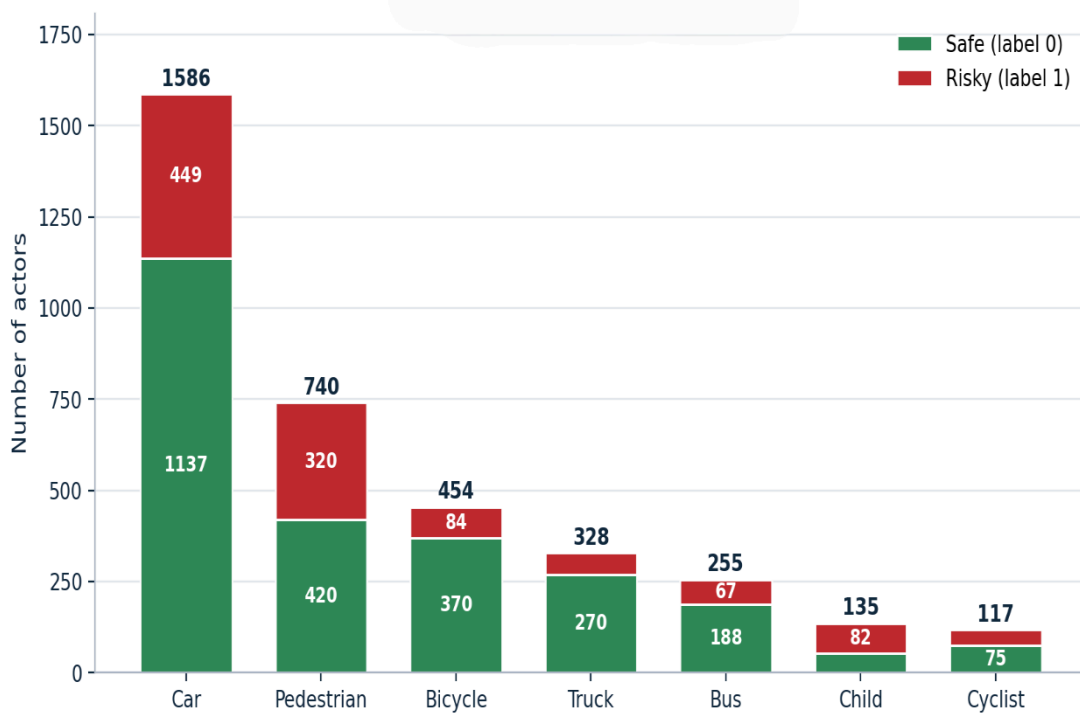


Figure 3.8: Ground truth class balance across the dataset. Both classes are substantially represented, so accuracy cannot be trivially achieved by always predicting the majority class.

3.4.2 The Motion Feature Set

Every actor trajectory is reduced to the same ten motion features, listed in Table 3.2. The features were chosen to capture the physical signatures of risky driving high and variable speed, harsh acceleration and braking, sharp lateral motion, and dangerous proximity to the ego vehicle [29], [31] while remaining simple enough that each can be inspected and understood. No feature encodes the actor’s type, the scenario, or the behavior name; the features describe motion and nothing else. This restriction is what makes the leakage investigation of Chapter 5 meaningful, because it establishes what the classifier is supposed to learn from.

Table 3.2: The ten motion features extracted from every actor trajectory.

No.	Feature	Description	Units
1	avg_speed	Mean speed over the observed trajectory	m/s
2	max_speed	Maximum instantaneous speed	m/s
3	speed_std	Standard deviation of speed (variability)	m/s
4	max_accel	Largest positive acceleration	m/s ²
5	max_decel	Largest deceleration (hardest braking)	m/s ²
6	max_lateral_rate	Peak lateral velocity (sideways motion)	m/s
7	yaw_rate_std	Variability of heading change	rad/s
8	min_dist_to_ego	Closest approach to the ego vehicle	m
9	avg_dist_to_ego	Mean distance to the ego vehicle	m
10	duration	Length of the observed trajectory	s

The tenth feature, duration, requires comment because it plays an outsized role later in the report. On clean ground truth trajectories, duration is a benign and mildly informative feature. On sensor tracked trajectories, however, it is precisely the feature most corrupted by the radar’s limited coverage and the tracker’s fragmentation, since an actor that is truly present for many seconds may be tracked for only a fraction of that time. The robustness analysis of Chapter 5 shows that this corruption of duration, propagating into the other duration dependent statistics, is a principal driver of the accuracy loss under realistic sensing.

It repays the effort to read these ten features not as an arbitrary list but as four physical groups, each targeting a different signature of danger, because the grouping explains both why the classifier succeeds on clean data and why it is fragile under sensing. The first group is the speed group the mean, maximum, and standard deviation of speed. Together these separate an actor travelling steadily and appropriately from one that is either travelling too fast or, more tellingly, varying its speed erratically; a high standard deviation of speed is a compact signature of the stop-start aggression that characterizes tailgating and impatient overtaking, whereas a disciplined driver holds a nearly constant speed and so registers a low value.

The second group captures the sharp transients of motion: the largest positive acceleration, the hardest deceleration, the peak lateral velocity, and the variability of the yaw rate. These are the features by which the most obviously dangerous maneuver's reveal themselves a sudden brake as a large negative acceleration, a swerve or an aggressive cut in as a spike in lateral velocity and a burst of rate variability. What unites this group is that every one of its members is an extreme value or a measure of variability, and extremes and variances are exactly the statistics that a truncated observation estimates most poorly. A hard brake that falls outside the observation window is invisible; a swerve seen only in part looks gentler than it was. This group therefore carries much of the classifier's discriminative power on clean data, as the feature-separation analysis of Chapter 5 confirms, and simultaneously much of its vulnerability once observation becomes partial.

The third group is the proximity group the closest approach and the mean distance to the ego vehicle and it is the most direct link between the learned classifier and the classical surrogate safety metrics reviewed in Chapter 2 [30], [31]. A small minimum distance to the ego is the defining fingerprint of tailgating and of an unsafe cut in, and it stands in for the proximity term of a time to collision calculation without requiring the closing speed to be estimated explicitly. Because these two features are computed from the relative geometry of two trajectories rather than from a single actor's speed profile, they degrade under sensing in a slightly different way from the transient group: they survive as long as the actor is seen at all near its closest approach, but they are lost entirely for the many actors that the radar never observes at close range. The fourth group is the single duration feature already discussed, which acts less as a descriptor of behavior than as an implicit measure of how much evidence the other nine features were computed from a role that turns from harmless to decisive exactly when sensing truncates the trajectory.

Feature computation.

Because one and the same extraction routine is applied to both the clean ground-truth trajectories and the sensor-tracked trajectories, its internal arithmetic determines exactly what the robustness comparison of Chapter 5 measures; it is therefore worth setting out precisely how each quantity is formed from an actor's time-stamped position and speed samples. Time steps are taken as the first differences of the timestamp vector, and any non-positive step which can occur when two tracker updates share a timestamp is replaced by the median positive step, so that no derivative can be corrupted by a division by zero or by a negative interval.

The speed derived features follow directly. Longitudinal acceleration is the first difference of speed divided by the time step,

$$a_k = (v_{k+1} - v_k) / \Delta t_k \quad (3.2)$$

from which the largest positive value gives max_accel and the most negative value gives max_decel, while the mean, standard deviation, and maximum of the raw speed samples give avg_speed, speed_std, and max_speed respectively. Lateral motion is captured as the absolute

first difference of the lateral coordinate over the same step, whose peak value becomes `max_lateral_rate`.

Heading is recovered from the direction of successive position increments, unwrapped to remove the artificial jumps at $\pm\pi$, and differentiated once more to yield the yaw rate,

$$\psi_k = \text{atan2}(\Delta y_k, \Delta x_k), \quad \omega_k = (\psi_{k+1} - \psi_k) / \Delta t_k \quad (3.3)$$

and the standard deviation of this yaw rate becomes `yaw_rate_std`, a compact measure of how erratically an actor is changing direction. When the ego trajectory is supplied interpolated onto the same timestamps — the Euclidean distance to the ego vehicle is formed at every sample,

$$dk = \sqrt{(x_k - X_k)^2 + (y_k - Y_k)^2} \quad (3.4)$$

whose minimum and mean give `min_dist_to_ego` and `avg_dist_to_ego`, and duration is simply the elapsed time between the first and last sample. A minimum length guard rejects any track observed for fewer than four samples, returning a row of missing values that is discarded downstream; this guard is rarely triggered on ground-truth data but fires often on tracked data, where the radar and tracker produce many short, fragmentary tracks a first, quiet indication of the coverage problem that Chapter 5 quantifies in full.

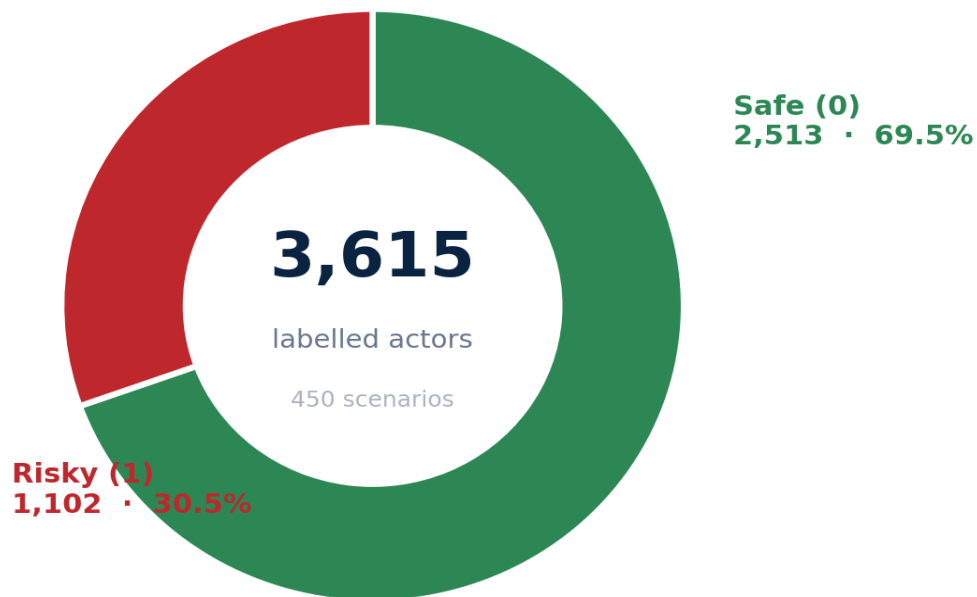


Figure 3.4: The script level build chain for scenario generation and labelling: road network construction, actor population, behavior assignment, and export of labelled trajectories to disk.

3.4.3 Tools and Technology

The entire system was implemented in MATLAB, drawing on three specialized toolboxes whose roles are summarized in Table 3.3. MATLAB was chosen because its Automated

Driving Toolbox provides, in one environment, both the scenario and sensor modelling needed to generate data and the tracking machinery needed to evaluate robustness, while its machine learning toolbox supplies the classifier allowing the whole pipeline to be built without stitching together disparate frameworks.

Table 3.3: The MATLAB toolboxes used in the project and their role in the pipeline.

Toolbox	Principal objects used	Role in the pipeline
Automated Driving Toolbox	driving Scenario, driving Radar Data Generator	Build road networks and actors; generate ground truth; model the forward radar
Sensor Fusion and Tracking Toolbox	Multi Object Tracker, constant velocity EKF	Consolidate radar detections into tracks with estimated position and velocity
Statistics and Machine Learning Toolbox	fitctree (fine decision tree)	Train, cross validate, and evaluate the behavior classifier
Base MATLAB and Simulink	scripting, tables, visualization	Orchestration, feature extraction, CSV datastore, and figures

The radar was configured as a single forward-looking sensor with a wide but shallow field of view of one hundred and forty degrees in azimuth and five degrees in elevation, a maximum range of one hundred and fifty meters, an update rate of ten hertz, and measurement noise enabled, mounted at the front of the ego vehicle. The tracker used a constant-velocity extended Kalman filter with a nearest neighbor assignment threshold [6], [39], requiring two confirming detections out of three to establish a track and deleting a track after five consecutive misses. The exact numerical values of every parameter are collected in Appendix B, and the experimental configuration is depicted in Figure 5.1 of the results chapter. These settings were held fixed across all experiments so that the only variable in the robustness study is the data itself.

3.5 Evaluation Protocol and Experimental Controls

A classifier’s reported accuracy is only ever as trustworthy as the protocol that produced it. The single most important methodological lesson of this project that an early, near-perfect result proved to be an artefact of how the model was evaluated rather than a property of the model itself makes the evaluation protocol worth setting out carefully in its own right, before any performance figure is quoted. Three decisions define that protocol: how the data is divided into training and test portions, how that division is validated against the accident of a single lucky split, and what deliberately contrived control conditions are run alongside the honest test in order to catch the specific ways such a result can be fooled. The protocol was designed, from the outset, to be capable of exposing its own errors, and it is described here so that the results of Chapter 5 can be read against the method that generated them.

3.5.1 Scenario Level Data Splitting

The dataset is partitioned into a training portion and a test portion in an eighty to twenty ratio, but the level at which the partition is made matters far more than the ratio, and it is central to the integrity of every number reported later. The split is made over whole scenarios, not over individual feature rows. The reason is that actors drawn from the same scenario are anything but independent of one another: they share the same road network, the same ego trajectory, the same traffic-signal timing, and frequently mirrored or complementary maneuvers, so two feature rows originating in a single scenario resemble each other far more than two rows drawn at random. A naive row level split would scatter an actor near twins across both sides of the divide, allowing the classifier to see, during training, rows almost identical to those on which it is later tested, and its apparent accuracy would be inflated by what is effectively memorization rather than generalization [21], [22]. Splitting by scenario keeps every actor belonging to a given scenario entirely on one side of the divide, so that the test set genuinely measures how well the classifier generalizes to situations it has never encountered.

Reproducibility is enforced by fixing the pseudo random seed to a single known value for the whole pipeline, so that scenario generation, the train test partition, the cross-validation folds, and the classifier’s own randomized steps all unfold identically on every run. Any result quoted in this report can therefore be reproduced exactly by re-executing the pipeline, and any discrepancy between two runs can be attributed to a genuine change in the code or configuration rather than to the luck of the draw. Determinism of this kind is a precondition for the leakage investigation of Chapter 5, because it is only against a perfectly repeatable baseline that the effect of removing a suspect feature can be measured cleanly.

3.5.2 Cross Validation on the Training Partition

A single train test split, however principled, can still be fortunate or unfortunate in the particular scenarios that happen to fall on either side. To guard against reading too much into one partition, the classifier is additionally assessed by k-fold cross validation on the training portion [20]. The training scenarios are divided into k equally sized folds; the model is trained on all but one-fold and evaluated on the fold left out; and the procedure rotates through the folds so that each is used exactly once for evaluation and k–1 times for training. The folds are formed at the level of whole scenarios, for exactly the same reason the train test split is, so that the cross-validation cannot itself become a source of leakage. The accuracy reported by cross validation is the mean over the folds, a more stable estimate than any single split can provide, and its close agreement with the accuracy on the untouched held out test set is what gives confidence that the headline performance reflects the classifier’s genuine behavior rather than the good fortune of one convenient division of the data.

3.5.3 Control Experiments by Design

Beyond the honest held-out test, the protocol deliberately incorporates a set of adversarial control conditions whose purpose is not to flatter the classifier but to attempt to break the

interpretation of its score. The first is a leakage probe [21], [22]. The same evaluation is run with an identifier-like field deliberately left in the feature set, so that if the classifier is able to reach near perfect accuracy by keying on a quantity that merely stands proxy for the label rather than on any property of the actor’s motion the protocol will surface that fact rather than bury it. It was precisely this probe that revealed the inflated early result described in Chapter 5, turning a silent flaw into a measured and correctable one.

The second control is a shuffled-metadata experiment [19], [38], in which non behavioral metadata associated with each actor is randomly permuted across the population before classification. A classifier that genuinely relies on motion features should be almost unaffected by this permutation, since the information it depends on is left intact; a classifier that has come to lean on metadata will collapse toward the chance level once that metadata is scrambled. The size of the gap between the ordinary and the shuffled conditions therefore acts as a direct diagnostic of how much of the reported score is truly behavioral and how much is an accident of the bookkeeping. The third control is the robustness contrast that forms the empirical heart of the project: the identical feature-extraction and classification procedure is applied first to complete ground-truth trajectories and then to the fragmentary trajectories emitted by the radar-and-tracker stack, with every other variable held fixed, so that the difference between the two accuracies isolates the effect of realistic sensing and nothing else. The numerical outcomes of running all three controls are presented and interpreted in Chapter 5; the point to be carried forward from the methodology is that each was built into the design in advance, so that the eventual results could be trusted precisely because the protocol had actively tried, and failed, to discredit them.

CHAPTER 4: IMPLEMENTATION

Where the previous chapter described the methodology in principle, this chapter describes how it was realized in code. It walks through the library of behavior modules that give each actor its labelled maneuver, the software build chain that turns those modules into a trained classifier, the datastore in which scenarios and features are persisted, and the core code logic at the heart of the pipeline including the coordinate-frame correction on which the entire robustness comparison depends. Throughout, the actual MATLAB code is shown, so that the description can be checked against the implementation rather than taken on trust; the complete listings appear in Appendix D.

4.1 Module Description

The behavioral richness of the dataset comes from a library of behavior modules, each of which scripts a single named maneuver and carries a fixed ground truth label. When a scenario is generated, each non ego actor is assigned one of these modules; the module determines how the actor moves, and its label becomes that actor's ground truth. Because the label is a property of the module rather than an after the fact judgement, it is exact and consistent across the entire dataset. Figure 4.1 shows the library, color coded by label.

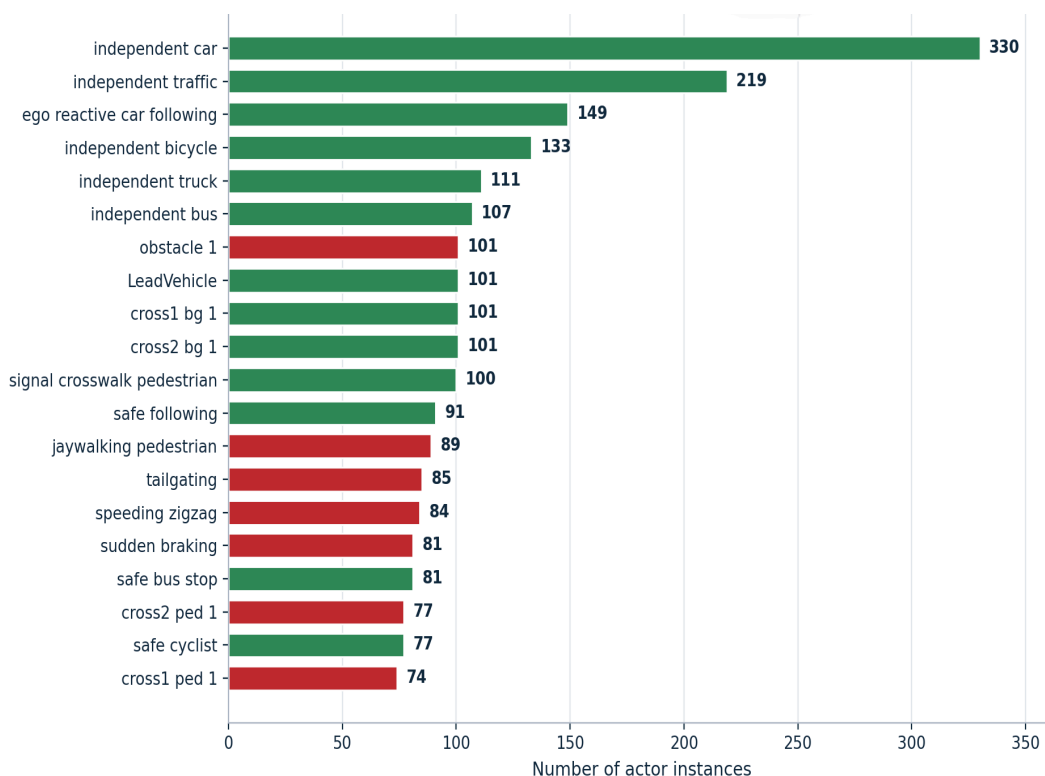


Figure 4.1: The behavior module library, color coded by ground truth label. Safe modules encode disciplined, predictable motion; risky modules encode the aggressive or careless maneuver's that a behavior classifier must learn to flag.

The modules divide cleanly into two families. The safe family encodes the disciplined, predictable motion of a careful road user: smooth car following that maintains a comfortable gap, steady lane keeping, gentle and correctly signaled lane changes, and lawful stopping at signals and crossings. The risky family encodes the motions that a safety system exists to catch: tailgating at an unsafe gap, sudden harsh braking, erratic swerving across a lane, cutting in too sharply, and running a red light or a level crossing. Table 4.1 lists a representative selection from each family, together with the motion signature that distinguishes it and the label it carries.

Table 4.1: A representative selection of behavior modules from the library, with their characteristic motion signature and ground truth label.

Module	Family	Characteristic motion signature	Label
Smooth car following	Safe	Steady speed, large gap to lead vehicle	Safe
Lanekeeping	Safe	Low lateral rate, low yaw-rate variability	Safe
Signaled lane change	Safe	Gradual lateral shift, moderate rate	Safe
Lawful stop at signal	Safe	Gentle deceleration to rest	Safe
Tailgating	Risky	Small min-distance to ego, high speed	Risky
Sudden braking	Risky	Large peak deceleration	Risky
Erratic swerving	Risky	High lateral rate, high yaw-rate variability	Risky
Red light / crossing run	Risky	No deceleration at stop line	Risky

The correspondence between the right-hand column of this table and the motion features of Table 3.2 is deliberate and important. Tailgating manifests as a small `min_dist_to_ego`; sudden braking as a large magnitude of `max_decel`; swerving as a high `max_lateral_rate` and `yaw_rate_std`. The features were designed so that each risky module would leave a detectable fingerprint in at least one of them, and Chapter 5 confirms, through the feature importance analysis, that the classifier does indeed rely on exactly these features rather than on any incidental artefact. This is the concrete sense in which the classifier is intended to learn motion rather than memory’s identity.

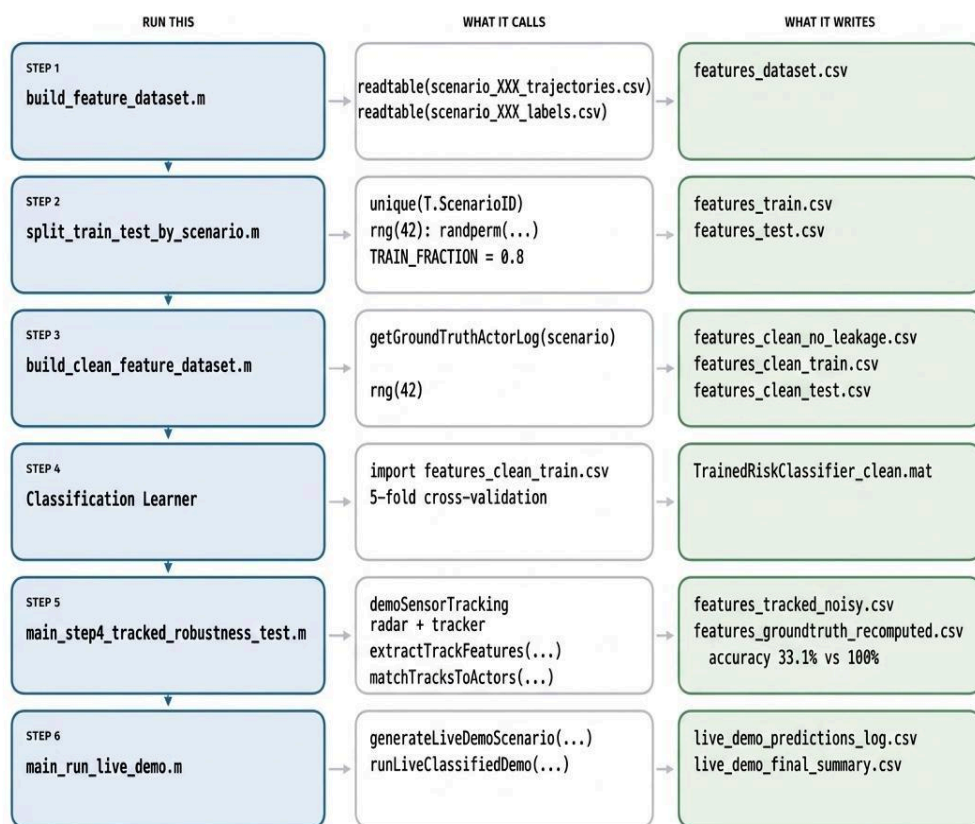
One further property of the library deserves mention, because it bears directly on the accuracy ceiling of the whole system. The boundary between safe and risky is not always crisp even in the generator’s own terms: a signaled lane change executed a little too briskly and an unsingable one executed with unusual caution can produce motion that overlaps in feature space, and a firm but timely braking maneuver shades into a sudden one by degrees rather than at a hard line. This overlap is intentional rather than a defect. A library in which every safe and every risky trajectory were trivially separable would train a classifier that looked flawless and taught nothing, and such a classifier would collapse the moment it met the genuine ambiguity of real driving. By allowing the two families to abut, and occasionally to overlap, the dataset ensures that the leakage-free accuracy reported in Chapter 5 is

bounded below a perfect hundred per cent for an honest reason the classes really do touch rather than for a spurious one, and it means the residual errors on the confusion matrix concentrate exactly where a human would also hesitate.

4.2 Software Implementation

The software is organized as a linear build chain in which each script consumes the output of the previous one and produces a file on disk for the next. This file passing discipline, rather than a monolithic program holding everything in memory, is what makes the pipeline reproducible and re-enterable: any stage can be re-run in isolation given the outputs of the stage before it. Figure 4.2 shows the build chain that leads to the trained classifier.

Figure 4.2: The software implementation build chain, from scenario generation through trajectory



logging, feature extraction, and the scenario level train/test split, to classifier training and evaluation. Each stage writes a file that the next stage reads.

The chain proceeds in six stages. First, generation scripts build the scenarios and save each as a MATLAB scenario file alongside a comma separated file of its actors' ground truth labels. Second, each scenario is replayed and every actor's time stamped position, velocity, speed, and heading is logged. Third, the feature extractor reduces each actor's log to the ten-element feature vector, appending the actor's label, and writes one combined feature file for the whole dataset. Fourth, that feature file is split into training and test sets at the scenario level. Fifth, a fine decision tree is trained on the training set, with cross validation used to guard against

over-fitting. Sixth, the trained model is evaluated on the held-out test set that it has never seen. The leakage investigation of Chapter 5 sits between the fourth and fifth stages, because it is precisely the choice of which columns to feed the trainer that determined whether the model learned or cheated.

The robustness experiment adds a parallel branch that rejoins the main chain at evaluation: the same scenarios are passed through the radar and tracker, features are re-derived from the tracked trajectories, each track is matched back to its ground truth actor to recover a label, and the trained classifier is then scored on these degraded rows. Table 4.3 names the principal script at each stage and the file it writes, so that the whole pipeline both the training chain and the robustness branch can be followed, and re-entered, one artefact at a time.

Table 4.3: The build chain by stage: the principal MATLAB script driving each stage and the artefact it writes to disk. The lower block is the robustness branch that rejoins the chain at evaluation.

Stage	Principal script(s)	Output written
Generate scenarios	main_generate_100_scenarios (+ 4 batch drivers)	scenario_NNN.mat, scenario_NNN_labels.csv
Log trajectories	runAndRecordScenario, getGroundTruthActorLog	Per-actor time-stamped motion logs
Extract features	build_feature_dataset → extractTrackFeatures	features_dataset.csv
Remove leakage	build_clean_feature_dataset	Clean table: 10 features + label
Split by scenario	split_train_test_by_scenario	features_train.csv, features_test.csv
Train & evaluate	fitctree (fine decision tree)	Trained model, test-set accuracy
Sense & track	demoSensorTracking, buildTrackedFeatureDataset	Tracked feature rows
Match & re-label	matchTracksToActors	features_tracked_noisy.csv
Robustness test	main_step4_tracked_robustness_test	Ground-truth vs tracked accuracy
Live demonstration	main_run_live_demo → runLiveClassifiedDemo	live_demo_predictions_log.csv

A representative excerpt from the build chain is the scenario-level split, shown in Listing 4.1. Its logic is deliberately simple, but the simplicity is the point: the split is performed over the unique scenario identifiers, not over the individual actor rows, so that every actor from a given scenario is assigned as a group to either training or testing. The random seed is fixed so that the same split is reproduced on every run.

Listing 4.1 Scenario-level train/test split (excerpt)

```
uniqueScenarios = unique(T.ScenarioID);
numScenarios    = numel(uniqueScenarios);

rng(RNG_SEED);           % fixed seed -> reproducible split
shuffled = uniqueScenarios(randperm(numScenarios));

numTrain    = round(TRAIN_FRACTION * numScenarios);
```

The split operates on unique ScenarioID values, so actors that share a scenario never straddle the train/test boundary the precaution that prevents leakage across near identical situations.

The choice of a fine decision tree as the classifier was governed by the interpretability principle stated in Chapter 3. A decision tree partitions the feature space with a sequence of axis aligned thresholds [17], [18], each of which can be read as a human legible rule for instance, that a small minimum distance to the ego combined with a high speed indicates risk. This transparency is what makes it possible, later, to say not merely that the classifier is accurate but why [44], and to attribute its errors under realistic sensing to specific corrupted features rather than to an inscrutable weight matrix.

Two disciplines run through the entire build chain and are worth stating explicitly, because they are what make the results of Chapter 5 reproducible rather than anecdotal. The first is determinism: every step that draws on randomness the placement of actors, the assignment of behaviors, and above all the train/test split is seeded from a fixed value, so that re-running the chain reproduces the identical dataset, the identical split, and therefore the identical accuracy, down to the last actor. The second is an idempotent, file-mediated hand off between stages: each stage reads named input files and writes named output files, overwriting rather than appending, so that re-running a stage can only reproduce or cleanly replace its output, never silently accumulate stale rows from a previous run. Together these disciplines mean that a reviewer given the scripts and the seed can regenerate every figure and every number in this report, and that the robustness branch can be re-entered in isolation, long after the training chain has finished, without any danger of contaminating it with residue from an earlier experiment.

4.3 Database Design (ER Model)

Although the project uses flat comma separated files rather than a relational database engine, the data has a definite relational structure, and making that structure explicit clarifies how the pieces fit together. Figure 4.3 presents the entity relationship model of the datastore, and Table 4.2 lists the principal files together with their schemas.

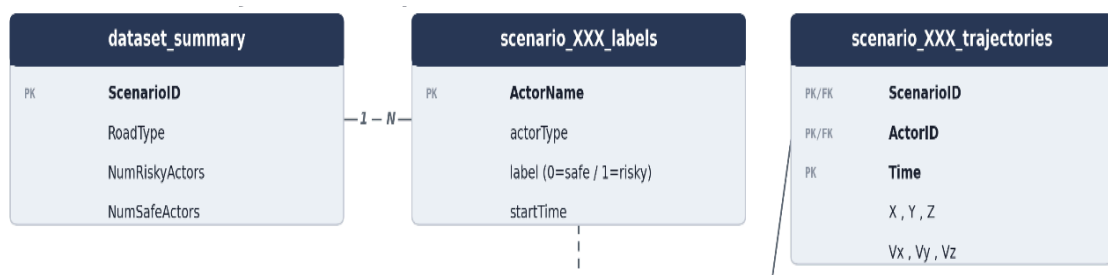


Figure 4.3: Entity relationship model of the scenario and feature datastore. A scenario contains many actors; each actor yields one ground truth feature row and, after sensing and tracking, one or more tracked feature rows; tracks are matched back to actors for the robustness comparison.

The central entity is the scenario, identified by a scenario identifier. Each scenario contains many actors; each identified within its scenario by name and carrying a type and a ground truth label. From each actor’s ground truth trajectory, a single feature row is derived. When the scenario is passed through the radar and tracker, the tracker produces tracks that do not carry the actors’ names a track is an anonymous identifier so a matching step associates each track with the actor whose true trajectory it most closely follows, allowing a tracked feature row to inherit the correct label for the robustness comparison. This matching is the relational join at the heart of the robustness experiment, and its implementation is discussed in the next section.

Table 4.2: Principal data files produced by the pipeline and the schema of each.

File	Key columns	Purpose
scenario_NNN.mat	scenario object	The saved driving scenario (roads, actors, motion)
scenario_NNN_labels.csv	ActorName, Label, Description	Ground truth label per actor
features_dataset.csv	ScenarioID, ActorName, ActorType, 10 features, Label	One ground truth feature row per actor
features_train.csv / _test.csv	as above	Scenario level split for honest evaluation
features_tracked_noisy.csv	10 features, label	Feature rows re-derived from radar tracked data
live_demo_predictions_log.csv	Time, ActorName, PredictedLabel	Per timestep live demo classifications

The schema of the combined feature file repays a second look, because it is where the data leakage story begins. Alongside the ten motion features and the label, the file also carries the scenario identifier, the actor’s name, the actor’s type, and a free-text description. Those last four columns are metadata: they are useful for bookkeeping and for the track-to-actor matching, but they must never be presented to the classifier as predictors, because some of them encode the answer [21], [22]. The description field in particular can name the very behavior whose label is to be predicted. The early, leaking classifier was trained with such columns still present; the leakage free classifier is trained on the ten motion features alone. Chapter 5 tells that story in full.

4.4 Core Code Logic

Three pieces of code form the logical core of the system: the feature extractor, which must behave identically on ground truth and tracked data; the coordinate frame correction, which makes the two comparable in the first place; and the track to actor matching, which lets a tracked trajectory inherit a ground truth label. Each is examined here.

4.4.1 A Single Feature Extractor for Both Paths

The cardinal rule of the robustness experiment is that any difference between ground truth and tracked accuracy must come from the data, never from the code. To guarantee this, exactly one feature extraction function is used on both paths. It takes a trajectory as time stamped positions, velocities, and speeds, and returns the ten-element feature table. Its first act is a guard: if fewer than four samples are present, it returns a row of missing values rather than attempt statistics that would be meaningless. Listing 4.2 shows this guard and the core of the computation.

Listing 4.2 The shared feature extractor (excerpt)

```
function feat = extractTrackFeatures(t, x, y, speed, egoX, egoY)
t = t(:); x = x(:); y = y(:); speed = speed(:);

if numel(t) < 4           % minimum-length guard
    feat = table(NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, ...
        'VariableNames', {'avg_speed','max_speed','speed_std', ...
        'max_accel','max_decel','max_lateral_rate','yaw_rate_std', ...
        'min_dist_to_ego','avg_dist_to_ego','duration'});
    return
end

dt      = diff(t); dt(dt <= 0) = median(dt(dt > 0));
accel   = diff(speed) ./ dt;
lateralRate = abs(diff(y)) ./ dt;
yawRate  = diff(unwrap(atan2(diff(y), diff(x)))) ./ dt(2:end);
% ... distances to ego, then assemble the 10-feature table ...
```

Because this identical function is applied to both ground truth and tracked trajectories, any accuracy gap between them is attributable to the input data, not to a difference in feature computation. The four-sample guard is what makes short, fragmented tracks drop out.

The guard is not a minor detail. On ground-truth trajectories almost every actor clears the four-sample threshold easily, so the guard rarely fires. On tracked trajectories, where the radar's shallow field of view and the tracker's fragmentation routinely reduce an actor to a handful of detections, the guard fires often, and every actor it removes is one the classifier never gets to judge. Part of the robustness loss reported in Chapter 5 is therefore not misclassification at all but abstention: the pipeline simply never forms a feature vector for a large share of actors it could see clearly in the ground truth world.

The extractor’s response to a too short trajectory returning a row of missing values rather than raising an error or fabricating a guess is a small design choice with a useful downstream property. Because the sentinel row has exactly the same shape as a valid feature row, it flows through the pipeline uneventfully and is removed by a single filtering step just before training or scoring, so that abstention is handled uniformly and visibly rather than through scattered special cases threaded through the code. It also makes abstention a measurable quantity in its own right: the difference between the number of actors present in a scenario and the number that yielded features is precisely the pipeline’s abstention rate, and it is by watching that rate climb under tracking that Chapter 5 is able to separate the accuracy lost to outright misclassification from the accuracy lost to never classifying an actor at all two failure modes that a less disciplined pipeline would blur together into a single, less informative number.

4.4.2 The Coordinate-Frame Correction

The radar and tracker report each track in the ego vehicle’s own moving frame: positions are relative to the car, and rotate with it as it turns. The ground-truth trajectories, by contrast, live in a fixed world frame. Comparing the two directly would be meaningless a stationary object would appear to move whenever the ego turned. Before any feature is computed from tracked data, therefore, each track’s coordinates are rotated and translated from the ego local frame back into the world frame, using the ego vehicle’s own precisely known position and heading. For a track reported at ego local offset

$$(rx, ry) \rightarrow X = Xe + rx \cos\psi - ry \sin\psi \quad (4.1)$$

$$Y = Ye + rx \sin\psi + ry \cos\psi \quad (4.2)$$

Were,

(X_e, Y_e) is the ego vehicle’s world position and ψ is its heading, both interpolated to the track’s timestamps. Speed, being rotation-invariant, needs no correction. Listing 4.3 shows the correction as implemented; it is a textbook rigid body transformation [45], but getting it right was essential, because an early version that omitted it made tracked features diverge wildly from ground truth for reasons that had nothing to do with sensor noise. Figure 4.4 illustrates the effect of the correction, returning ego local returns to their true world positions.

Listing 4.3 Ego-local to world-frame correction (excerpt)

The
rigid
body

```
% ego's own known position + heading, interpolated to track times
egoX_at = interp1(egoTraj.Time, egoTraj.X,      trackLog.Time, 'linear',
'extrap');
egoY_at = interp1(egoTraj.Time, egoTraj.Y,      trackLog.Time, 'linear',
'extrap');
egoYaw_at = interp1(egoTraj.Time, egoTraj.YawUnwrapped, trackLog.Time,
'linear', 'extrap');

relX = trackLog.X; relY = trackLog.Y;           % ego-local returns
trackLog.X = egoX_at + relX .* cos(egoYaw_at) - relY .* sin(egoYaw_at);
trackLog.Y = egoY_at + relX .* sin(egoYaw_at) + relY .* cos(egoYaw_at);
% speed is rotation-invariant, so Vx/Vy magnitude is unchanged
```

transform of equations (4.1), (4.2) in code. Without it, the ego vehicle’s own turning would masquerade as target motion and corrupt every downstream feature.



Figure 4.4: The coordinate frame correction in action: ego local radar returns (left) are rotated and translated by the ego’s known pose into the world frame (right), where they align with the ground truth trajectories and can be compared feature for feature.

It is worth being clear about what this correction can and cannot fix. Because it relies on the ego vehicle’s own pose its position and heading at each instant any error in that pose feeds directly into the reconstructed world coordinates of every track. In this simulation the ego pose is known essentially exactly, so the correction is faithful and the residual disagreement between tracked and ground-truth trajectories can be attributed to sensing and tracking alone, which is precisely the isolation the robustness experiment requires. On a real vehicle the ego pose would itself have to be estimated, and its uncertainty would layer a further distortion on top of the fragmentation studied here. The implication is sobering rather than reassuring: the accuracy fall reported in Chapter 5 is measured under the most favorable possible assumption about self-localization, so the figure obtained is, if anything, an optimistic bound relative to a physical system that must locate itself as imperfectly as it locates its neighbors.

4.4.3 Matching Tracks Back to Actors

The tracker knows nothing of the actors’ names; it emits numbered tracks. To evaluate whether a tracked trajectory was classified correctly, each track must be paired with the ground-truth actor it corresponds to, so that it can inherit that actor’s label. The matching routine does this by comparing trajectories: for each track, it finds the actor whose ground-truth path, interpolated onto the track’s timestamps, has the smallest average distance to the track [6], [39]. Listing 4.4 shows the essence of the comparison.

Listing 4.4 Nearest-trajectory track-to-actor matching (excerpt)

```
for each track tid:
    bestActor = ""; bestDist = inf;
    for each ground-truth actor aName:
        gx = interp1(gt.Time, gt.X, tk.Time, 'linear', 'extrap');
        gy = interp1(gt.Time, gt.Y, tk.Time, 'linear', 'extrap');
        avgD = mean( sqrt((tk.X - gx).^2 + (tk.Y - gy).^2) );
        if avgD < bestDist
            bestDist = avgD; bestActor = aName;
        end
    end
end
record (tid, bestActor, bestDist)
```

Each track is assigned to the ground truth actor whose interpolated path it follows most closely on average. This nearest trajectory join lets a tracked feature row inherit the correct safe/risky label for the robustness comparison.

This matching also quantifies fragmentation, which becomes central to the results. When a single real actor is tracked as several short-lived tracks because the tracker briefly lost and re-acquired it several distinct track identifiers all match back to the same actor [9], [10]. Counting how many tracks match each actor therefore measures directly how badly the tracker fragmented that actor's trajectory, and Chapter 5 uses exactly this count to explain why the observed durations collapse and, with them, the classifier's accuracy. The core logic of the system thus not only performs the robustness experiment but instruments it, so that the cause of any accuracy loss can be named rather than guessed at.

4.5 Reactive Closed-Loop Control

Not every scenario in the dataset is a scripted playback. A distinct family of scenarios generated by the reactive and complex reactive pipelines places actors under closed loop control, so that instead of following a pre-recorded path regardless of their surroundings, they sense the vehicle ahead, the state of a traffic signal, and any pedestrian on the crosswalk, and choose an acceleration in response at every timestep. This closed-loop realism matters for the integrity of the dataset: a classifier trained only on choreographed motion might learn the choreography rather than the behavior, whereas actors that genuinely brake for a leader and creep toward a red light produce emergent, physically consistent trajectories that no author scripted by hand. The controller that produces this behavior is implemented in `reactiveCarFollowingLib.m`, and because it shapes a large share of the training data it deserves to be documented in its own right.

4.5.1 The Intelligent Driver Model

The car following behavior is governed by the Intelligent Driver Model, a standard continuous time model from the traffic flow literature [46] that expresses a following

vehicle's acceleration as the sum of a free road term, which urges the vehicle toward its desired speed, and an interaction term, which forces it to back off as it closes on whatever lies ahead. The implementation follows the model exactly, as equations (4.3) and (4.4) state.

$$a = a[1 - (v/v_0)\delta - (s^*/s)^2] \quad (4.3)$$

$$s^* = s_0 + \max(0, vT + v\Delta v / (2\sqrt{ab})) \quad (4.4)$$

were,

v is the current speed, v_0 the desired free road speed, s the bumper-to-bumper gap to the leader, and Δv the approach rate (the follower's speed minus the leader's). The parameter struct supplies the desired time headway T , the minimum jam distance s_0 , the maximum acceleration a , the comfortable deceleration b , and the acceleration exponent δ . The quantity s^* is the desired dynamic gap: it grows with speed through the headway term and, crucially, grows further whenever the follower is approaching faster than the leader, which is what produces anticipatory rather than merely reactive braking.

The behavior that emerges from these two equations is exactly what a careful human driver does. On an empty road the interaction term vanishes and the vehicle accelerates smoothly toward its desired speed; as it closes on a slower leader the ratio of desired to actual gap climbs, the squared interaction term dominates, and the acceleration turns sharply negative, easing off again once a comfortable following distance is restored. When the gap collapses because a leader brakes hard or a stationary obstacle appears, the same term produces an emergency deceleration far exceeding the comfortable value, so the model brakes as hard as the situation demands without ever being told a fixed rule for doing so. A stationary obstacle is handled simply by evaluating the model with a leader speed of zero, and the free road case by dropping the interaction term entirely, as the companion free Road Acceleration routine does.

4.5.2 Arbitrating Leader, Signal, and Pedestrian Constraints

A real approach to an intersection is rarely governed by a single consideration. The ego vehicle in a reactive scenario may simultaneously be following a leader, approaching a signal that has turned amber, and nearing a crosswalk a pedestrian is about to enter. The controller resolves these competing demands by a principle that is both simple and safe: each active constraint is converted into a virtual leader and passed through the very same Intelligent Driver Model, and the most restrictive of the resulting accelerations the smallest, and therefore the most cautious is the one applied. A red or amber signal becomes a stationary leader sitting at the stop line; an active pedestrian event becomes a stationary leader at the edge of the crosswalk; and the vehicle physically ahead becomes a moving leader in the usual way. Listing 4.5 shows the arbitration.

Listing 4.5 Constraint arbitration in ego Control Acceleration (excerpt)

```

candidates = []; modes = {};
if hasLeader                                % (1) vehicle ahead
    gap = leaderPos - posEgo - p.vehicleLength;
    if gap > 0
        candidates(end+1) = idmAcceleration(vEgo, leaderVel, gap, p);
        modes{end+1} = 'leader';
    end
end

```

Each active constraint is mapped to a virtual leader and evaluated with the same IDM. Taking the minimum acceleration guarantees the vehicle obeys whichever constraint is most demanding, and the recorded mode names the binding constraint at every instant.

Two properties of this arbitration are worth drawing out. The first is that taking the minimum is probably safe in the sense that matters here: if any single constraint would require the vehicle to brake, the vehicle brakes at least that hard, so no constraint can ever be overridden by a less cautious one. The second is that the controller records which constraint was binding at each timestep in an info. Mode field free, leader, signal, or pedestrian which turns the closed-loop simulation into an instrumented one, exactly as the track-matching routine of the previous section did for fragmentation. That record makes it possible to verify that a scenario unfolded as intended and to explain, after the fact, why a given actor slowed where it did. The behavior that reaches the classifier as a labelled trajectory is therefore not scripted but genuinely emergent, arising from the interaction of these constraints, which is precisely what lends the reactive portion of the dataset its realism.

4.6 Closing the Loop: Classification-Driven Control

The controller of the previous section shapes how the actors in a scenario move while the data is being generated. This section describes a different and, for the project’s fifth objective, decisive loop: one in which the trained behavior classifier itself sits inside a live driving simulation and its real time verdicts actively change how the ego vehicle drives. Where everything up to this point has treated classification as something measured after the fact, the live demonstration implemented in `runLiveClassifiedDemo.m` treats it as a control input [27], [28], and in doing so tests whether a safe or risky judgement is not merely accurate on paper but useful in the loop. Importantly, the model it loads is the clean, leakage free classifier retrained after the audit of Chapter 5, so that the driving behavior on display reflects genuine learned motion patterns rather than the artefact that the leakage investigation removed.

4.6.1 The Perception Classification Control Cadence

The demonstration runs the three stages on distinct clocks, in imitation of a real embedded system in which perception, inference, and control need not tick together. As the ego drives its route, a rolling per-actor history of position and speed is accumulated at every step. Every twenty steps two seconds at the ten-hertz update rate each actor that has accumulated at least four samples of history, the same minimum length guard used throughout the project, is passed through the feature extractor and the classifier, and the resulting safe or risky verdict is cached in a per-actor map. The display is redrawn more often, every five steps, coloring each actor green for a safe verdict, red for a risky one, and yellow for an actor not yet classified because it has too little history. The two-second re-classification interval is a deliberate compromise: frequent enough that a verdict tracks an actor whose behavior changes, yet infrequent enough to mimic the bounded inference budget of a real vehicle rather than assuming a fresh classification on every frame.

This cadence also makes the demonstration’s headline activity figures meaningful. Over a full drive the loop produced several thousand individual predictions across the actors it encountered a volume reported precisely in Chapter 5 because each actor is re-judged repeatedly as it moves through the scene rather than labelled once. A verdict is therefore not a permanent verdict on an actor but a running assessment that can flip from safe to risky and back as the evidence in the rolling buffer evolves, which is exactly the behavior a deployed monitor would need and exactly the behavior that a single, trajectory complete classification could never exhibit.

4.6.2 Longitudinal and Lateral Response to a Risky Verdict

A verdict earns its place in the loop only by changing what the ego does, and it does so along both the longitudinal and the lateral axes. Longitudinally, the ego maintains a desired following gap that scales with its own speed, and a risky verdict on the vehicle ahead inflates that gap by a fixed multiplier, so that the ego deliberately hangs further back from an actor it distrusts. Listing 4.6 shows the rule.

Listing 4.6 Classification-driven following-gap control (excerpt)

A

risky

```
riskyGapMultiplier = 1.8; % extra following distance from a risky lead
safeTimeGap = 2.0; minGap = 5; maxAccel = 2; maxDecel = 4;
% ... every 2 s, each actor with >= 4 samples is (re)classified ...
leadIsRisky = isKey(riskLabel, nm) && riskLabel(nm) == 1;
gapMultiplier = 1;
if leadIsRisky
    gapMultiplier = riskyGapMultiplier;          % widen the gap by 80%
end
desiredGap = gapMultiplier * (minGap + safeTimeGap * egoSpeed);
if gapToLead < desiredGap                        % too close -> ease off
    speedTarget = max(0, leadSpeed - (desiredGap - gapToLead) * 0.5);
else
    speedTarget = maxSpeed;                      % clear -> resume cruise
end
% speed slews toward the target within comfortable accel/decel limits
if egoSpeed < speedTarget, egoSpeed = min(speedTarget, egoSpeed +
maxAccel*dt);
else
    egoSpeed = max(speedTarget, egoSpeed - maxDecel*dt);
end
```

verdict on the lead actor multiplies the desired following gap by 1.8, so the ego holds a following distance eighty per cent larger than it would behind an identically placed actor judged safe; the ego then slews its speed toward the resulting target within fixed comfort limits.

The numbers make the effect concrete. Behind an actor judged safe, the ego seeks a gap of the minimum five meters plus two seconds of headway at its current speed; behind an identically placed actor judged risky, that same gap is multiplied by one-point-eight, so the ego trails almost twice as far back. When the actual gap falls short of this target the ego eases its speed toward the leader's, braking harder the larger the shortfall, and when the road ahead is clear it resumes cruising toward its maximum speed always slewing within a comfortable acceleration of two meters per second squared and a firmer deceleration of four, so that the response reads as measured driving rather than as an on off switch.

Laterally, a risky verdict makes the ego both more anticipatory and more evasive. The distance at which it begins to react to an obstacle ahead is extended when that obstacle is a predicted risky actor, so the ego starts responding earlier to the neighbors it most distrusts, and predicted risky actors nearby also induce a lateral offset that nudges the ego away from them within the width of its lane, a miniature avoidance maneuver bounded by a maximum lateral rate so that it never swerves abruptly. The combined result is a vehicle that keeps its distance from, reacts sooner to, and edges around exactly those actors the classifier flags a visible, physical consequence of an otherwise abstract safe or risky label.

4.7 Testing and Verification

A pipeline of this length fails quietly. A sign error in a rotation, a track identifier that is reused after an occlusion, or a feature column that is silently shifted by one sample will not raise an error in MATLAB; it will produce a plausible looking number that is wrong. Verification was therefore not deferred to a final testing phase. Each stage of the pipeline carries its own checks, and the properties those checks assert are stated here so that the results of Chapter 5 can be read with a clear view of what has been established and what has not.

4.7.1 Stage Level Consistency Checks

Every stage writes its output to disk before the next stage reads it, and that boundary is where the cheapest and most useful checks live. After scenario generation the recorded structure is checked for the properties the rest of the pipeline assumes: that the time vector is strictly increasing and uniformly spaced at the configured sample time, that an actor which leaves the scene does not reappear later under a different identifier, and that no pose field contains NaN or Inf. After feature extraction the resulting table is checked for shape and range rather than for exact values. The row count must equal the number of actor sample pairs the scenario actually contains, speeds must be non-negative, ranges must be positive, and the accelerations and yaw rates obtained by differencing must stay inside bounds that are physically reasonable for the vehicle classes being simulated. A value outside those bounds is treated as a defect in the differencing or in the frame handling rather than as an interesting observation, because the scenarios are authored and contain no such maneuver.

The label column receives separate attention, because it is the one field that cannot be reconstructed if it is wrong. Labels follow the deterministic actor identifier convention

described in Chapter 3 rather than a hand-maintained list, so the check is that every actor identifier appearing in the feature table maps to exactly one label across the whole batch, and that the class counts summed over scenarios match the counts the generator intended. A mismatch points either at an actor that was renumbered when a scenario was edited or at a scenario that was regenerated without its label mapping being carried forward.

4.7.2 Verifying the Coordinate Frame Transform

The transform of Section 4.4.2, given in equations (4.1) and (4.2), is verified by the property it must satisfy rather than by inspection of its output. The simulator can report actor poses in both the ego frame and the world frame, so the transform applied to the ego frame pose must reproduce the world frame pose the simulator already holds. Agreement is required to within the tolerance expected from floating point arithmetic and from the interpolation used to align the two records in time; a residual larger than that indicates a sign error in the rotation or a mismatch between the yaw convention assumed by the transform and the one used by the simulator. The test is worth more than its size suggests, because the failure it catches is exactly the failure that looks correct. A rotation with the wrong sign still produces smooth, bounded trajectories, and the error surfaces only as a systematic bias in the range and heading features that the classifier subsequently learns from.

A second check covers the stationary case. An actor that does not move in the world frame must, after the transform, hold a constant world frame position while the ego drives past it. Any residual drift in that position is attributable to the transform rather than to the actor, which makes the stationary actors present in each scenario a free and continuously available regression test.

4.7.3 Verifying the Tracker and the Track to Actor Join

The extended Kalman filter is checked against its own statistics [4], [7]. The innovation of equation (A.4) should, if the process and measurement noise models are consistent with the data, remain small relative to the innovation covariance of equation (A.5), and for a correctly associated detection the squared Mahala Nobis distance of equation (A.14) should stay below the gate. A filter whose innovations are persistently large relative to their predicted covariance is over confident [8], and the remedy lies in the noise model rather than in the gate; a filter whose gate is never approached is under confident and is accepting associations it ought to question. Inspecting the distribution of gating distances over a full batch therefore does more than confirm that the tracker runs, it shows whether the covariance is calibrated.

The join between tracks and ground truth actors is verified separately, because a correct tracker combined with an incorrect join yields features attached to the wrong label. The assignment of equation (A.15) is checked for the two properties it must have [39]: within a sample it must be one to one, so that no actor receives two tracks and no track is credited to two actors, and the actor a track is joined to must remain stable for as long as the track lives. Identifier changes inside a continuous track, and tracks whose assigned actor changes

mid-life, are counted rather than suppressed, and those counts are what the fragmentation results in Chapter 5 report.

4.7.4 Determinism and Repeatability

None of the above is meaningful unless a run can be repeated. Every stochastic element in the pipeline draws from an explicitly seeded random number generator and the seed is recorded with the scenario, so a scenario regenerated later produces the same detections and the same tracks rather than a statistically similar set. The data partition used by the classifier is likewise fixed rather than redrawn, which is what allows the confusion matrices of Chapter 5 to be compared with one another at all.

Repeatability was checked end to end by regenerating the batch from the recorded seeds and comparing the resulting feature tables against the tables the reported results were computed from. Equality there establishes something narrow but valuable: that the numbers of Chapter 5 are a property of the system and its configuration rather than of one particular execution. Where a stage cannot be made bit identical, as is the case for the live closed loop demonstration of Section 4.6 because it depends on wall clock timing, the run is reported as a demonstration rather than as a measurement, and no quantitative claim in this thesis rests on it.

Taken together these checks establish internal consistency, that the implementation computes what Chapter 3 says it computes and that it does so reproducibly. They do not establish that the simulated scenarios resemble the road, which is a separate question and is treated among the limitations discussed in Chapter 6.

CHAPTER 5: RESULTS AND ANALYSIS

This chapter presents what the system actually does and how well it does it. It begins with the experimental setup the radar, tracker, and runtime configuration under which every measurement was taken. It then shows the system in operation through a set of scenario visualizations and the live closed loop demonstration. The heart of the chapter is the performance analysis, which unfolds in three movements: the discovery and elimination of data leakage, the honest leakage-free performance of the classifier, and the robustness analysis that exposes how sharply that performance falls under realistic sensing. A closing discussion interprets the numbers and draws out their meaning for the objectives set in Chapter 1.

5.1 Experimental Setup

All experiments were conducted in MATLAB using scenarios generated by the pipeline of Chapters 3 and 4. The dataset comprises 450 driving scenarios across seven road network types, populated with 3,615 labelled actors. After feature extraction and the four sample minimum length guard, 3,464 actors yielded valid feature rows; the remaining 151 were observed too briefly, even in ground truth, to produce meaningful statistics. Figure 5.1 depicts the sensing configuration: a single forward-looking radar with a wide but shallow field of view, together with the numerical parameters that define it.

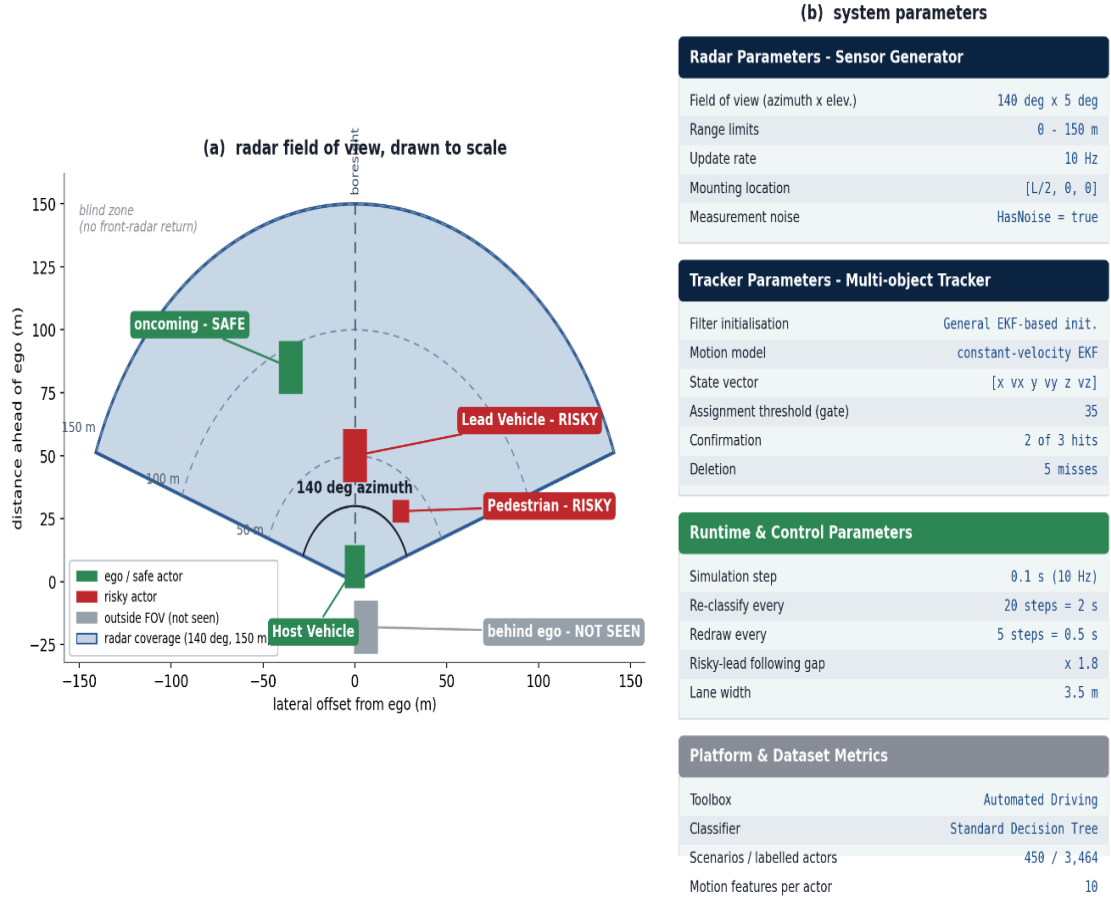


Figure 5.1: Experimental setup: the forward-looking radar’s coverage a 140° azimuth wedge out to 150 m overlaid on the ego vehicle, with the full parameter table. The shallow wedge is the geometric root of the observation coverage problem analyzed later in this chapter.

The complete set of radar, tracker, and runtime parameters is given in Table 5.1. These values were held fixed across every experiment, so that the only variable distinguishing the ground truth and sensor tracked conditions is the data itself, never the configuration.

Table 5.1: Radar, tracker, and runtime parameters used throughout the experiments.

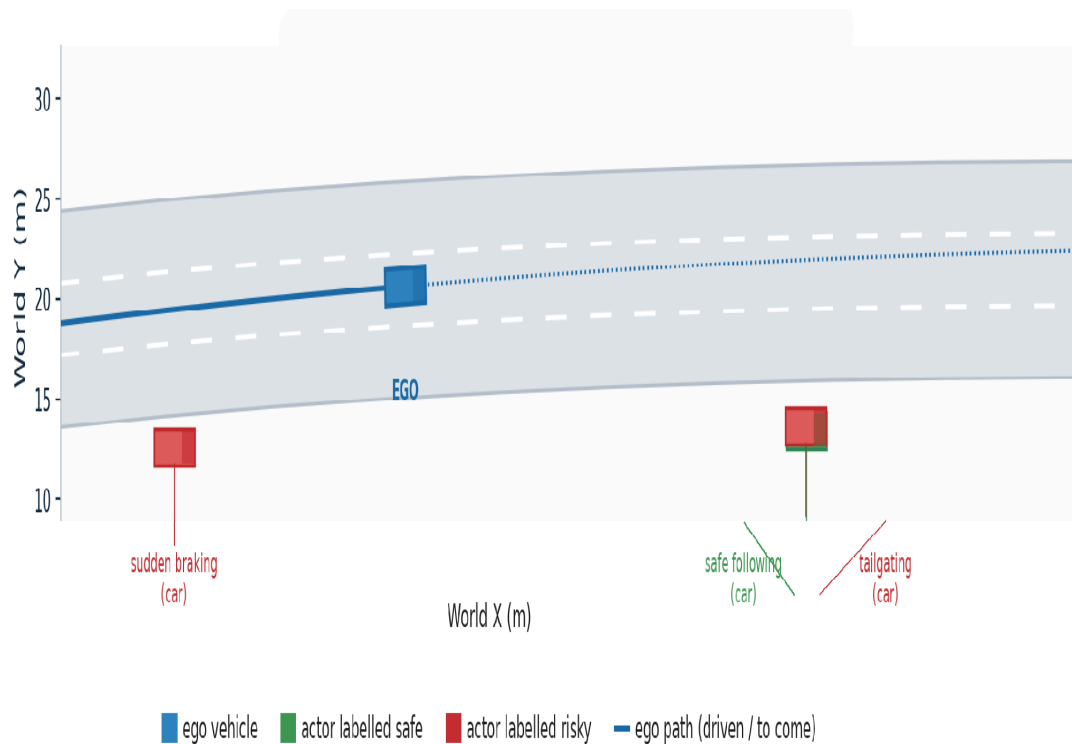
Component	Parameter	Value
Radar	Field of view (azimuth × elevation)	$140^\circ \times 5^\circ$
Radar	Range limits	0 – 150 m
Radar	Update rate	10 Hz
Radar	Mounting location	Front of ego (length/2, 0, 0)
Radar	Measurement noise	Enabled
Tracker	Filter	Constant-velocity EKF (initcvekf)
Tracker	Assignment threshold	35
Tracker	Confirmation threshold	2 out of 3
Tracker	Deletion threshold	5 misses
Tracker	Track state	[x vx y vy z vz]

The single most consequential number in this table is the field of view. A 140-degree azimuth span sounds generous, but paired with only a 150-meter range and, above all, with the fact that it points in one direction, it means that any actor outside the forward wedge behind the ego, or beside it during a turn is simply invisible.

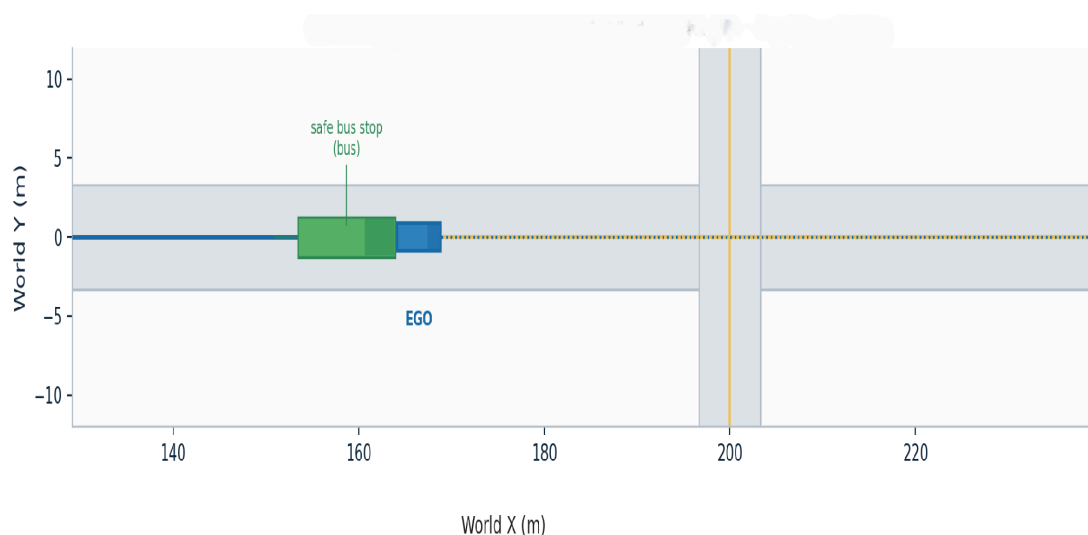
5.2 Output Visualizations

Before turning to numbers, it is useful to see the scenarios the system reasons about. Figures 5.2 through 5.6 show representative scenario canvases spanning the range of road types in the dataset, and Figure 5.7 and Figure 5.8 show the live closed-loop demonstration in operation.

Figure 5.2: A highway scenario canvas featuring tailgating and sudden braking behaviors, with the ego



vehicle and surrounding traffic. Risky actors are those whose motion signatures match the risky modules of Table 4.1.



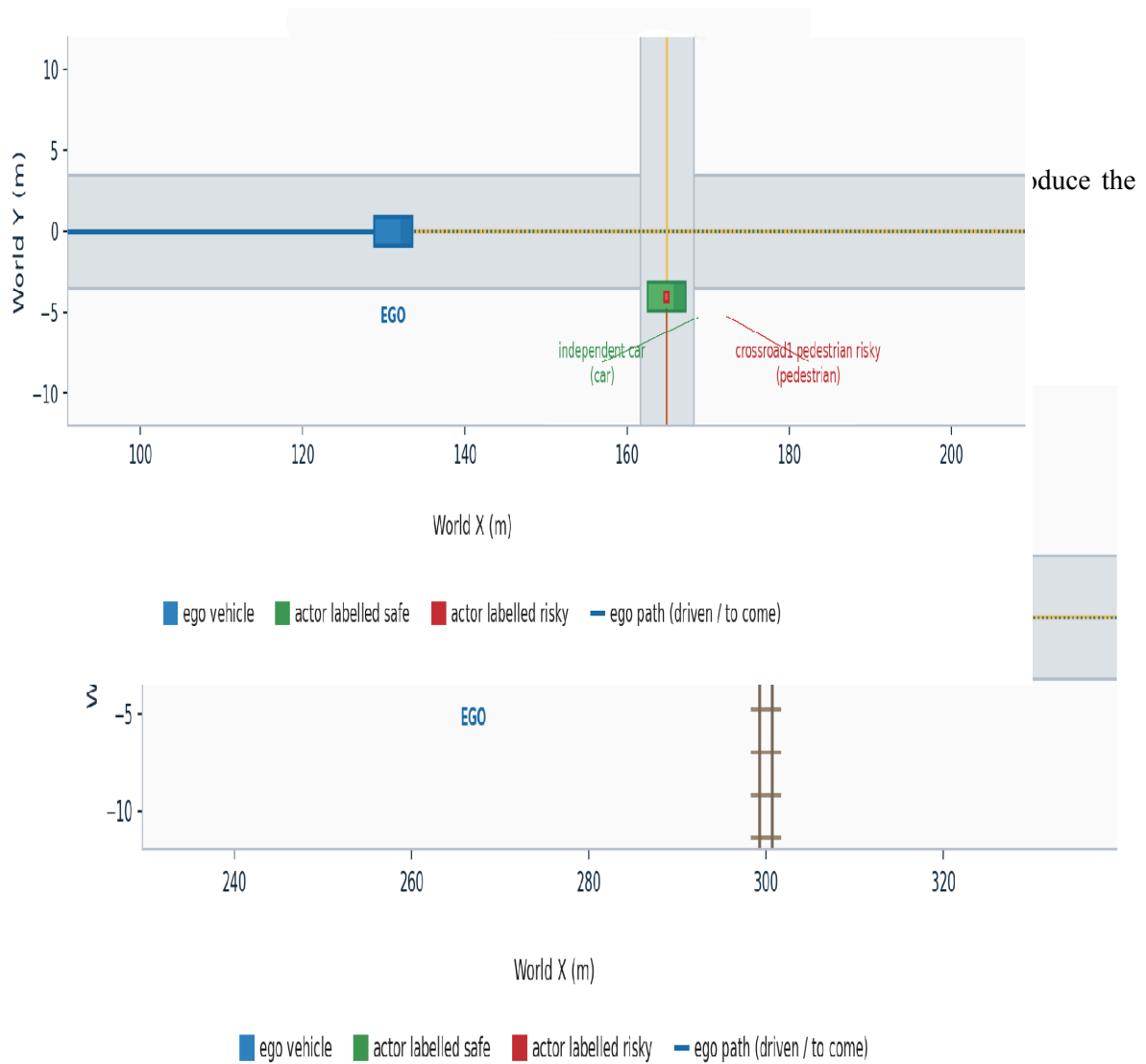


Figure 5.4: A railway level crossing scenario canvas, one of the rare high-risk situations concentrated in the most complex tier, in which a train constrains the ego vehicle’s options.

Figure 5.5: A multi-crossroad complex route scenario canvas, in which the ego vehicle executes a sequence of turns through several intersections the situations in which the forward radar’s coverage gaps are most severe.

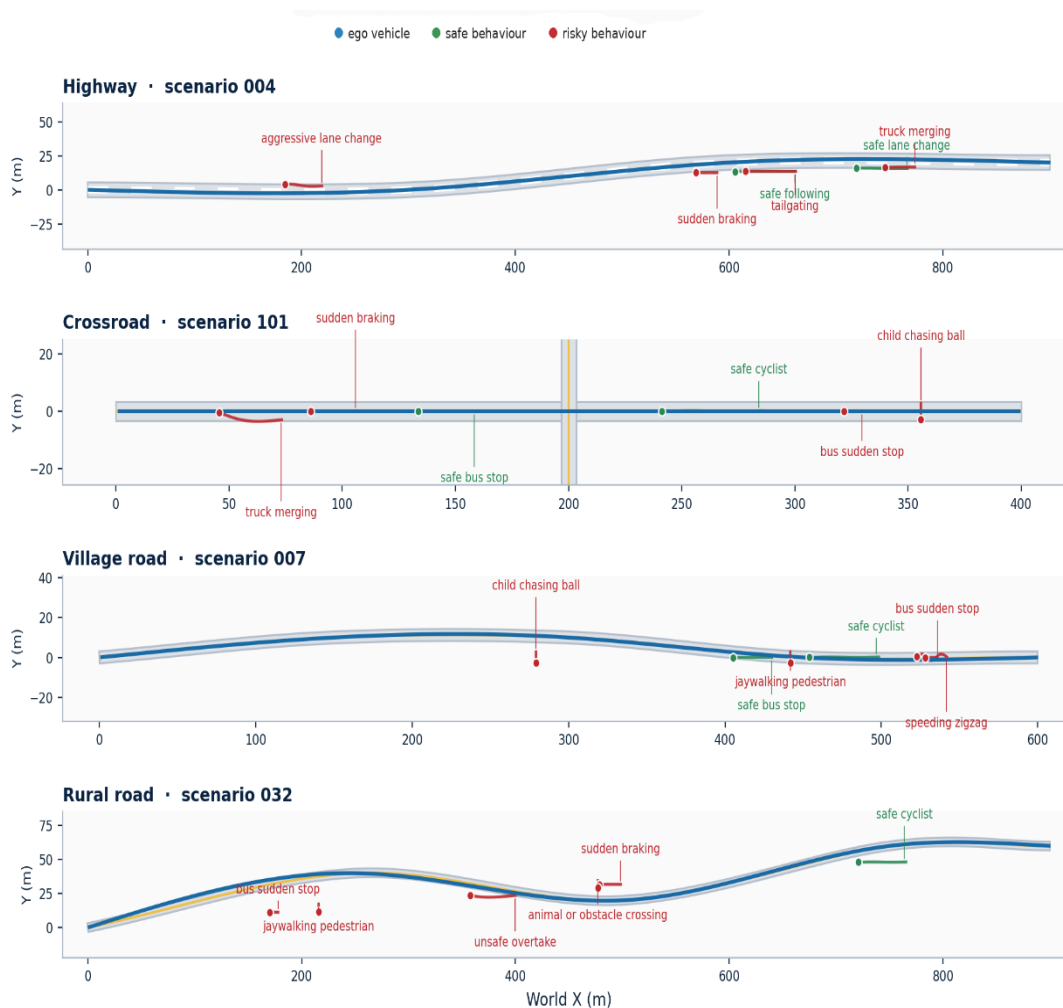


Figure 5.6: Bird's-eye view of representative full ego routes across the dataset, illustrating the diversity of paths the ego vehicle follows and the road geometries it encounters.

These five canvases are worth reading together as a spectrum of difficulty rather than as five unrelated pictures, because each road type shifts where the risk lives. On the highway of Figure 5.2 the danger is almost purely longitudinal: vehicles travel fast in the same direction, and the risky behaviors that occur there tailgating and sudden braking express themselves as small following gaps and hard decelerations along the direction of travel. At the signaled crossroad of Figure 5.3 the risk becomes a question of right of way rather than of speed: the dangerous behaviors are running a red light and failing to yield when turning across oncoming traffic, and they are legible only in relation to the signal phase and the paths of cross traffic. The railway level crossing of Figure 5.4 introduces a hazard of a third kind a hard external constraint in the form of an approaching train, against which the only safe behavior is to wait and the characteristic risky one is to dash across as the barrier falls. The village and rural roads that dominate Figure 5.6's routes shift the risk again onto vulnerable and unpredictable actors: a child chasing a ball into the carriageway, a jaywalking pedestrian, a cyclist swerving, or an overtake attempted into oncoming traffic on a two-lane road.

This progression is not merely descriptive; it foreshadows exactly where realistic sensing will struggle, which is the subject of the next section. The multi-crossroad route of Figure 5.5 is the hardest case for the sensing configuration of Figure 5.1 precisely because the ego is turning through a sequence of intersections, so that actors relevant to a decision routinely approach from directions the single forward-facing radar’s wedge does not cover. A hazard the eye picks out instantly from the bird’s-eye canvas a vehicle entering from the left as the ego turns right may never enter the radar’s field of view at all. The canvases therefore do double duty: they confirm that the dataset spans a genuine range of road types and risk structures, and they locate in advance the situations in which the coverage limitation quantified in Section 5.3 will do the most damage.

The live demonstration exercises the classifier in real time. Over a full drive it produced 5,832 per-timestep predictions across 18 tracked actors, adjusting the ego vehicle’s following distance and lane change decisions whenever an actor was judged risky, according to the classification driven control rule set out in Section 4.6. Figure 5.7 shows the per-actor classification timeline over the whole drive, and Figure 5.8 shows the final verdict reached for each actor.

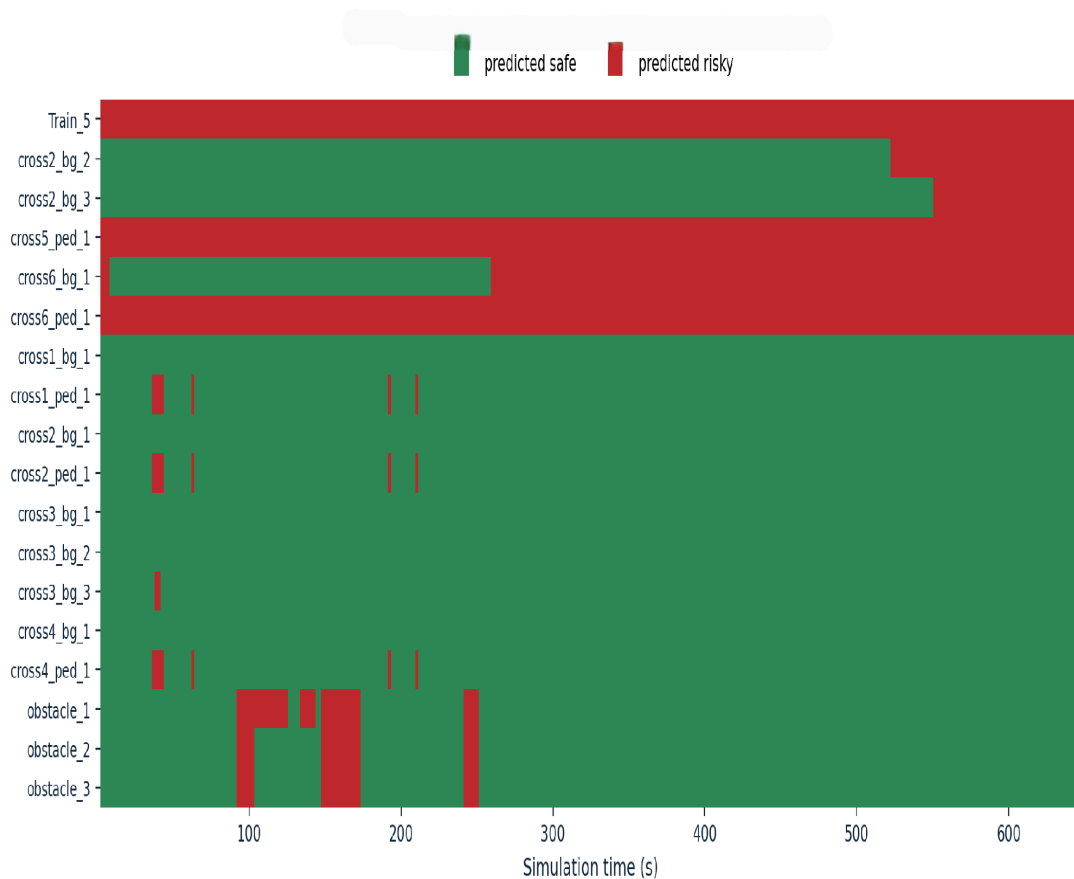


Figure 5.7: Live demonstration per-actor classification timeline over the full drive. Each row is an actor; the color at each instant is the classifier’s real time safe/risky verdict, showing how judgements evolve as actors move through the radar’s field of view.

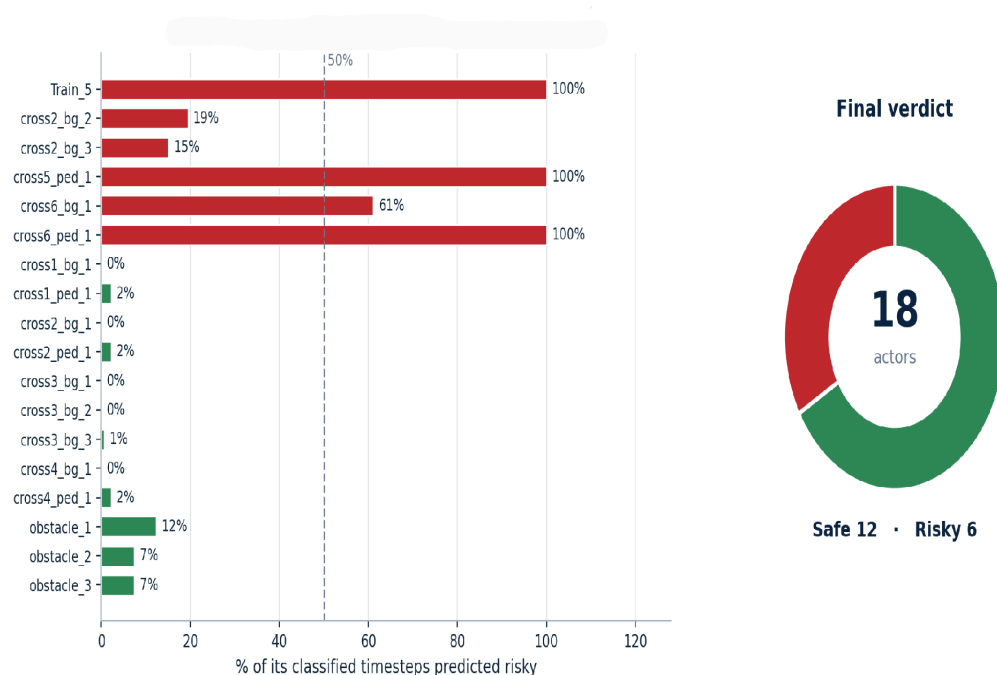


Figure 5.8: Live demonstration final summary: the share of risky timesteps and the final verdict recorded for each actor. The final verdict is each actor’s last prediction rather than a majority vote, so an actor risky for only part of the drive can still be filed as risky.

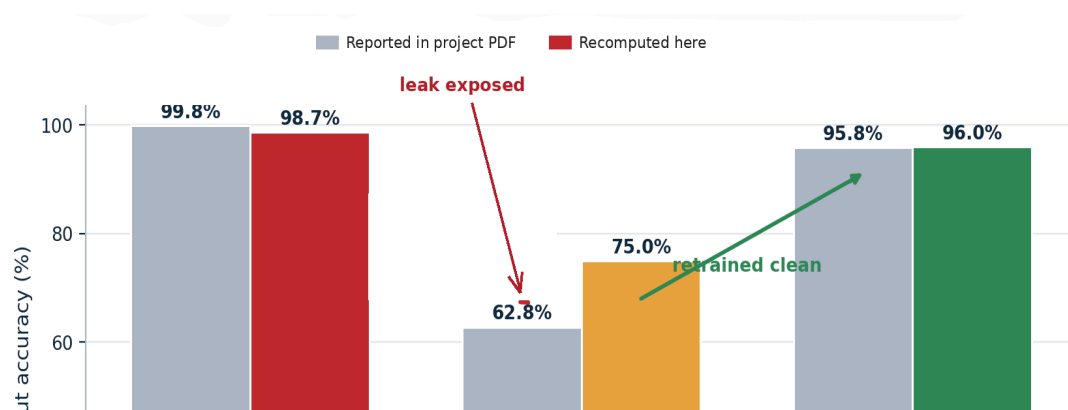
The last point in the caption of Figure 5.8 is a genuine design choice with observable consequences. Because the recorded verdict is the actor’s final prediction rather than a majority over the drive, an actor that behaved riskily only briefly in one case, for less than a fifth of its observed timesteps is nonetheless filed as risky. This is defensible for a safety system, where the most recent evidence matters most, but it is stated plainly here so that the summary figures are not misread as time-averaged judgements.

5.3 Performance Analysis

The performance analysis proceeds in three stages, deliberately ordered to tell an honest story: first the leakage that had to be found and removed, then the genuine performance that remained, and finally the robustness test that is the project’s central contribution.

5.3.1 The Data Leakage Investigation

An early version of the classifier achieved an accuracy close to 100%. Rather than accept this flattering result, the investigation treated it as a warning sign, because near-perfect accuracy on a

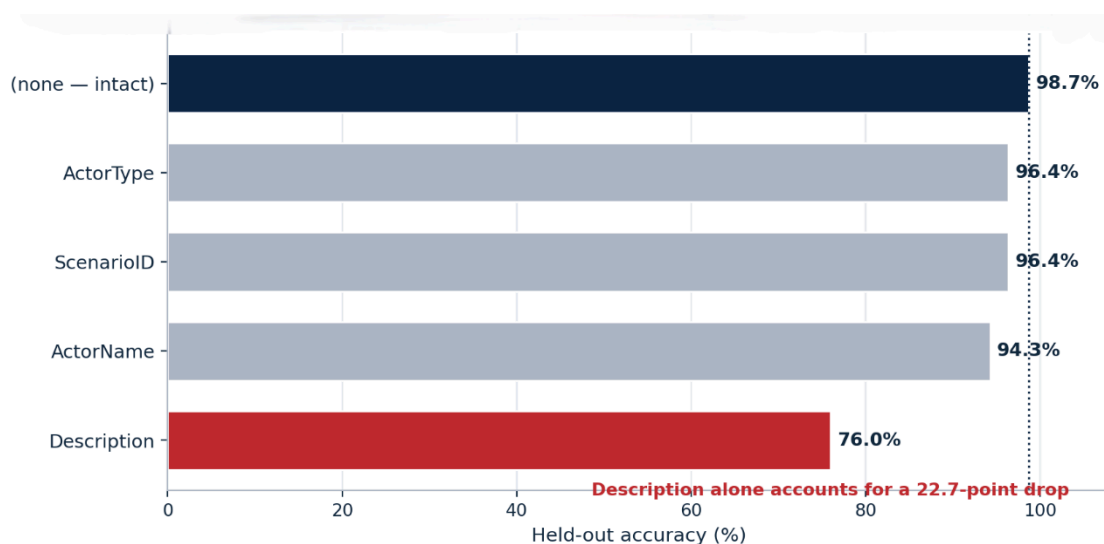


genuinely hard behavioral problem is far more often a symptom of leakage than of success [21], [22]. The feature file, as noted in Section 4.3, carried four metadata columns alongside the ten motion features: the scenario identifier, the actor’s name, the actor type, and a free text description. The description in particular could name the very behavior to be predicted. The hypothesis was that the tree had learned to read the answer from these columns instead of learning motion.

Figure 5.9: Data leakage diagnostic. The leaked model, trained with the metadata columns present, scores close to 100%. When those columns are randomly shuffled the accuracy collapses to about 62.8%, proving the model had leaned on them. The clean model, trained on the ten motion features alone, recovers a genuine 96.0%.

The diagnostic in Figure 5.9 confirms the hypothesis decisively. Three conditions are compared. With the metadata present, accuracy is close to 100% the leaked, dishonest result. When the metadata columns are randomly shuffled so that they can no longer encode the label but remain present as predictors, accuracy collapses to roughly 62.8%, which could only happen if the model had been relying on them heavily. When the metadata columns are removed altogether and only the ten genuine motion features are used, accuracy settles at a genuine 96.0% on the held-out test set. The gap between the first and third conditions is the exact measure of how much the leak had inflated the original result.

The middle condition shuffling the metadata in place rather than deleting it is the most diagnostic of the three, and it is worth saying why. Simply removing the suspect columns and observing a lower accuracy would be ambiguous, because a fall could equally mean that those columns carried legitimate predictive signal that any model would forfeit once they were gone. Randomly permuting a column, by contrast, destroys the association between that



column and the label while preserving the column’s distribution and its mere presence as a predictor, so that whatever accuracy the model retains must be coming from the other features. This is exactly the logic that underlies permutation feature importance [19], [38], and applying it here turns a vague suspicion of leakage into a controlled experiment. Because

accuracy falls so sharply from near-perfect to about 62.8% when nothing changes but the shuffling of the metadata, the metadata cannot have been carrying honest motion signal in that proportion; the greater part of the model’s apparent skill was reading label information that those columns quietly encoded, precisely the fingerprint of leakage rather than of learning.

Figure 5.10: Attribution of the leak to individual metadata columns. The free text description column carries the overwhelming majority of the leakage, with actor name and type contributing less; the scenario identifier is a weaker but still improper predictor.

Figure 5.10 attributes the leak to specific columns. The free text description carries the overwhelming majority of it, which is unsurprising given that it can literally contain the behaviors name; the actor’s name and type contribute smaller amounts, and the scenario identifier a smaller amount still. On the strength of this attribution, all four metadata columns were dropped. The feature file shrank from 3,464 rows by 15 columns to a clean 3,360 rows by 11 columns ten motion features plus the label and the classifier was retrained from scratch on this honest basis. Every result reported hereafter uses this leakage-free model.

5.3.2 Leakage Free Performance

Trained on the ten motion features alone, with a scenario level eighty–twenty train/test split seeded for reproducibility [20], the fine decision tree achieved a cross-validated accuracy of 95.4% and a held-out test accuracy of 96.0%. These numbers, summarized in Table 5.2, are the project’s honest headline result: high, but earned from motion rather than metadata, and measured on scenarios the model never saw during training.

Table 5.2: Leakage free classifier performance and configuration.

Quantity	Value
Cross-validated accuracy	95.4%
Held-out test accuracy	96.0%
Predictor features	10 motion features
Leaky metadata columns removed	4 (description, actor name, actor type, scenario ID)
Train / test split	Scenario-level, 80 / 20, seed 42
Clean dataset size	3,360 rows × 11 columns

Figure 5.11 shows the confusion matrix on the held-out actors [37], and Figure 5.15 shows which motion features do the work of separating the two classes. The confusion matrix confirms that errors are distributed across both classes rather than concentrated in a systematic bias, and the feature separation analysis confirms that the classifier relies on exactly the physically meaningful features one would hope for.

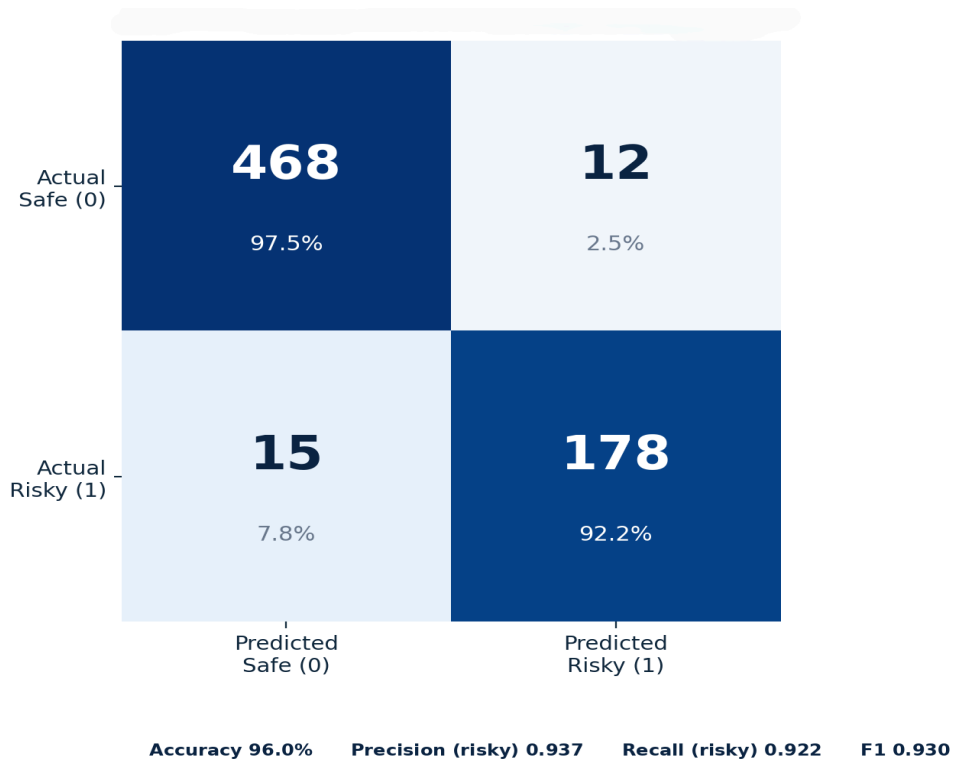


Figure 5.11: Confusion matrix of the leakage free classifier on the held-out test actors. Correct classifications dominate the diagonal, and the off-diagonal errors are modest and reasonably balanced between false safe and false-risky mistakes.

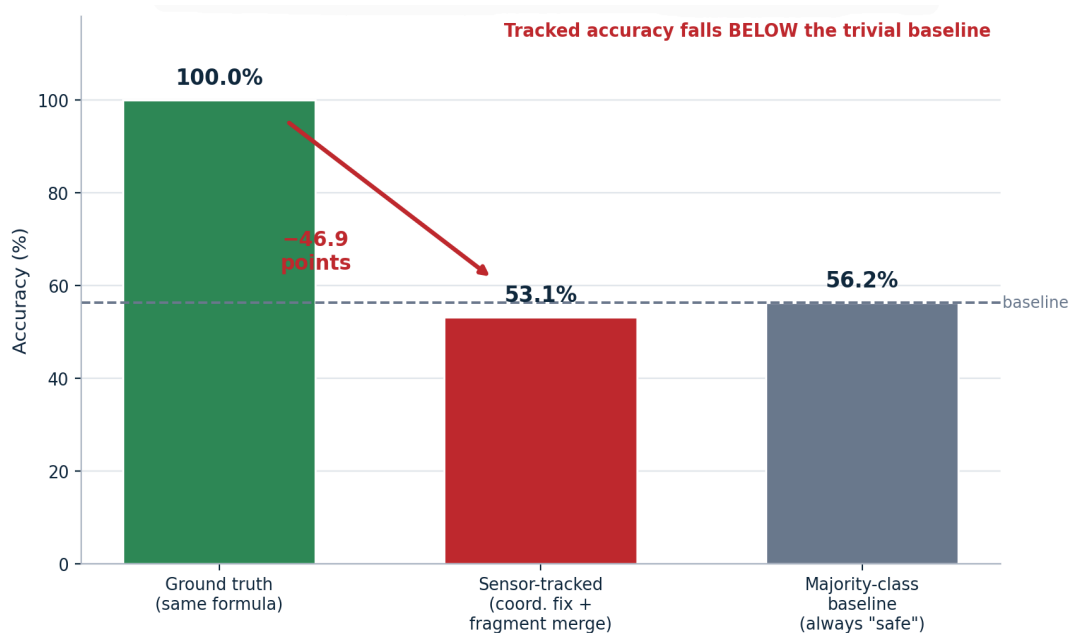


Figure 5.15: Which motion features separate safe from risky behaviors. Minimum distance to the ego, peak deceleration, and the lateral-motion measures carries the most discriminative power precisely the features by which the risky modules of Table 4.1 were designed to reveal themselves.

The concordance between Figure 5.15 and the module design of Chapter 4 is the strongest available evidence that the leakage free model learned what it was supposed to. Minimum

distance to the ego separates tailgating from safe following; peak deceleration separates sudden braking from gentle stopping; the lateral and yaw measures separate swerving from lane-keeping. The classifier is, in this sense, legible: its reliance on these features can be read directly and matched against physical intuition, which is exactly the advantage that motivated the choice of an interpretable decision tree over a black-box model [44].

The close agreement between the two accuracy figures is itself worth a remark, because they are computed in different ways and could easily have disagreed. The cross-validated figure of 95.4% is an average over folds in which every scenario takes its turn in the validation set, estimating how the method generalizes across resampling's of the data; the held-out figure of 96.0% is a single verdict on one reserved set of scenarios the model never touched. When a model has overfitted, these two numbers characteristically diverge [20] the resampled estimate sinks below the optimistic single split, or the reserved set turns out to be unrepresentative. That they instead fall within half a percentage point of one another is evidence that the decision tree, for all that a fine tree can in principle memorize its training data, has learned structure that transfers rather than idiosyncrasies that do not, and that the scenario-level split has done its job of keeping the estimate honest. It is precisely because this leakage free performance is trustworthy that the collapse reported next can be read as a real property of the sensing pipeline rather than as an artefact of a fragile model.

5.3.3 Robustness Under Realistic Sensing and Tracking

The final and most important experiment removes the assumption on which every result so far has rested: that the classifier is fed complete, clean trajectories. In deployment it would instead be fed whatever the radar and tracker manage to observe. To measure the consequence, a mixed sample of ten scenarios was passed through the full radar and tracker stack of Chapter 4; features were re-derived, using the identical extractor, from both the ground-truth trajectories and the sensor tracked trajectories; and the same leakage free classifier was applied to each. The outcome is stark, and is summarized in Table 5.3.

Table 5.3: Classifier accuracy underground truth and sensor tracked conditions, on a mixed sample of ten scenarios (32 tracked feature rows).

Condition	Accuracy	Mean observed duration
Ground truth (re-computed features)	100.0%	125.02 s
Sensor-tracked (radar + tracker)	53.1%	25.18 s
Chance level (binary problem)	50.0%	-----

On the ground truth trajectories, the classifier scores a perfect 100% on this sample, confirming that its learned logic is sound. On the sensor-tracked trajectories the accuracy falls to 53.1% barely above the 50% one would obtain by guessing on a two-class problem. A model that is essentially flawless on clean data is reduced to little better than a coin toss once

it must rely on realistic perception. This single comparison is the central empirical finding of the project.

The natural question is why. Figure 5.12 shows the accuracy gap directly, and the answer lies in the third column of Table 5.3. The mean duration for which an actor is observed falls from 125.02 seconds in ground truth to just 25.18 seconds when tracked roughly a fifth. Figure 5.13 shows this collapse in observed duration, and Figure 5.14 shows its mechanism: track fragmentation, in which a single real actor is repeatedly lost and re-acquired by the tracker and therefore appears as a succession of short, broken tracks rather than one continuous trajectory.

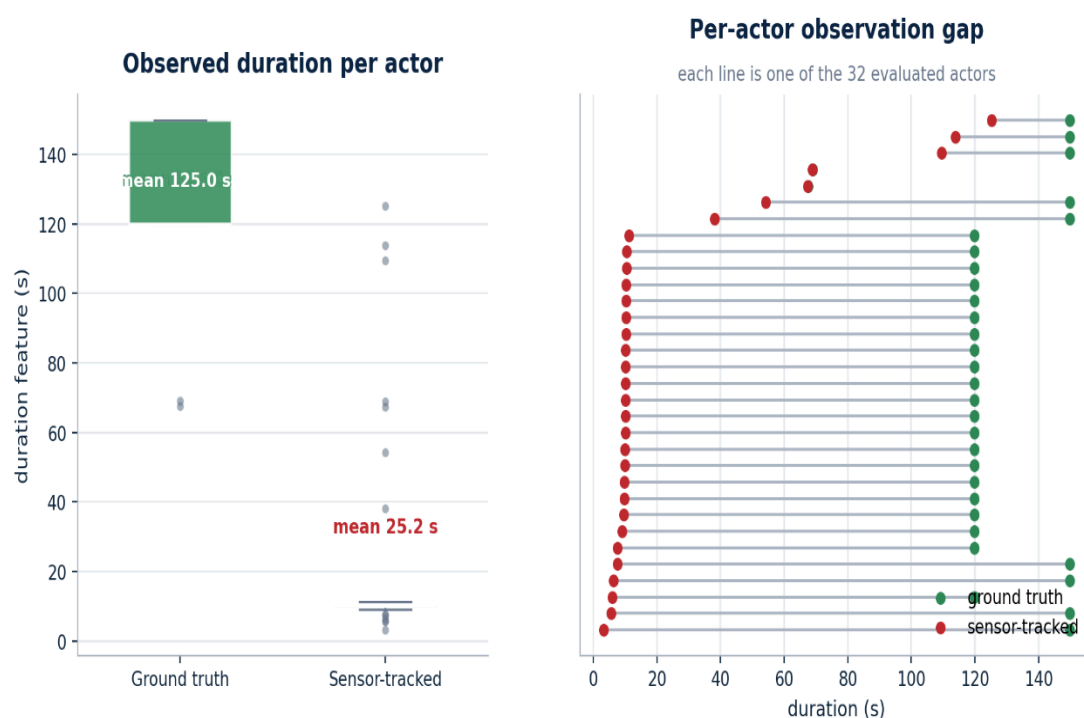


Figure 5.12: Robustness accuracy: the leakage free classifier scores 100% on ground truth features but only 53.1% on sensor tracked features from the same scenarios a fall to near the chance level for a binary problem.

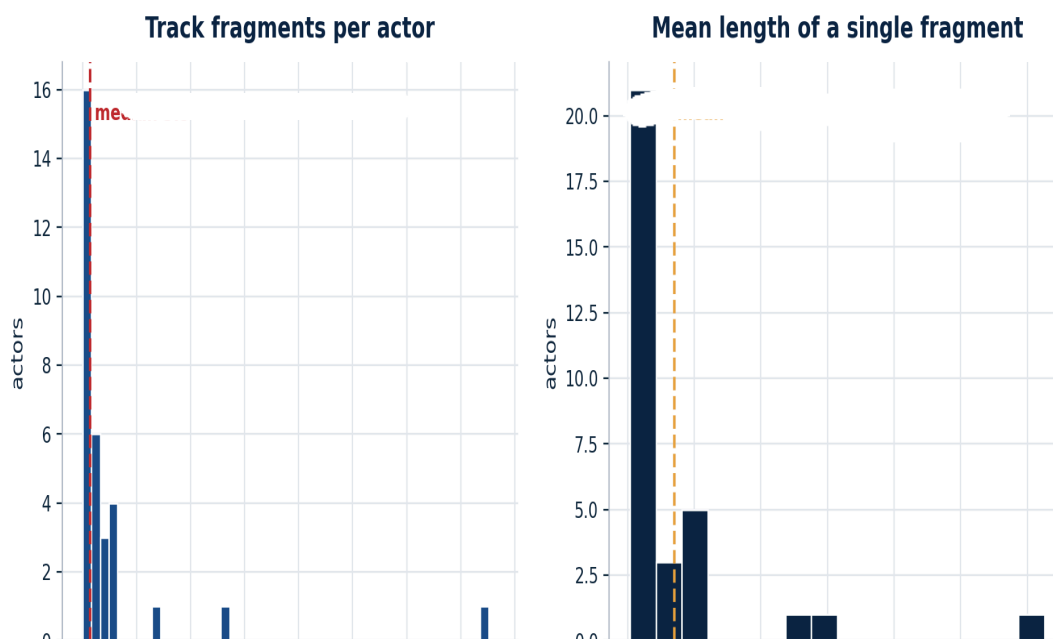


Figure 5.13: Observation duration comparison. The mean time an actor is observed collapses from 125.02 s in ground truth to 25.18 s when tracked, because the radar’s shallow wedge and the tracker’s fragmentation see each actor only briefly and intermittently.

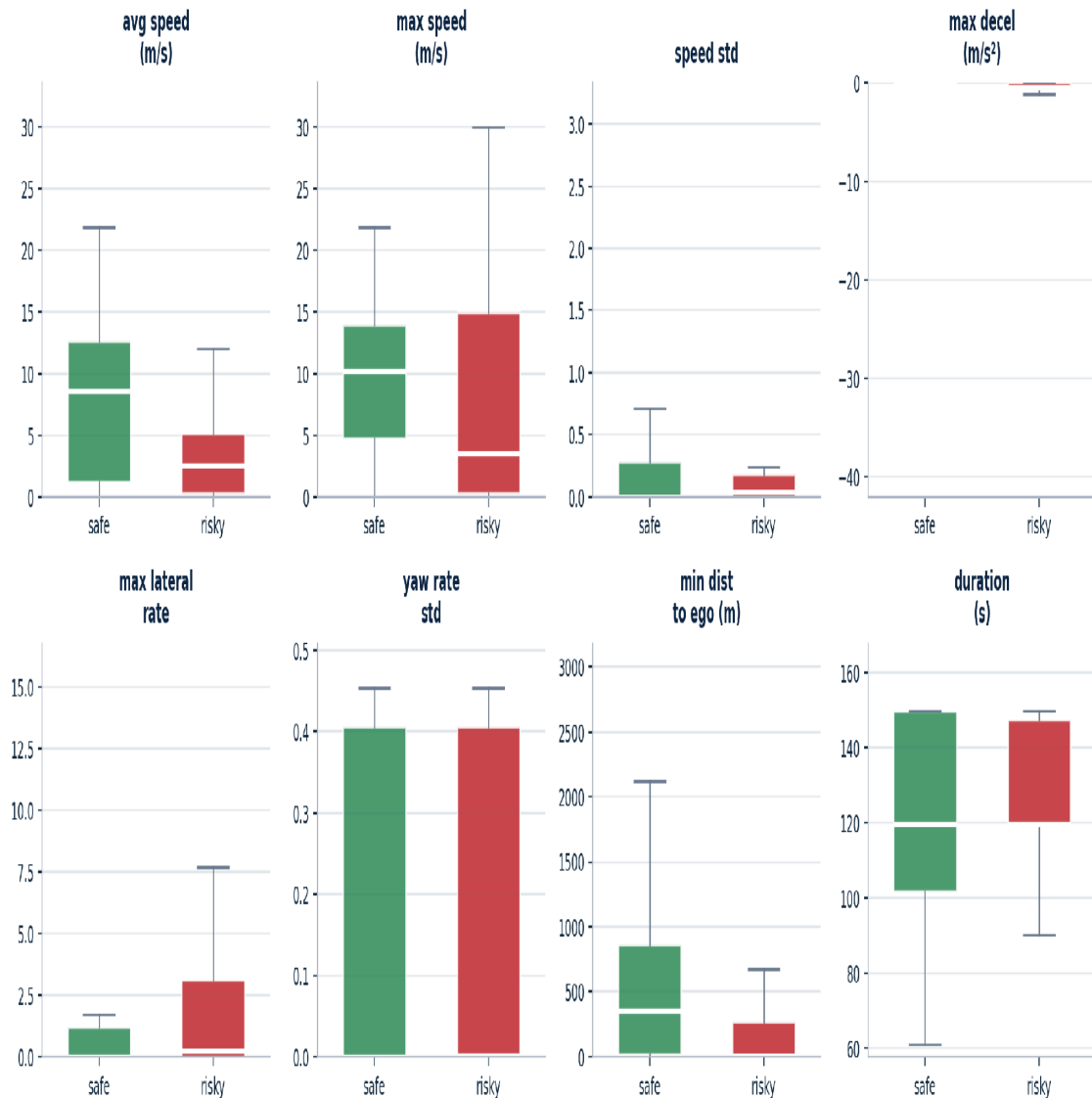


Figure 5.14: Track fragmentation distribution: a single real actor is frequently split across several short-lived tracks, the direct mechanism behind the collapse in observed duration and, through it, the corruption of the duration dependent motion features.

The consequences propagate straight into the features. Several of the ten motion features depend on observing an actor for long enough the duration feature itself, of course, but also the maxima and standard deviations of speed and acceleration, which cannot register an event the sensor never saw. When an actor is observed for a fifth of its true lifetime, in broken fragments, these features are computed over the wrong, truncated windows and take values that no longer match what the classifier learned. Figure 5.16 makes this corruption visible, plotting ground-truth against sensor tracked feature values and showing how far the tracked values drift from the diagonal of perfect agreement.

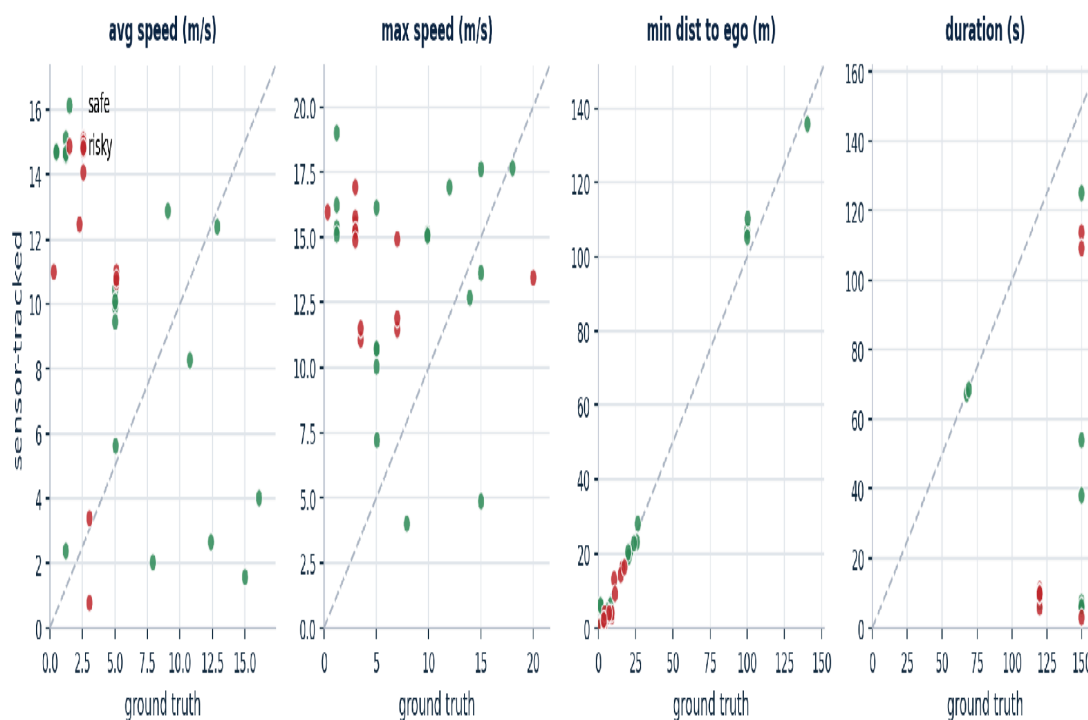


Figure 5.16: Feature corruption under tracking: ground truth feature values (horizontal) against the same features re-derived from sensor tracked data (vertical). Points far from the diagonal are features whose values the sensing pipeline has distorted, the duration dependent statistics most of all.

It is instructive to put a number on the mechanism. If an actor truly present for around 125 seconds is observed for only about 25, then roughly four-fifths of its motion history is simply unavailable to the feature extractor. The features that summaries extremes the maximum speed, the hardest deceleration are then computed from a fifth of the evidence, and on average they will understate those extremes, because the single largest values are more likely than not to lie somewhere in the unseen majority of the trajectory. A hard brake that happened to occur outside the observation window is, as far as the classifier can tell, a brake that never happened at all, and an actor stripped of its one risky moment reads as safe. Worse still, fragmentation does not excise a random fifth of the trajectory but a contiguous, sensor determined portion of it whatever fell inside the forward wedge while the geometry happened to be favorable so the surviving sample is not merely smaller but systematically biased, and no amount of averaging over more actors can correct a bias that pushes every one of them in the same direction. This is why accuracy under tracking does not merely become noisier but falls, and falls almost the whole way to chance.

The diagnosis is therefore precise, and it is worth stating carefully because it changes the moral of the result. The classifier did not fail because its logic was wrong; on ground truth that same logic is perfect. It failed because the inputs it received in the tracked condition were systematically corrupted by partial, fragmented observation, and because a meaningful share of actors never cleared the four-sample guard at all and so were never classified. The weakness exposed by the robustness test lies in the interaction between duration dependent

features and a limited, single-sensor perception stack not in the classifier itself. That distinction is what points directly to the remedies proposed in Chapter 6.

5.4 Error Analysis

The headline figure of 53.1% is a single number laid over two quite different kinds of failure, and separating them is essential both to understanding the result and to repairing it. When an actor that is risky in the world is not flagged as risky by the system, the failure can take one of two forms: the system may have declined to judge the actor at all, or it may have judged it and judged it wrongly. These are the two error channels abstention and misclassification and they have different causes, different consequences, and different remedies.

5.4.1 Abstentions versus Misclassifications

An abstention arises from the four-sample guard described in Chapter 4. Before an actor's tracked trajectory is turned into a feature row, it must contain at least a minimum number of samples; a trajectory shorter than that is discarded rather than classified, because the motion statistics computed from a handful of points would be meaningless. On complete ground truth trajectories almost, every actor clears this guard comfortably, but under fragmented tracking a large share of actors precisely those seen only briefly, at the edge of the radar wedge, in unfavorable geometry fall below the threshold and are never classified at all. An abstention is not a wrong answer but an absent one, and to that extent it is the honest outcome: the system is declining to judge on evidence it considers insufficient. Yet from the standpoint of a vehicle that must decide how to act, an actor that is never classified is an actor that is never monitored, and the safety consequence of a silent abstention on a genuinely risky neighbor is no better than that of an outright error.

A misclassification, by contrast, arises among the actors that do clear the guard and are assigned a label that turns out to be wrong. The analysis of the previous section establishes the dominant direction of these errors: because truncated observation systematically removes the extreme events the hard brake, the sharp swerve by which a risky actor betrays itself, the characteristic misclassification is a risky actor read as safe rather than the reverse. The two channels therefore compound rather than cancel. The contrast with the ground truth condition is decisive: there, abstention is negligible because trajectories are complete, and misclassification is essentially absent because the classifier scores a perfect hundred per cent, so virtually the entire shortfall observed under tracking is attributable to the introduction of these two mechanisms by the sensing pipeline, not to any defect the classifier carried into the experiment.

5.4.2 The Safety Critical Asymmetry

Treating all errors as equivalent, as a single accuracy figure implicitly does, obscures the most important feature of the result. The two possible misclassifications are not symmetric in their cost. A safe actor mistaken for a risky one is a false alarm: it makes the system

needlessly conservative, perhaps braking or hesitating where it need not, at a cost measured in comfort and efficiency. A risky actor mistaken for a safe one is a false negative: it leaves a genuine hazard unflagged, at a cost measured in safety. Any monitoring system for a safety critical application must be judged first on the second kind of error [37], and it is exactly the second kind that fragmented observation produces, because stripping an actor of its extreme moments can only ever make it look safer than it is, never more dangerous.

When the two channels are read together, the errors are seen to concentrate in the worst possible place. Abstentions fall disproportionately on the briefly seen, heavily fragmented actors; false-negative misclassifications fall disproportionately on the actors whose risky moments were truncated away; and these are largely the same population the neighbors in fast changing, close-range, high-risk geometry whose trajectories the forward radar captures least completely. The residual 53.1% is therefore drawn substantially from the easy safe cases that survive truncation intact, while the hard risky cases are precisely those lost to abstention and to false negative error. Read in this light the accuracy figure, far from overstating the severity of the problem, understates it: the system is weakest exactly where a monitoring system most needs to be strong. This is the sharpest possible statement of why the remedies developed in the next chapter features that survive truncation, a wider or fused field of view that shortens the unobserved intervals, and online classification that commits early rather than waiting for a trajectory that may never complete are not incremental refinements but responses to a structural weakness the error analysis has now localized.

5.5 Discussion

Taken together, the results tell a coherent and deliberately unflattering story. A behaviors classifier can be trained to high accuracy on motion features but only after a leak that inflated an early result to near perfection was found and removed, and only when evaluated on the clean, complete trajectories it was trained on. Confronted with the fragmentary output of a realistic radar and tracker, the same classifier degrades to near chance performance, for reasons that the analysis is able to name exactly: the collapse of observation duration through track fragmentation, and the consequent corruption of duration dependent features.

It is worth addressing the majority class baseline directly, because it is easy to misuse [37]. A naive classifier that always predicted the more common label would achieve an accuracy equal to that label's prevalence, and an earlier account of this project quoted a figure of 79.5% for such a baseline. That specific figure could not be reproduced from the recomputed dataset and is therefore not relied upon here; what can be said with confidence is that the sensor tracked accuracy of 53.1% sits only marginally above the 50% chance level of the binary problem, and that this alone is sufficient to establish the severity of the degradation. The honest comparison is not against a contested baseline but against the classifier's own 100% on ground truth from the very same scenarios.

Before these numbers are pressed too hard, it is worth stating candidly what could undermine them and what could not. The external validity of the robustness figure is genuinely limited:

the radar is an idealized model that does not reproduce multipath, clutter, or the target dependent radar cross section of a real sensor [36], [43], the study models a single forward radar rather than the fused multi sensor suite a production vehicle would carry, and the classifier is a compact decision tree rather than a temporal model that might tolerate gaps more gracefully [23], [42]. Each of these could move the tracked accuracy number up or down if it were changed. What none of them threatens is the internal validity of the central comparison, because the same feature extraction routine is applied to the ground truth and the tracked trajectories of the very same actors in the very same scenarios; the drop from complete to fragmentary observation is therefore measured under otherwise identical conditions, and it is that controlled contrast, not the absolute value of either number, that the project relies on.

Read this way, the degradation is not a quirk of one classifier but a structural consequence of how the features are built. Any system that characterizes an actor by summarizing its motion over an observation window inherits the same vulnerability [29]: when the sensor truncates that window, every duration-dependent statistic is computed over too little evidence, and the summary drifts away from the quantity it was meant to estimate. The lesson therefore generalizes beyond decision trees and beyond radar. It also points to concrete remedies, which Chapter 6 develops: features that are robust to truncation because they describe instantaneous rates rather than whole-trajectory aggregates; sensor fusion or a wider field of view to extend the observation window; and online classification that commits to a judgement early and updates it, rather than waiting for a complete trajectory that a real sensor may never deliver.

The value of this project lies precisely in its willingness to report that fall rather than to stop at the flattering 96.0%. Many studies would have ended at the confusion matrix; this one deliberately built an adversarial perception stack in order to find out what would happen next, and what happened next is the most useful thing the project has to say. The gap between 96.0% and 53.1% is a quantified statement of how much a behaviors classifier can be misjudged if it is never tested against realistic sensing and it is a warning that carries well beyond this particular system.

CHAPTER 6: CONCLUSION

This report set out to build a system that could judge the behaviors of the objects around an autonomous vehicle as safe or risky, and its distinguishing ambition to measure honestly how well that judgement survives contact with a realistic, imperfect sensing and tracking stack. This concluding chapter assesses what was achieved against the objectives declared at the outset, states the limitations of the work candidly, sets out the future work that would carry it towards deployment, and closes with a few remarks on what the project ultimately demonstrates.

6.1 Summary of Achievements

Chapter 1 committed the project to five concrete objectives. Each was met, and the evidence for each has been presented in the preceding chapters. A dataset of 450 labelled scenarios across seven road network types was generated programmatically, yielding 3,464 valid feature rows from 3,615 actors. Each actor's trajectory was reduced to ten interpretable motion features and used to train a decision tree classifier [17], [18]. A data leakage investigation revealed that an early near perfect model was reading behaviors names from metadata rather than learning motion; removing the four offending columns and retraining on motion alone produced a leakage free classifier scoring 95.4% under cross validation and 96.0% on held out scenarios. That classifier was then confronted with radar tracked data and its accuracy shown to fall to 53.1%, a collapse traced precisely to track fragmentation and the resulting corruption of duration dependent features. Finally, the classifier was embedded in a live loop that produced 5,832 real time predictions across 18 actors while actively shaping the ego vehicle's driving. Table 6.1 records this objective by-objective outcome.

Table 6.1: The objectives set in Chapter 1 and the degree to which each was met.

Objective	Outcome	Evidence
1. Generate labelled scenario dataset	Met	450 scenarios, 3,464 feature rows (5.1)
2. Extract features and train classifier	Met	Ten features, fine decision tree (4, 5.3.2)
3. Detect and eliminate data leakage	Met	$\approx 100\% \rightarrow 96.0\%$ after removing 4 columns (5.3.1)
4. Evaluate robustness under sensing	Met	$100\% \rightarrow 53.1\%$; cause identified (5.3.3)
5. Demonstrate closed-loop operation	Met	5,832 live predictions, 18 actors (5.2)

The achievement of which the project is most proud is not the 96.0% but the 53.1%. It would have been easy, and common, to report the former and stop. Choosing instead to build an

adversarial perception stack and to publish the fall is what gives the work its value: it converts an untested assumption into a measured, explained fact.

6.2 Limitations

The work has real limitations, and stating them plainly is part of reporting it honestly. They fall into three groups.

The first concerns sensing. The system models a single forward-looking radar. This was a deliberate choice that made the observation coverage problem vivid, but it also means that the robustness figures reflect a worst-case single sensor configuration rather than the multi-sensor suite a production vehicle would carry. A vehicle fusing several radars, cameras, and a lidar would observe each actor for longer and more continuously [36], [43], and the accuracy fall would very likely be smaller than the one reported here.

The second concerns the model and features. The classifier is a decision tree over ten hand designed features, several of which depend on observation duration and are therefore intrinsically fragile under fragmentation. This fragility is precisely what the robustness test exposed. A model built on features that do not assume a long, continuous observation or one that reasons explicitly over partial sequences would be expected to degrade more gracefully [23], [42]. The interpretability that the decision tree buys come at the cost of an inability to model interaction or long-range temporal structure [44].

The third concerns validation. Everything reported here was produced in simulation. Simulation is what makes the perfect ground truth and therefore the whole robustness comparison possible [35], but it also means the results inherit the reality gap between a synthetic radar model and a physical sensor on a real road. The numbers should be read as a rigorous statement about the simulated system, and as a strong hypothesis, not a proof, about a physical one. In addition, the robustness figure of 53.1% was computed on a mixed sample of ten scenarios rather than the entire dataset, because the full sensor and tracking simulation is computationally expensive; the sample was chosen to be representative, but a larger sample would tighten the estimate.

These three limitations interact in a way that is worth naming, because it guards against a facile reading of the headline figure. The single sensor choice, the duration dependent features, and the simulation only validation each shape the 53.1%, and they do not all push in the same direction. The single forward radar and the fragile, duration dependent features conspire to make that number a pessimistic estimate of what a better equipped or better featured system would achieve. The perfectly known ego poses and the noise free label generation, by contrast, make it an optimistic estimate of what a physical vehicle contending with its own localization error and with messier data than any simulator produces would see. The honest position is therefore not that 53.1% is the accuracy a real system would obtain, but that it is a controlled measurement of one specific, well understood configuration, whose assumptions have been stated plainly enough that a reader can reason about which way each

of them would move the result. A single number quoted without those assumptions would be worth far less than the qualified one offered here.

6.3 Future Work

The limitations map directly onto the most promising avenues for future work. The robustness analysis does more than merely list these avenues, however; it ranks them, because it identifies short and fragmented observation not any defect of the classifier as the mechanism of failure, and so points to the extension and stabilization of observation as the highest value directions. The five subsections below set out the programmed in the order that this mechanism suggests, beginning with the interventions that attack the identified cause most directly and least expensively.

6.3.1 Multi Sensor Fusion

The single forward-looking radar, with its shallow one hundred and forty degree by five-degree wedge, is the ultimate source of the limited coverage that drives the robustness collapse. Adding further radars to the rear and sides, together with a camera or a lidar, and fusing their detections before the tracking stage [36], would lengthen and stabilize each actor's observed trajectory by keeping it within some sensor's field of view for far more of its lifetime. Because the collapse was shown to be driven by short, fragmented observation rather than by any flaw in the classifier, extending coverage in this way is expected to be the single most effective remedy available. Its benefit is also directly measurable with the apparatus already built: one need only widen the sensor suite, re-run the identical ground truth versus tracked comparison, and watch the tracked accuracy recover from its present level toward the ground truth ceiling, with the size of that recovery quantifying exactly how much of the degradation was attributable to coverage alone.

6.3.2 Duration Independent Feature Design

The mechanism identified in Chapter 5 is that features which aggregate motion over a whole trajectory the maxima and standard deviations of speed and acceleration, and the duration feature itself are computed over truncated, fragmentary windows once observation becomes partial, and so drift away from the values on which the classifier was trained. Redesigning the feature set so that it no longer presumes a long, continuous observation would attack this mechanism head-on. Instantaneous rate-based descriptors, statistics computed over short fixed-length sliding windows [30], [31], quantities normalized by the observed duration, or running estimates updated sample by sample would all degrade gracefully as observation shortens rather than collapsing abruptly. This direction is especially attractive because, unlike sensor fusion, it requires no change whatever to the hardware and can be evaluated immediately on the tracked dataset already in hand, which makes it the cheapest of the high value interventions.

6.3.3 Sequence Models over Partial Tracks

A temporal model a recurrent network, a temporal convolution, or an attention-based sequence model [23], [24], [42] operating on the per-sample track rather than on ten summary scalars could in principle learn to make confident judgements from short and interrupted observations, and to represent its own uncertainty explicitly while an actor is still being observed. The cost of this power is the transparency that the present decision tree enjoys, and which Section 2.1.6 argued was load-bearing for the diagnostic work of this project [44]: a recurrent classifier would not offer the axis aligned, human readable thresholds that made the failure analysis possible. A careful comparison would therefore have to weigh accuracy under truncation against interpretability, and a hybrid arrangement an interpretable surrogate distilled from a temporal backbone might recover a useful measure of both rather than forcing a choice between them.

6.3.4 Track Stitching and Re-Identification

Rather than tolerating fragmentation downstream, a fourth direction attacks it at its source. Logic to re-associate the several short-lived tracks of a single physical object using motion model gating to bridge brief gaps [5], [40], and appearance or re-identification cues where a camera is available [9], [10] would stitch those fragments back into one continuous track, restoring the observation duration that fragmentation destroys without changing the sensor at all. The fragmentation count instrumentation introduced in Section 4.4.3, which already measures how many tracks each real actor was split across, provides a ready-made metric for such work: success would show up directly as a fall in that count and a corresponding recovery in the mean observed duration, both of which the existing pipeline reports without modification.

6.3.5 Real Vehicle Validation

The final and most demanding direction is to test how far the simulated findings transfer to physical hardware. The radar model used here is idealized: it omits the multipath, clutter, and target dependent radar cross section of a real sensor [43], and, as Section 4.4.2 noted, it assumes the ego vehicle's own pose is known exactly, whereas a real vehicle must estimate its own position as imperfectly as it estimates its neighbors'. Replaying recorded real sensor datasets through the same feature extraction and classification pipeline [11], [12], [13], or ultimately deploying it onto an instrumented vehicle, would measure how much of the degradation observed in simulation survives contact with reality and would close the reality gap acknowledged throughout this report. Because the report has been careful to treat the simulated tracked accuracy as an optimistic bound, the honest expectation is that real vehicle performance would be no better, and quite possibly worse which is exactly why this validation is the ultimate test of the entire line of work.

These five directions are not independent items on a wish-list but a connected programmed, and the robustness analysis suggests an order in which to pursue them. The first two

extending sensor coverage and redesigning the features to tolerate truncation attack the identified cause most directly, and each could be evaluated with the very apparatus already built, simply by re-running the ground truth versus tracked comparison and watching the gap narrow. The remaining three are larger undertakings that trade some of the present system’s simplicity, or its simulation only convenience, for greater realism. Taken together they describe a credible path from the honest but limited demonstration reported here toward a system whose behaviors judgements might eventually be trusted on a road. Importantly, none of them requires discarding what has been built; each extends it along the single axis that the robustness test identified as weakest, which is itself a vindication of having run that test rather than stopping at the flattering headline accuracy.

6.4 Concluding Remarks

The central lesson of this project is methodological as much as technical. A behaviors classifier that appears excellent on clean data can be little better than a coin toss once it must rely on realistic perception, and the only way to know is to build the perception stack and measure the fall. By generating its own perfectly labelled data, auditing that data for leakage, and then deliberately degrading it through a simulated radar and tracker, this project turned a comfortable 96.0% into an uncomfortable but far more useful 53.1%, and was able to say exactly why the difference arose. The system is not ready for a road, and this report has been careful never to claim otherwise. But it stands as a small, honest demonstration of how safety critical machine learning ought to be evaluated [22], [44]: not by the accuracy one can report, but by the accuracy that survives contact with reality.

BIBLIOGRAPHY

- [1] SAE International, “J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles,” Standard J3016_202104, Warrendale, PA, 2021.
- [2] World Health Organization, “Global Status Report on Road Safety 2018,” WHO, Geneva, 2018.
- [3] National Highway Traffic Safety Administration, “Automated Driving Systems 2.0: A Vision for Safety,” U.S. Department of Transportation, Washington, DC, 2017.
- [4] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [5] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York: Wiley, 2001.
- [6] S. S. Blackman and R. Popoli, *Design and Analysis of Modern Tracking Systems*. Norwood, MA: Artech House, 1999.
- [7] G. Welch and G. Bishop, “An Introduction to the Kalman Filter,” Technical Report TR 95-041, University of North Carolina at Chapel Hill, 1995.
- [8] S. J. Julier and J. K. Uhlmann, “Unscented Filtering and Nonlinear Estimation,” *Proceedings of the IEEE*, vol. 92, no. 3, pp. 401–422, 2004.
- [9] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple Online and Realtime Tracking,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2016, pp. 3464–3468.
- [10] N. Wojke, A. Bewley, and D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2017, pp. 3645–3649.
- [11] A. Geiger, P. Lenz, and R. Urtasun, “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3354–3361.
- [12] H. Caesar et al., “nuScenes: A Multimodal Dataset for Autonomous Driving,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11621–11631.
- [13] P. Sun et al., “Scalability in Perception for Autonomous Driving: Waymo Open Dataset,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 2446–2454.
- [14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [15] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 91–99.

- [16] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 12697–12705.
- [17] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.
- [18] J. R. Quinlan, “Induction of Decision Trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [19] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [20] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009.
- [21] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in Data Mining: Formulation, Detection, and Avoidance,” *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [22] S. Kapoor and A. Narayanan, “Leakage and the Reproducibility Crisis in Machine-Learning-Based Science,” *Patterns*, vol. 4, no. 9, 100804, 2023.
- [23] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, “Social LSTM: Human Trajectory Prediction in Crowded Spaces,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 961–971.
- [24] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, “Social GAN: Socially Acceptable Trajectories with Generative Adversarial Networks,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 2255–2264.
- [25] N. Deo and M. M. Trivedi, “Convolutional Social Pooling for Vehicle Trajectory Prediction,” in *Proc. IEEE/CVF CVPR Workshops*, 2018, pp. 1468–1476.
- [26] A. Kuefler, J. Morton, T. Wheeler, and M. Kochenderfer, “Imitating Driver Behavior with Generative Adversarial Networks,” in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2017, pp. 204–211.
- [27] D. Sadigh, S. Sastry, S. A. Seshia, and A. D. Dragan, “Planning for Autonomous Cars that Leverage Effects on Human Actions,” in *Proc. Robotics: Science and Systems (RSS)*, 2016.
- [28] C. Hubmann, J. Schulz, M. Becker, D. Althoff, and C. Stiller, “Automated Driving in Uncertain Environments: Planning with Interaction and Uncertain Maneuver Prediction,” *IEEE Trans. Intelligent Vehicles*, vol. 3, no. 1, pp. 5–17, 2018.
- [29] S. Lefèvre, D. Vasquez, and C. Laugier, “A Survey on Motion Prediction and Risk Assessment for Intelligent Vehicles,” *ROBOMECH Journal*, vol. 1, no. 1, pp. 1–14, 2014.
- [30] J. C. Hayward, “Near-Miss Determination Through Use of a Scale of Danger,” *Highway Research Record*, no. 384, pp. 24–34, 1972.
- [31] K. Vogel, “A Comparison of Headway and Time to Collision as Safety Indicators,” *Accident Analysis & Prevention*, vol. 35, no. 3, pp. 427–433, 2003.
- [32] The MathWorks, Inc., *Automated Driving Toolbox: User’s Guide (R2023b)*. Natick, MA, 2023.

- [33] The MathWorks, Inc., Sensor Fusion and Tracking Toolbox: User's Guide (R2023b). Natick, MA, 2023.
- [34] The MathWorks, Inc., Statistics and Machine Learning Toolbox: User's Guide (R2023b). Natick, MA, 2023.
- [35] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, "CARLA: An Open Urban Driving Simulator," in Proc. Conf. on Robot Learning (CoRL), 2017, pp. 1–16.
- [36] H. Cho, Y.-W. Seo, B. V. K. V. Kumar, and R. R. Rajkumar, "A Multi-Sensor Fusion System for Moving Object Detection and Tracking in Urban Driving Environments," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2014, pp. 1836–1843.
- [37] D. M. W. Powers, "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation," Journal of Machine Learning Technologies, vol. 2, no. 1, pp. 37–63, 2011.
- [38] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [39] J. Munkres, "Algorithms for the Assignment and Transportation Problems," Journal of the Society for Industrial and Applied Mathematics, vol. 5, no. 1, pp. 32–38, 1957.
- [40] D. B. Reid, "An Algorithm for Tracking Multiple Targets," IEEE Trans. Automatic Control, vol. 24, no. 6, pp. 843–854, 1979.
- [41] C. Schöller, V. Aravantinos, F. Lay, and A. Knoll, "What the Constant Velocity Model Can Teach Us About Pedestrian Motion Prediction," IEEE Robotics and Automation Letters, vol. 5, no. 2, pp. 1696–1703, 2020.
- [42] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [43] D. Feng et al., "Deep Multi-Modal Object Detection and Semantic Segmentation for Autonomous Driving: Datasets, Methods, and Challenges," IEEE Trans. Intelligent Transportation Systems, vol. 22, no. 3, pp. 1341–1360, 2021.
- [44] C. Rudin, "Stop Explaining Black Box Machine Learning Models for High-Stakes Decisions and Use Interpretable Models Instead," Nature Machine Intelligence, vol. 1, pp. 206–215, 2019.
- [45] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA: MIT Press, 2005.
- [46] M. Treiber, A. Hennecke, and D. Helbing, "Congested Traffic States in Empirical Observations and Microscopic Simulations," Physical Review E, vol. 62, no. 2, pp. 1805–1824, 2000.

APPENDIX A: MATHEMATICAL FORMULATIONS

This appendix collects the mathematical formulations underlying the tracking and risk assessment components referred to throughout the report. They are standard results, reproduced here in the notation used by the implementation so that the report is self-contained.

A.1 The Extended Kalman Filter

The multi object tracker maintains, for each track, a state estimate and its error covariance, and updates them recursively as detections arrive. With the planar constant velocity state written as position and velocity in each axis, the prediction step advances the state and covariance from one time step to the next:

$$\hat{x}_{k|k-1}^- = F \cdot \hat{x}_{k-1|k-1} \quad (\text{A.1})$$

$$P_{k|k-1}^- = F \cdot P_{k-1|k-1} \cdot F^T + Q \quad (\text{A.2})$$

where F is the state-transition matrix, Q the process-noise covariance, and the minus superscript denotes a predicted (prior) quantity. For the constant velocity model with state $[x \ y \ v_x \ v_y]$ and time step Δt , the transition matrix is

$$F = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (\text{A.3})$$

When a detection z arrives, the filter forms the innovation the difference between the measurement and its prediction and uses it to correct the state:

$$\tilde{y}_k = z_k - h(\hat{x}_{k|k-1}^-) \quad (\text{A.4})$$

$$S_k = H_k \cdot P_{k|k-1}^- \cdot H_k^T + R \quad (\text{A.5})$$

$$K_k = P_{k|k-1}^- \cdot H_k^T \cdot S_k^{-1} \quad (\text{A.6})$$

$$\hat{x}_k = \hat{x}_{k|k-1}^- + K_k \cdot \tilde{y}_k \quad (\text{A.7})$$

$$P_k = (I - K_k \cdot H_k) \cdot P_{k|k-1}^- \quad (\text{A.8})$$

Here R is the measurement noise covariance, S_k the innovation covariance, and K_k the Kalman gain. Because the radar measures range, azimuth, and range rate nonlinear functions of the Cartesian state the measurement function h is nonlinear, and H_k is its Jacobian, the matrix of partial derivatives of h evaluated at the predicted state. Linearizing in this way is exactly what distinguishes the extended Kalman filter of equations (A.4), (A.8) from the linear filter, and it is the standard treatment given by Kalman [4], Welch and Bishop [7], and Bar-Shalom et al. [5].

A.2 Time to Collision

Time to collision is a widely used surrogate safety metric [30], [31] that estimates how long, at the current closing rate, before a following object reaches a lead object. For a target at range R approaching along the line of sight with range-rate $\dot{R}$ (negative when closing), it is defined as

$$TTC = R / (- \dot{R}), \quad for \dot{R} < 0 \quad (A.9)$$

with TTC taken to be infinite when the target is not closing. A small time to collision indicates imminent danger. The classifier developed in this project does not compute time to collision as an explicit input; it instead relies on the minimum and average distance to the ego and on the speed and deceleration features of Table 3.2, which together capture the same danger that time to collision summarizes. The formulation is included here because it is the canonical safety metric against which proximity-based risk is usually framed, and because the distance features the classifier does use are its close relatives.

A.3 Risk Determination and the Risky-Share Score

The behaviors decision itself is produced by the trained decision tree, which maps a feature vector to a binary label, with zero denoting safe and one denoting risky. The tree realizes this mapping as a sequence of axis-aligned threshold tests on the features for example, whether the minimum distance to the ego falls below a learned threshold while the speed exceeds another so that the decision for any actor can be traced as an explicit path of rules.

$$\rho = N_{risky} / N_{total} \quad (A.10)$$

where N_{risky} is the number of timesteps at which the actor was predicted risky and N_{total} the number at which it was classified at all. This score ranges from zero to one and is plotted per actor in the live-demonstration summary of Figure 5.8. It is important to note, as recorded in Chapter 5, that the actor's recorded final verdict is its last prediction rather than a threshold on ρ ; the risky share score is reported as a descriptive summary of the drive, not as the decision rule. An actor may therefore carry a modest ρ yet still be filed as risky if its final observed behaviors were risky.

A.4 The Process Noise and Measurement Models

Section A.1 set out the filter's recursion but left two matrices unspecified: the process-noise covariance Q that appears in the prediction (A.2), and the measurement Jacobian H that appears in the update (A.5), (A.6). This section states both in the forms the tracker uses, since together they determine how the filter balances its model against its measurements.

The process noise follows the discrete white noise acceleration model, in which the departure from constant velocity over one step is treated as an unknown constant acceleration drawn

from a zero mean distribution of variance σ^2 . For each axis the resulting contribution to Q is the two-by-two block

$$Q_{axis} = \sigma^2 \cdot \begin{bmatrix} \Delta t^4/4 & \Delta t^3/2 \\ \Delta t^3/2 & \Delta t^2 \end{bmatrix} \quad (\text{A.11})$$

and the full four-by-four Q is block-diagonal in the two axes. The single scalar σ therefore governs how readily the filter revises its velocity estimate: too small and it lags genuine maneuver's, too large and it chases measurement noise. Its magnitude is left at the level inactive derives from the detection accuracy, so that the model uncertainty and the measurement uncertainty are kept commensurate rather than tuned by hand.

The radar reports range, azimuth, and range-rate, which are nonlinear functions of the Cartesian state $[x \ v_x \ y \ v_y]$. Writing $r = \sqrt{(x^2+y^2)}$ for the range, the measurement function is

$$h(x) = [r; \text{atan2}(y, x); (x \cdot v_x + y \cdot v_y) / r] \quad (\text{A.12})$$

and the Jacobian H of equations (A.5), (A.6) is the matrix of its partial derivatives evaluated at the predicted state,

$$H = \begin{bmatrix} x/r & 0 & y/r & 0 & -y/r^2 & 0 & x/r^2 & 0 \\ \partial \dot{r} / \partial x & x/r & \partial \dot{r} / \partial y & y/r \end{bmatrix} \quad (\text{A.13})$$

in which the range row depends only on position, the azimuth row is the derivative of the arctangent, and the range rate row couples' position and velocity. Re-evaluating H at each predicted state rather than holding it fixed is precisely the linearization that makes the filter extended rather than linear, and it is what allows a Cartesian constant-velocity state to be corrected by measurements expressed in polar coordinates.

A.5 Data Association and Track Management

With a prediction available for every existing track, each radar update must decide which detection, if any, belongs to which track before the update equations can be applied. The tracker makes this decision in two stages: gating, which rules out implausible pairings, and assignment, which chooses among those that remain.

Gating uses the innovation and its covariance from equations (A.4), (A.5) to form the squared Mahala Nobis distance between a detection and a track's prediction,

$$d^2 = \tilde{y}^T \cdot S^{-1} \cdot \tilde{y} \quad (\text{A.14})$$

which measures the discrepancy in units of its own expected spread, so that a large but well anticipated innovation is penalized less than a small but surprising one. A detection to track pairing is admitted only if d^2 falls below the assignment threshold, set to 35 in this system; pairings above it are treated as impossible and never considered. Among the admitted pairings the tracker solves a global nearest neighbor assignment, choosing the one-to-one matching of detections to tracks that minimizes the total gated distance,

$$\text{minimize} = \sum d^2(n) \quad \text{subject to one detection per track} \quad (\text{A.15})$$

so that a locally attractive but globally inconsistent match can be rejected in favor of the arrangement that best explains the whole frame at once. This matters most in the multi-crossroad scenarios, where several actors pass close to one another and a greedy pairing would be prone to swapping their identities.

To make the gate concrete, suppose a track predicts a detection at 40 m range with an innovation covariance whose standard deviations are 1 m in range, half a degree in azimuth, and half a meter per second in range-rate. A candidate detection differing from that prediction by 3 m, one degree, and 1 m/s yields, by equation (A.14), a squared Mahala Nobis distance of about $3^2 + 2^2 + 2^2 = 17$ inside the gate of 35 and therefore eligible for assignment whereas a detection differing by 6 m in range alone already contributes 36 from that single term and is excluded outright. The threshold thus corresponds, loosely, to admitting a combined discrepancy of up to roughly six standard deviations: generous enough to retain a genuinely maneuvering target, yet tight enough to reject a detection belonging to a different actor a few meters away.

These gating and assignment choices are well matched to a radar updating at 10 Hz. At that rate a road user advances at most a meter or so between frames, so the predicted and measured positions stay close and the gate seldom has to arbitrate between comparably good matches.

Track lifecycle is then governed by two count-based rules. A tentative track is confirmed only once it has been assigned a detection in at least two of its first three opportunities the M of N confirmation rule with M and N set to two and three which suppresses spurious tracks born of isolated false detections. A confirmed track that subsequently goes unassigned for five consecutive updates is deleted, releasing its identity; this coasting allowance lets a track survive a brief occlusion or a missed detection without being prematurely discarded.

APPENDIX B: SYSTEM PARAMETERS

This appendix consolidates, for reference, the numerical parameters that define the dataset, the sensing and tracking stack, and the classifier and live demonstration configuration. All values were held fixed across the experiments reported in Chapter 5.

Table B.1: Dataset and train/test-split parameters.

Parameter	Value
Total scenarios	450
Road-network types	7
Labelled actors	3,615
Valid feature rows (after 4-sample guard)	3,464
Clean rows after removing leaky columns	3,360
Feature columns (clean)	10 + label
Train / test split	Scenario-level, 80 / 20
Random seed	42
Classes	Safe (0), Risky (1)

Table B.2: Radar and multi object tracker parameters (as configured in the source).

Parameter	Value
Radar field of view	140° azimuth $\times$ 5° elevation
Radar range limits	0 – 150 m
Radar update rate	10 Hz
Radar mounting	Front of ego (length/2, 0, 0)
Radar noise	Enabled (HasNoise = true)
Tracker filter	Constant-velocity EKF (initcvekf)
Assignment threshold	35
Confirmation threshold	2 of 3 detections
Deletion threshold	5 consecutive misses
Track state vector	[x vx y vy z vz]

Table B.3: Classifier and live demonstration parameters.

Parameter	Value
Classifier	Fine decision tree (CART)
Predictors	10 motion features
Minimum samples to classify	4
Re-classification interval (live)	every 2.0 s (20 steps)
Render interval (live)	every 0.5 s (5 steps)
Safe following time gap (live control)	2.0 s
Risky actor gap multiplier (live control)	1.8 $\times$
Live demo predictions logged	5,832 across 18 actors

APPENDIX C: THE DATASET

This appendix describes the generated dataset in more detail and shows additional scenario visualizations not included in the main text. The dataset is entirely synthetic, generated in MATLAB, and carries perfect ground truth labels by construction, which is what makes the robustness comparison of Chapter 5 possible.

Table C.1: The dataset at a glance.

Quantity	Value
Scenarios	450
Road-network types	7 (highway, rural, village, crossroad, multi-turn, rail crossing, mixed)
Complexity tiers	3 (basic, intermediate, complex)
Labelled actors	3,615
Actor types	Car, truck, bus, bicycle, pedestrian, child, train
Valid feature rows	3,464
Behavior classes	Safe, Risky

The dataset was grown in five batches, each adding a new kind of situation as the generators acquired new capabilities, and each continuing the scenario numbering so that earlier scenarios were never overwritten. Table C.2 records this composition. The earliest batches use fixed, scripted trajectories; the later batches are closed loop, with the ego vehicle and the background traffic driven reactively, and it is these later, more demanding batches that most exercise the forward radar’s coverage limits.

Table C.2: Composition of the dataset by generation batch, in order of scenario identifier.

Scenario IDs	Batch	Situations introduced
1 – 100	Baseline	Highway, rural, village, and mixed roads; scripted safe and risky maneuver’s
101 – 200	Baseline extended	Adds signalized crossroads and railway level crossings
201 – 250	Reactive	Closed loop car following (IDM), live traffic signals, pedestrian crossings
251 – 350	Complex reactive	Two three crossroads, multi-lane traffic, lane changes, turns and U-turns
351 – 450	Complex routed	Long multi-intersection routes with turns, obstacles, and lane changes

The full library of behaviors modules from which every actor’s ground truth label is drawn is reproduced in Table C.3. Each module names a specific real-world maneuver, fixes the actor type and the road contexts on which it is valid, and carries a hard safe or risky label. Because the label is a property of the module rather than a subsequent human judgement, it is exact

and perfectly consistent across all 450 scenarios the property that makes the ground truth accuracy of Chapter 5 a meaningful reference point.

Table C.3: The complete behaviors module library. Label 1 denotes risky, label 0 safe; the road context lists the road types on which each module is used.

Behaviour module	Actor	Label	Road context
aggressive_lane_change	Car	Risky	Highway
sudden_braking	Car	Risky	Highway, rural
tailgating	Car	Risky	Highway, rural
unsafe_overtake	Car	Risky	Rural
speeding_zigzag	Bicycle	Risky	Village, rural
child_chasing_ball	Child	Risky	Village
jaywalking_pedestrian	Pedestrian	Risky	Village, rural
bus_sudden_stop	Bus	Risky	Village, rural
truck_merging	Truck	Risky	Highway
animal_or_obstacle_crossing	Obstacle	Risky	Rural
running_red_light	Car	Risky	Crossroad
failure_to_yield	Car	Risky	Crossroad
pedestrian_jaywalk_intersection	Pedestrian	Risky	Crossroad
railway_dash_crossing	Car	Risky	Rail crossing
pedestrian_railway_dash	Pedestrian	Risky	Rail crossing
safe_lane_change	Car	Safe	Highway
safe_following	Car	Safe	Highway, rural
safe_crosswalk_pedestrian	Pedestrian	Safe	Village
safe_cyclist	Bicycle	Safe	Village, rural
safe_bus_stop	Bus	Safe	Village, rural
safe_intersection_stop	Car	Safe	Crossroad
safe_pedestrian_crosswalk_intersection	Pedestrian	Safe	Crossroad
safe_railway_wait	Car	Safe	Rail crossing

Fifteen of the twenty-three modules encode risky behaviors and eight encode safe behaviors, but this is not the class balance of the dataset itself: how often each module is instantiated, and how many actors each scenario carries, together determine the final balance shown in Figure 3.8, which is far closer to even. The library spans every actor type the project models car, truck, bus, bicycle, pedestrian, and child and every road type, so that no single class of road user or road geometry is systematically tied to one label. The distribution of scenario durations across the dataset is shown in Figure C.1.

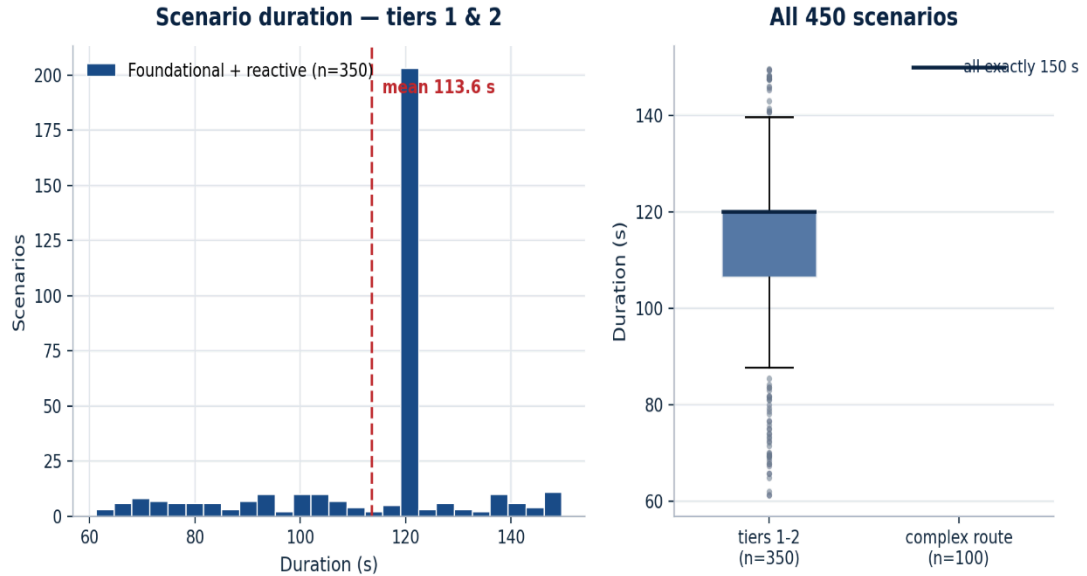


Figure C.1: Distribution of scenario durations across the dataset, the ground truth reference for the observation duration analysis of Chapter 5.

Figure C.2 shows a further scenario canvas, a village road, that complements the highway, crossroad, rail crossing, and multi-crossroad canvases shown in the main text, illustrating the breadth of road geometry the dataset spans.

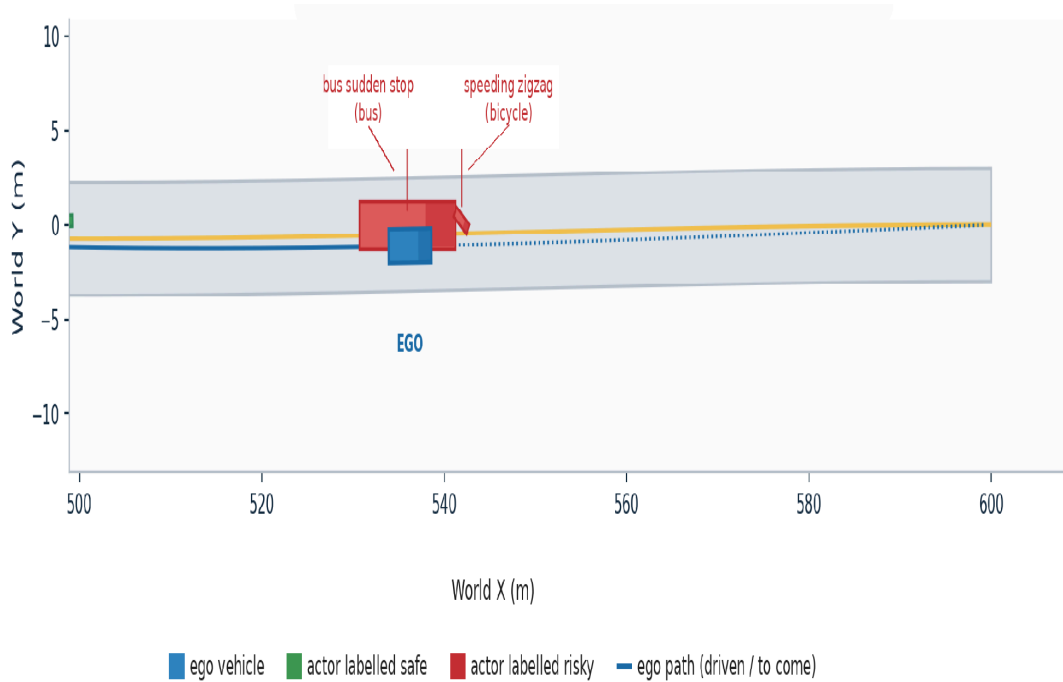


Figure C.2: A village road scenario canvas, with mixed slow traffic and vulnerable road users typical of the intermediate complexity tier.

APPENDIX D: MATLAB SOURCE CODE

This appendix reproduces the MATLAB source of the pipeline in full, so that every description given in Chapters 3 and 4 can be checked against the implementation rather than taken on trust. The listings are the genuine project source exactly as run; the comments are the author’s own, retained here because they document intent at the point of use. The code is grouped by role. Appendix D.1 covers feature extraction and dataset construction; D.2 the sensing, tracking, and robustness experiment; D.3 the baseline scenario generator; D.4 the reactive car following scenarios; D.5 the complex multi-crossroad route scenarios; D.6 the live closed loop demonstration; D.7 the batch drivers that produced the 450-scenario dataset; and D.8 the visualization and diagnostic utilities used during development. Together these files constitute the complete apparatus behind the results of Chapter 5.

A word on how to read the listings. The pipeline is deliberately organized as a set of small, single purpose functions that pass files between one another rather than as a monolithic program, so each listing can be understood largely on its own; where one function calls another, the callee appears in the same group or in an earlier one. The MATLAB Automated Driving, Sensor Fusion and Tracking, and Statistics and Machine Learning toolboxes supply the primitives `drivingScenario`, `drivingRadarDataGenerator`, `multiObjectTracker`, and the classification objects around which this code is written.

D.1 Feature Extraction and Dataset Construction

The four files in this group turn logged trajectories into the labelled feature table on which the classifier is trained. The shared feature extractor is the single most important function in the project, because it is applied identically to ground truth and tracked data and therefore guarantees that any accuracy gap between the two arises from the data and not from the code. *extractTrackFeatures.m*, the shared feature extractor, computes the ten motion features from a single trajectory and is applied identically to ground truth and sensor-tracked paths. The four-sample guard near the top is what causes short, fragmented tracks to drop out rather than yield meaningless statistics; that abstention is a significant part of the robustness loss analyzed in Section 5.3.3.

```
function feat = extractTrackFeatures(t, x, y, speed, egoX, egoY)
t = t(:); x = x(:); y = y(:); speed = speed(:);
if numel(t) < 4 % four-sample guard: abstain, don't fake it
    feat = table(NaN,NaN,NaN,NaN,NaN,NaN,NaN,NaN,NaN,NaN, ...
        'VariableNames', ...
            {'avg_speed','max_speed','speed_std','max_accel','max_decel',
             'max_lateral_rate','yaw_rate_std','min_dist_to_ego','avg_dist_to_ego','duration'});
else
    return
end
dt = diff(t); dt(dt <= 0) = median(dt(dt > 0));
accel = diff(speed) ./ dt;
lateralRate = abs(diff(y)) ./ dt;
yawRate = diff(atan2(diff(x), diff(y))) / dt(2:end); % heading rate
```

build_feature_dataset.m combines all 450 scenarios into one feature table, one row per labelled actor, ready for the Classification Learner. It reads each scenario's trajectory log, actor label file, and actor-name map, applies the shared extractor to every actor, and concatenates the results, carrying the ground-truth label and the metadata columns whose later removal is the subject of the leakage investigation.

build_clean_feature_dataset.m, the leakage free builder, rebuilds the feature dataset with the leaking metadata columns removed, keeping only the ten genuine motion features and the label, and drops rows with missing values. This is the dataset on which the reported 95.4% cross-validated and 96.0% held-out figures were obtained.

```
matFiles = [dir(fullfile(datasetDir,'scenario_*.mat')); ...
    dir(fullfile(datasetDir,'egofollow_scenario_*.mat'))];
rng(42); n = numel(matFiles);
shuffledIdx = randperm(n);
trainIdx = shuffledIdx(1:round(0.8*n)); testIdx =
shuffledIdx(round(0.8*n)+1:end);
splitLabel = repmat({'train'}, n, 1); splitLabel(testIdx) = {'test'};

allRows = {};
for k = 1:numel(matFiles)
    data = load(fullfile(matFiles(k).folder, matFiles(k).name));
    gtLog = getGroundTruthActorLog(data.scenario);
    egoTraj = gtLog(strcmp(gtLog.ActorName, 'Ego'), :);
    for r = 1:height(labelsTable)
        gt = gtLog(strcmp(gtLog.ActorName, labelsTable.name{r}), :);
        if height(gt) < 4, continue; end
        egoX = interp1(egoTraj.Time, egoTraj.X, gt.Time, 'linear', 'extrap');
        egoY = interp1(egoTraj.Time, egoTraj.Y, gt.Time, 'linear', 'extrap');
        f = extractTrackFeatures(gt.Time, gt.X, gt.Y, gt.Speed, egoX, egoY);
        f.label = labelsTable.label(r); f.split = splitLabel(k);
        allRows{end+1} = f;
    end
end
cleanFeatures = vertcat(allRows{:});
cleanFeatures = cleanFeatures(~any(ismissing(cleanFeatures), 2), :);
writetable(cleanFeatures(strcmp(cleanFeatures.split,'train'),:),
'features_clean_train.csv');
writetable(cleanFeatures(strcmp(cleanFeatures.split,'test'),:),
'features_clean_test.csv');
```

split_train_test_by_scenario.m splits the feature table into training and test sets at the scenario level rather than the row level, so that all actors from a given scenario are assigned as a group to one side of the split. This is the precaution that prevents near-identical actors from a shared scenario straddling the train/test boundary and inflating the reported accuracy.

```
T = readtable('features_dataset.csv');
uniqueScenarios = unique(T.ScenarioID);
rng(42);
shuffled = uniqueScenarios(randperm(numel(uniqueScenarios)));
numTrain = round(0.8 * numel(uniqueScenarios));
trainScenarios = shuffled(1:numTrain);
testScenarios = shuffled(numTrain+1:end);
trainTable = T(ismember(T.ScenarioID, trainScenarios), :);
testTable = T(ismember(T.ScenarioID, testScenarios), :);
writetable(trainTable, 'features_train.csv');
writetable(testTable, 'features_test.csv');
```

D.2 Sensing, Tracking, and the Robustness Experiment

This group implements the adversarial half of the project: it takes a saved scenario, views it through a simulated radar, tracks the returns, and re-derives features from the tracked trajectories so that they can be compared, actor for actor, against the ground truth. The robustness collapse reported in Chapter 5 is produced entirely by these four files.

demoSensorTracking.m radar simulation and multi-object tracking. Replays a saved scenario, simulates radar detections from the ego vehicle's point of view with a 140-degree azimuth, 150-metre forward radar, tracks the surrounding actors with a constant velocity multi-object tracker, and logs each track's estimated position and velocity over time. The radar and tracker parameters here are exactly those consolidated in Appendix B.

```
function trackLog = demoSensorTracking(scenarioMatFile)
    data = load(scenarioMatFile); scenario = data.scenario; restart(scenario);
    % identify ego actor by name; fallback to Actors(1)
    radarSensor = drivingRadarDataGenerator('RangeLimits',[0
150],'FieldOfView',[140 5],'HasNoise',true);
    tracker =
multiObjectTracker('FilterInitializationFcn',@initcvekf,'AssignmentThreshold'
,35,...);
    while advance(scenario)
        [detections,numDets,isValidTime] =
radarSensor(targetPoses(egoVehicle), scenario.SimulationTime);
        if isValidTime, tracks = tracker(detections,time); log pos/vel per track; end
    end
    trackLog =
cell2table(logRows,'VariableNames',{'Time','TrackID','X','Y','Z','Vx','Vy','Vz'});
end
```

buildTrackedFeatureDataset.m tracked versus ground truth features. Runs the sensing and tracking stack on one scenario, matches the resulting tracks back to the ground truth actors, extracts the same ten features from both the tracked and the ground truth trajectories, and joins the correct safe/risky label to each matched actor. Its output is the paired sample on which the robustness comparison is computed.

```
function [trackedFeatures,gtFeatures] =
buildTrackedFeatureDataset(scenarioMatFile,labelsCsvFile)
    trackLog = demoSensorTracking(scenarioMatFile);
    gtLog = getGroundTruthActorLog(load(scenarioMatFile).scenario);
    % rotate trackLog from ego-relative to world coords using ego heading
    matchTable = matchTracksToActors(trackLog, gtLog);
    for each matched actor:
        stitch track fragments -> extractTrackFeatures(...) -> fT
        extractTrackFeatures on ground-truth path -> fG
        join label/actorType/description from labelsCsvFile
    trackedFeatures = vertcat(fT rows, drop unlabeled); gtFeatures likewise;
end
```

matchTracksToActors.m nearest trajectory matching. Pairs each anonymous track with the ground truth actor whose interpolated path it follows most closely on average, so that a tracked feature row can inherit the correct label. Counting how many tracks match a single actor also measures directly how badly the tracker fragmented that actor's trajectory.

main_step4_tracked_robustness_test.m the robustness driver. The top-level script for the robustness experiment. It runs the sensing and tracking pipeline over a mixed sample of scenarios, extracts tracked and ground truth features for each, applies the trained classifier to both, and reports the accuracy on each the comparison that yields the fall from near perfect ground truth accuracy to 53.1% under tracking.

```

scenarioIDs = [5 10 15 20 25 30 205 210 355 360];
for id = scenarioIDs
    [tracked,gt] = buildTrackedFeatureDataset(matFile,labelsFile); accumulate
end
writetable(trackedFeatures,'features_tracked_noisy.csv');
writetable(gtFeatures,'features_groundtruth_recomputed.csv');
gtAcc = mean(trainedModel01.predictFcn(gtFeatures)==gtFeatures.label);
trackedAcc =
mean(trainedModel01.predictFcn(trackedFeatures)==trackedFeatures.label);
fprintf('Ground-truth accuracy: %.2f%% Tracked accuracy: %.2f%% Drop: %.2f
pp',...);

```

D.3 Baseline Scenario Generation

The remaining groups build the scenarios themselves. This first group generates the baseline scenarios: a road network is created, actors are populated from the behavior module library with fixed trajectories, and the whole scenario is played back and logged. These files produced the earliest and simplest tier of the dataset.

generateScenario.m

Builds one 120 scenarios with 5-6 mixed safe/risky complexity events, drawing actors (car, truck, bus, bicycle, pedestrian, child) from the behavior module library and attaching each to the road network. Every actor's ground truth label is fixed by the module it draws from.

```

function [scenario, gtTable, meta] = generateScenario(scenarioID, rngSeed)
[scenario, roadInfo] = createRoadNetwork(roadType);
lib = complexityLibrary(); % pick 5-6 valid events for this road type
% place ego with fixed trajectory along the road
for each chosen event: build actor, waypoints, speed profile
    trajectory(act, wp, spd);    end
% return scenario, ground-truth table, meta ; end

```

createRoadNetwork.m

Creates the drivingScenario and lays out road geometry for a requested road type highway, rural, village, mixed, crossroad, or rail_crossing returning the scenario plus a roadInfo struct (centers, speed limit, intersection/crossing points) used to place actors.

```
function [scenario, roadInfo] = createRoadNetwork(roadType)
    scenario = drivingScenario('StopTime', 120);
    switch roadType
        case 'highway'/'rural'/'village'/'mixed': road(scenario, centers, lanespec);
        case 'crossroad': two roads crossing, roadInfo.intersectionPoint
        case 'rail_crossing': road + roadInfo.crossingPoint
    end
end
```

complexityLibrary.m

Defines the library of named behavior modules actors draw from. Each entry has a name, actor type, valid road types, a fixed ground truth label (0=safe, 1=risky), and a description. This library is the origin of every label in the dataset (Figure 4.1).

```
function library = complexityLibrary()
    library(end+1) = struct('name','aggressive_lane_change', ...
        'actorType','car','roadTypes',{ 'highway' }, 'label',1, ...
        'description','Abrupt unsignaled lane change at speed. ');
    % ... ~20 more entries: safe + risky, across car/truck/bus/
    %   bicycle/pedestrian/child, and highway/rural/village/
    %   crossroad/rail_crossing event types
end
```

runAndRecordScenario.m

Advances a generated scenario to completion, logs every actor's pose at every timestep, and saves the per-scenario dataset (.mat + trajectory/label CSVs) into outDir for the feature extractor to consume.

```
function recordTable = runAndRecordScenario(scenario, gtTable, meta, outDir)
    while advance(scenario)
        poses = actorPoses(scenario); % log X,Y,Z,Vx,Vy,Vz,Yaw per actor
    end
    save([scenarioName '.mat'], scenario, gtTable, meta, recordTable);
    writetable(recordTable, ..._trajectories.csv);
    writetable(gtTable, ..._labels.csv);
end
```

runEgoFollowingScenario.m

Advances a scenario while manually driving the ego vehicle: its target speed each step is the minimum of a car following speed, a traffic signal stop speed, and a pedestrian yield speed. All other actors follow their own predefined trajectories.

```

function recordTable = runEgoFollowingScenario(scenario, ego, meta, outDir)
while advance(scenario)
    % 1) car-following speed from nearest lead vehicle
    % 2) signal speed: brake to stop before red stop line
    % 3) pedestrian-yield speed: stop if ped in crosswalk ahead
    speedTarget = min([lead, signal, ped, maxSpeed]);
    egoSpeed = clamp toward speedTarget by accel/decel limits;
    egoPos = egoPos + egoSpeed*dt; ego.Position = egoPos;
end
bakeEgoTrajectoryFromLog(ego, recordTable, 1);
end

```

getGroundTruthActorLog.m

Re-simulates a scenario from the start and logs every actor's true position/speed at every timestep. Used to match tracker identities to ground truth actors and to compute ground truth features for comparison against noisy tracked features.

```

function gtLog = getGroundTruthActorLog(scenario)
restart(scenario);
while advance(scenario)
    poses = actorPoses(scenario);
    % log Time, ActorName, X, Y, Speed, Yaw for every actor
end
end

```

D.4 Reactive Car-Following Scenarios

The reactive generators drive both ego and background traffic closed loop using an Intelligent Driver Model (IDM), live traffic signals, and pedestrian crossings, so behavior emerges from interaction rather than from pre-scripted trajectories.

generateReactiveScenario.m

Builds one reactive car following scenario on a crossroad road network. Ego speed is computed step by step via IDM against the nearest leader. A randomized traffic signal (green/yellow/red) and 1-3 pedestrian crossing windows are injected. Background vehicles move independently. Returns scenario, gtTable, meta, and simInfo for the closed-loop runner.

```

function [scenario, gtTable, meta, simInfo] = generateReactiveScenario(scenarioID,
rngSeed)
rng(rngSeed);
[scenario, roadInfo] = createRoadNetwork('crossroad');
duration = 60 + 30 * rand(); scenario.StopTime = duration;
% Traffic signal (randomized)

```

```

sig.greenDur = 15+10*rand(); sig.yellowDur = 2.5+1.5*rand(); sig.redDur =
15+10*rand();
% IDM parameters for ego
p.v0 = 10+6*rand(); p.T = 1.2+0.6*rand(); p.a = 1.2+0.8*rand(); p.b =
1.5+1.0*rand();
% Create ego, background vehicles (3-6), pedestrians (1-3)
egoActor = vehicle(scenario, 'ClassID', 1, 'Name', 'ego_reactive_car_following',
...);
% Returns: scenario, gtTable, meta, simInfo
end

```

reactiveCarFollowingLib.m

A function handle library used by the reactive runner: IDM acceleration with a leader, free road acceleration without one, traffic signal phase lookup, pedestrian activity check, and a combined ego control law that takes the most restrictive (minimum) acceleration across all active constraints.

```

function lib = reactiveCarFollowingLib()
lib.idmAcceleration = @idmAcceleration;
lib.freeRoadAcceleration = @freeRoadAcceleration;
lib.trafficSignalState = @trafficSignalState;
lib.pedestrianActive = @pedestrianActive;
lib.egoControlAcceleration = @egoControlAcceleration; end
function acc = idmAcceleration(v, vLead, gap, p)
sStar = p.s0 + max(0, v*p.T + (v*(v-vLead))/(2*sqrt(p.a*p.b)));
acc = p.a * (1 - (v/p.v0)^p.delta - (sStar/max(gap,0.1))^2); end
function [acc, info] = egoControlAcceleration(t, posEgo, vEgo, ...)
[acc, idx] = min(candidates); info.mode = modes{idx}; end

```

runAndRecordReactiveScenario.m

Advances a single crossroad reactive scenario frame by frame. Each step: (1) logs all actor poses, (2) ramps background vehicles toward a randomly changing target speed, (3) linearly interpolates pedestrians through their crossing windows, and (4) integrates ego motion under the IDM+signal+pedestrian acceleration. Outputs the standard .mat, trajectories.csv, and labels.csv files.

```

function recordTable = runAndRecordReactiveScenario(scenario, gtTable, meta,
simInfo, outDir)
lib = reactiveCarFollowingLib(); dt = scenario.SampleTime;
restart(scenario);
while advance(scenario)
t = scenario.SimulationTime;
% 1) Log all actor poses
% 2) Step background vehicles (random target speed, velocity ramping)
% 3) Step pedestrians (linear interpolation across crossing window)

```

```

% 4) Compute ego IDM acceleration; clip to [-6, p.a]; integrate
newVEgo = max(0, vEgo + acc*dt);
newXEgo = posEgo + vEgo*dt + 0.5*acc*dt^2;
end
% Save .mat, _trajectories.csv, _labels.csv
end

```

runAndRecordComplexReactiveScenario.m

The most elaborate driver: multi-crossroad network, multi-lane traffic, mixed actor types (pedestrian/child/cyclist, safe or risky), and a three-phase ego state machine (approach → turning → post_turn) for right-turn, left-turn, or U-turn manoeuvres. Background vehicles include occasional 15 % sudden brake pulses and autonomous lane changes. Same output layout as the simpler runners.

```

function recordTable = runAndRecordComplexReactiveScenario(scenario, gtTable,
meta, simInfo, outDir)
lib = complexScenarioLib(); dt = scenario.SampleTime;
egoPhase = 'approach'; % states: approach | turning | post_turn
while advance(scenario)
% 1) Log poses
% 2) Background: random speed + 15% sudden-brake pulses + lane changes
% 3) Crossing actors: linear interpolation per event window
% 4) Ego state machine:
%   approach -> IDM + multi-crossroad signals + lane changes
%   turning  -> smooth ease-in/out arc (right/left/U-turn)
%   post_turn -> cruise at 0.8*v0 along postTurnDir
end
% Save .mat, _trajectories.csv, _labels.csv
end

```

D.5 Complex Multi-Crossroad Route Scenarios

The most demanding tier of the dataset places the ego vehicle on a long route through several crossroads, with multiple lanes, lane changing background traffic, stalled obstacles to avoid, and mixed crossing actors at every intersection. The ten files below generate, route, and run these scenarios.

generateComplexReactiveScenario.m multi-crossroad reactive generator

Builds the richest scenario in the project: 2-3 signalized crossroads on one long road, 2 lanes with reactive lane changing, mixed background traffic (car/truck/bus/bicycle), and mixed crossing actors at each crossroad (65% cross safely on red, 35% jaywalk on green and are labelled risky). At one random crossroad the ego may turn right/left or U-turn (else goes

straight). Road length is sized so the ego can never run off the end before the simulation finishes.

```
function [scenario,gtTable,meta,simInfo] = ...
    rng(seed); duration=90..150s; build 2-lane road
    place 2-3 crossroads with random signal timings
    pick ego maneuver: straight/right/left/uturn (weighted)
    spawn ego + 5-9 background vehicles (mixed types)
    at each crossroad spawn 1-2 crossing actors,
        65% safe-on-red (label 0), 35% jaywalk-on-green (label 1)
    build gtTable of all actors + descriptions; return meta/simInfo
```

complexScenarioLib.m multi-crossroad control helpers

A helper function library for the reactive generator. Combines the IDM car following model with every upcoming signal and every upcoming pedestrian crossing, always applying the single most restrictive (minimum) acceleration. Also finds the nearest leading vehicle in lane and decides when a lane change is warranted.

```
idmAcceleration(v,vLead,gap,p) % IDM car-following
freeRoadAcceleration(v,p)      % no leader present
trafficSignalState(t,sig)      % red/yellow/green
egoControlAccelerationMulti(...) % min() over leader,
                                % all signals, all crossings
findLeaderInLane(...)          % nearest same-lane vehicle ahead
shouldChangeLane(...)          % change if leader close+slow
                                % and target lane is clear
```

createMultiCrossroadNetwork.m Long crossed-road network

Constructs a long main road (spanning ± 3000 m) crossed by 2–3 perpendicular crossroads whose positions are spread along the forward section of the road with small random jitter, guaranteeing the ego never runs out of road during an extended drive.

```
function [scenario,roadInfo] = createMultiCrossroadNetwork(n)
    build main road, length  $\pm 3000$  m, 2 lanes
    choose n crossroad x-positions (base + random jitter)
    add each crossroad as a perpendicular 2-lane road
    return scenario + roadInfo (crossroadXs, mainSpan)
```

buildChainedEgoRoute.m Chained multi-leg route

Builds a long ego route by chaining any sequence of straight segments and turns, using the constant-curvature unicycle path formula. This formula stays well behaved for left turns, right turns and U-turns alike, so no leg combination produces a malformed path.

```

function wp = buildChainedEgoRoute(legs)
pos=[0 0]; heading=0
for each leg:
    advance 'straightLength' forward, append waypoint
    if turnAngle ~= 0:
        kappa = sign(angle)/R % unicycle curvature
        sample arc points, update pos & heading
return wp % Nx3 waypoint list

```

generateComplexRouteScenario.m Long routed scenario

Builds a long, multi-intersection scenario in which the ego performs a straight/turn/U-turn maneuver while reacting live to a lead vehicle, per-crossroad signals, pedestrians, and a stalled obstacle that forces a lane change. The ego has no pre-scripted trajectory its motion is computed step by step later.

```

function [scenario,ego,gtTable,meta] = ...
    createMultiCrossroadNetwork(2-3 crossroads)
    pick turnType (straight/right/left/uturn) + turnX
    egoWaypoints = buildEgoRoute(turnType,turnX,far)
    place ego (position only, driven live later)
    per crossroad: random signal, 1-3 bg vehicles,
        0-2 pedestrians (safe/risky mix)
    add LeadVehicle + 1-2 stalled obstacle vehicles
    build gtTable + meta (waypoints, signals, obstacles)

```

runComplexRouteScenario.m Routed simulation and logging

Drives the ego along its precomputed route. At every timestep its forward speed is the minimum of three targets car-following speed, next-signal stop speed, and pedestrian-yield speed while it also shifts laterally to avoid stalled obstacles. The resulting path is logged and baked back into a real trajectory for later replay.

```

while advance(scenario):
    speed_lead = car-following speed vs nearest actor ahead
    speed_signal = 0 if next signal red & within braking dist
    speed_ped = 0 if pedestrian in crosswalk ahead
    egoSpeed -> min(all three), ramp by max accel/decel
    lateralOffset -> shift toward obstacle-avoid lane if needed
    s += egoSpeed*dt; update ego Position/Yaw/Velocity
    log row -> recordTable
bakeEgoTrajectoryFromLog(); save .mat + .csv

```

buildEgoRoute.m Single maneuver route builder

Builds the ego route centerline for one maneuver type straight, left turn, right turn, or U-turn at a given crossroad x-position, giving the ego a definite path through the intersection for the routed generator.

```
function wp = buildEgoRoute(turnType,turnX,farDistance)
    switch turnType
        straight: wp = [0 0 0; far 0 0]
        right/left: pre-turn straight + quarter-circle arc
                     (mirrored in y for left) + post-turn leg
        uturn: pre-turn straight + half-circle arc +
               return leg on opposite lane
```

findPathCrossingsAtX.m Locating crossroads along a route

Scans the ego route's waypoint segments and returns every arc-length distance at which the path crosses a given x-coordinate, considering only the roughly horizontal (main-road) segments. This is how the routed simulator maps each perpendicular crossroad and its signal onto the route.

```
function sList = findPathCrossingsAtX(wp,cumDist,xVal)
    for each segment (p0,p1):
        skip if mostly vertical (turn arc / crossroad segment)
        if xVal within [xmin,xmax] of segment:
            interpolate fractional s, append to sList
```

pathPointAtDistance.m Interpolating position and heading

Interpolates position [x y z] and heading (yaw, degrees) at a given arc-length distance along a waypoint polyline, letting the ego's pose be read continuously as it advances along its route.

```
function [pos,yawDeg] = pathPointAtDistance(wp,cumDist,s)
    clamp s to [0,totalLen]
    find bracketing segment via cumDist
    interpolate pos = p0 + frac*(p1-p0)
    yawDeg = atan2d(dy,dx) of that segment
```

pathCumulativeDistance.m Arc-length along a polyline

A short utility that returns the cumulative arc-length along a waypoint polyline, providing the s-parameterization that the two routing helpers above (D.5.8, D.5.9) depend on.

```
function cumDist = pathCumulativeDistance(wp)
    cumDist(1) = 0
    for i = 2:n
        cumDist(i) = cumDist(i-1) + norm(wp(i)-wp(i-1))
```

D.6 The Live Closed-Loop Demonstration

The culmination of the project is a live demonstration in which the trained classifier runs in real time and its verdicts actively shape the ego vehicle's driving. The files below generate the long demonstration scenario, run the classified closed loop, and visualize the risk signal as it unfolds.

generateLiveDemoScenario.m

Builds one 10–12-minute demo scenario: a multi-turn route with a U-turn, six crossroads (mixed car/truck/bicycle/pedestrian traffic, one converted into a rail crossing with a train), and three lane blocking stalled vehicle obstacles. The ego has no fixed trajectory; it is driven live and reactively later.

```
function [scenario, ego, gtTable, meta] = generateLiveDemoScenario(rngSeed)
    % Build route, place ego (position only, no trajectory)
    % Add 6 crossroads: 5 with mixed traffic, 1 rail crossing + train
    % Add safe/risky pedestrians and 3 lane-blocking obstacles
    % Return scenario, ego, ground-truth actor table, meta (route, events)
end
```

runLiveClassifiedDemo.m

The core closed-loop driver and the longest function in the project. Every simulation step it logs each actor's recent motion, and every 2 s it extracts features and classifies each actor with the trained model. Ego reacts to the verdicts live keeping extra following distance and steering around actors predicted risky, braking for signals, and yielding to pedestrians/trains while every prediction is logged. This produced the 5,832 live predictions across 18 actors reported in Section 5.2.

```
function summary = runLiveClassifiedDemo(scenario, ego, meta, trainedModel)
    while advance(scenario)
        % update per-actor history buffers
        % every 2s: extract features -> predict safe/risky -> log
        % adjust ego speed (car-following, signals, pedestrians/train)
        % adjust ego lateral offset to avoid obstacles / risky actors
        % render top-down chase view (green=safe, red=risky, yellow=?)
    end
    % return prediction log + final per-actor summary
end
```

live_risk_visualization.m

A radar based real time demo: simulates a radar sensor on the ego vehicle, tracks surrounding actors with a multi-object tracker, and classifies each confirmed track as safe/risky using a classifier trained without the ActorType predictor (radar alone cannot tell a car from a pedestrian). Renders a 3-D chase view and a bird's-eye view (green/red/gray boxes, yellow detection dots). Produces the summary artefacts behind Figures 5.7, 5.8, and E.1.

```
% Build scenario with SafeCar / RiskyCar / SafePedestrian / ChildChasingBall
radarSensor = drivingRadarDataGenerator(...);
tracker = multiObjectTracker(...);
while advance(scenario)
    detections = radarSensor(targetPoses(egoVehicle), time);
    tracks = updateTracks(tracker, detections, time);
    % update per-track history, classify once enough history exists
    % plot safe (green) / risky (red) / pending (gray) outlines
end
```

main_run_live_demo.m

Short entry point script: loads the trained classifier, generates the long demo scenario, launches the classified closed-loop run, and saves the prediction log and final summary to CSV.

```
load('trainedRiskClassifier_clean.mat');
[scenario, ego, gtTable, meta] = generateLiveDemoScenario(1);
summary = runLiveClassifiedDemo(scenario, ego, meta, trainedModel101);
writetable(summary.predLog, 'live_demo_predictions_log.csv');
writetable(summary.finalTable, 'live_demo_final_summary.csv');
```

bakeEgoTrajectoryFromLog.m

Converts a recorded ego position/speed log (from manually driving the actor in the closed loop) into an ordinary trajectory() assigned to the actor, so the run can be reloaded and replayed instead of the ego sitting motionless.

```
function bakeEgoTrajectoryFromLog(ego, recordTable, sampleEverySec)
    % sample recordTable at fixed interval -> waypoints + speeds
    % drop duplicate/consecutive-zero-speed waypoints
    trajectory(ego, wp, spd);
end
```

bakeReactiveTrajectoryForDesigner.m

Converts a saved reactive scenario's recorded closed-loop motion into ordinary trajectory-based actors, purely so it can be opened and played inside Driving Scenario Designer for visual inspection. The original ground truth reactive data file is left untouched.

```
function bakedScenario = bakeReactiveTrajectoryForDesigner(scenarioID,
datasetDir, outDir)
    data = load(scenario_XXX.mat);
    for each actor: rebuild waypoints/speeds from recordTable
        trajectory(act, wp, spd);
    save baked scenario for drivingScenarioDesigner(bakedScenario)
end
```

D.7 Batch Generation Drivers

The batch drivers are the scripts that were actually executed to produce the dataset in bulk, looping the generators of the previous groups over hundreds of random seeds and recording every scenario to disk. Five representative drivers are shown, together accumulating the full 450-scenario dataset.

main_generate_100_scenarios.m Baseline batch driver

Loops the baseline generator over 100 random seeds, generating and recording each scenario (120 s, highway/rural/village/mixed roads, 5-6 complexity events, mixed actor types) to dataset_100_scenarios/, then writes a summary CSV. Successive blocks like this accumulate the full 450-scenario dataset.

```
numScenarios = 100; outDir = 'dataset_100_scenarios'
for i = 1:numScenarios
    seed = 1000+i
    [scenario,gtTable,meta] = generateScenario(i,seed)
    runAndRecordScenario(scenario,gtTable,meta,outDir)
    summaryRows += {ID,roadType,duration,numEvents,
                    numRisky,numSafe}
    writetable(summaryTable, dataset_summary.csv)
```

main_generate_50_reactive_scenarios.m Reactive batch driver

Generates 50 closed-loop reactive scenarios (IDs 201-250): ego uses IDM car following against whichever vehicle is ahead, stops for a signalized crossroad, and yields to pedestrians at random crossing windows. Appends results into the same dataset folder and merges into the existing summary CSV.

main_generate_next_100_scenarios.m Appending the second batch

Appends 100 more scenarios (IDs 101-200), now including crossroad and rail crossing road types, into the same dataset folder without overwriting the first batch. Shows how the dataset grew incrementally as new road types were added to the generators.

main_generate_100_complex_reactive_scenarios.m Complex reactive batch

Generates 100 complex, multi-crossroad reactive scenarios (IDs 251-350): 2-3 signalized crossroads, 2-lane traffic with reactive lane changes, mixed background vehicle types, mixed crossing actors (35% risky), and an optional ego turn/U-turn per scenario. Invokes the richest generator from Appendix D.5 across 100 seeds.

main_generate_100_complex_route_scenarios.m Complex routed batch

Generates the final 100 scenarios (IDs 351-450): long multi-intersection routes where ego turns/U-turns, obeys each signal, yields to pedestrians, and changes lanes around stalled obstacles. Completes the 450-scenario dataset; writes per-scenario labels plus a separate complex route summary CSV (kept apart since its columns differ from the earlier summary format).

D.8 Visualization and Diagnostics

This group collects the utilities used to inspect scenarios visually and to diagnose problems in the generated data. They do not participate in training or evaluation, but they were indispensable during development several of the design decisions recorded earlier in the report were made only after one of these diagnostic scripts revealed exactly how a data file or a detection stream was malformed.

visualizeScenario.m Bird's-Eye Playback

Plays back a generated scenario as a top-down bird's-eye animation. This is the quickest way to visually confirm that a scenario's roads and actors were built as intended. Usage: generate a scenario, then call `visualize Scenario(scenario)`; for a richer 3-D view, `drivingScenarioDesigner(scenario)` can be opened instead.

visualizeReactiveScenario.m Reactive Replay

Replays a saved reactive scenario from its recorded trajectories. Reactive actors are moved step-by-step inside the simulation loop rather than through a preset trajectory, so driving Scenario's own `plot()` `advance()` playback cannot show them moving this function instead reconstructs the animation from the record Table saved to `scenario_XXX.mat`, drawing each actor as a colored marker and stepping through the unique recorded time values.

visualizeComplexReactiveScenario.m Complex Reactive Replay

Extends the reactive replay to the complex, multi-crossroad scenarios. Same principle as D.8.2, generalized to iterate over multiple crossroads (each with its own stop line and crosswalk markers) instead of a single intersection, and animates the recorded trajectories over the longer, multi-intersection road layout.

export_batch2_trajectories.m Unifying the Trajectory Format

Scenarios 351-450 were saved only as `*complexroute.mat` files containing the full `drivingScenario` object, without the per-actor trajectory CSV used by scenarios 1–350. This script loads each saved scenario, replays it frame-by-frame with `restart()/advance()`, reads `actorPoses()` at every step, and writes the result out as a trajectories CSV in exactly the same schema as batch 1–350 (ScenarioID, Time, ActorID, X, Y, Z, Vx, Vy, Vz, Yaw), plus an ActorID–Name lookup CSV so `labels.csv` can be joined correctly. It was written after a schema mismatch between batches was discovered, and it presents the whole dataset to the feature extractor as one uniform format.

```
for sid = 351:450
    S = load('scenario_sid_complexroute.mat');
    scenario = find drivingScenario object in S;
    save ActorID<->Name map to scenario_sid_actormap.csv
    restart(scenario);
    while advance(scenario)
        poses = actorPoses(scenario);
        append [sid, time, ActorID, X,Y,Z, Vx,Vy,Vz, Yaw] per actor
    end
    write rows to scenario_sid_trajectories.csv
end % failures are logged, not silently dropped
```

diagnose_radar_detections.m Detection Stream Diagnostics

Builds a minimal scenario (ego vehicle, one car, one pedestrian) and a `drivingRadarDataGenerator`, then prints the exact size and contents of every detection's Measurement vector for the first 15 simulation steps. This let inconsistencies in the detection measurement vectors be seen precisely, rather than guessed at, and directly informed the correct configuration of the tracker.

diagnose_csv_issues.m Data-File Diagnostics

Prints the raw first few lines of a handful of problem data files exactly as they sit on disk, with no MATLAB CSV parsing or interpretation applied. This makes it possible to identify precisely how a file is malformed and write a targeted fix, rather than guessing it is the tool behind several of the data cleaning decisions in the pipeline.

APPENDIX E: PERFORMANCE METRICS

This appendix defines the evaluation metrics used in Chapter 5. All are standard for binary classification and follow the definitions consolidated by Powers [37]. They are computed from the four entries of the confusion matrix: true positives (TP) and true negatives (TN), the correctly classified risky and safe actors, and false positives (FP) and false negatives (FN), the misclassified ones. Taking the risky class as the positive class throughout, the four counts arrange into the two-by-two table shown in Table E.1.

Table E.1: The confusion matrix for the binary safe/risky problem, with the risky class taken as positive.

	Predicted risky	Predicted safe
Actually risky	True positive (TP)	False negative (FN)
Actually safe	False positive (FP)	True negative (TN)

From these four counts the headline metrics are defined as follows.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (\text{E.1})$$

$$Precision = TP / (TP + FP) \quad (\text{E.2})$$

$$Recall = TP / (TP + FN) \quad (\text{E.3})$$

$$F_1 = 2 \cdot (Precision \cdot Recall) / (Precision + Recall) \quad (\text{E.4})$$

Accuracy, used as the headline figure throughout Chapter 5, is the proportion of all actors classified correctly. It is a fair summary here because, as Figure 3.8 showed, the two classes are both well represented; on a heavily imbalanced dataset accuracy would be misleading and precision, recall, and the F-score would be the more informative measures. In the safety context of this project, recall on the risky class the fraction of genuinely risky actors that the system flags is the metric with the gravest consequences, since a false negative is a missed danger, and it is the quantity most damaged by the robustness collapse of Section 5.3.3.

Two further quantities help interpret the robustness result. Specificity, the fraction of genuinely safe actors correctly passed, measures the opposite error to recall, and its complement the false-positive rate governs how often the vehicle would break or hesitate for an actor that was in fact safe, the nuisance behaviors that erodes a driver’s trust in an assistance system.

$$Specificity = TN / (TN + FP) \quad (\text{E.5})$$

Because precision, recall, and specificity each tell only part of the story, they are most safely read together. Balanced accuracy, the mean of recall and specificity, is particularly informative when a classifier collapses toward one class, because it does not reward a model that simply predicts the majority label.

$$\text{Balanced accuracy} = (\text{Recall} + \text{Specificity}) / 2 \quad (\text{E.6})$$

When both classes matter equally, as they do here, the per class F-scores are combined by a simple unweighted mean the macro-average rather than by a frequency weighted mean, so that good performance on the more common class cannot mask poor performance on the rarer one. All of these metrics are computed on the held-out test set only, and, as Section 5.3 stresses, that test set is separated from the training set at the level of whole scenarios rather than individual actors; without this precaution the metrics above would be optimistically biased, because near-identical actors from a shared scenario could otherwise appear in both sets and inflate every figure in this appendix.

Figure E.1 shows the live demonstration risky share signal over the full run, the time resolved companion to the summary of Figure 5.8. It illustrates how the fraction of actors judged risky rises and falls as the drive unfolds and as actors move through the radar’s field of view.

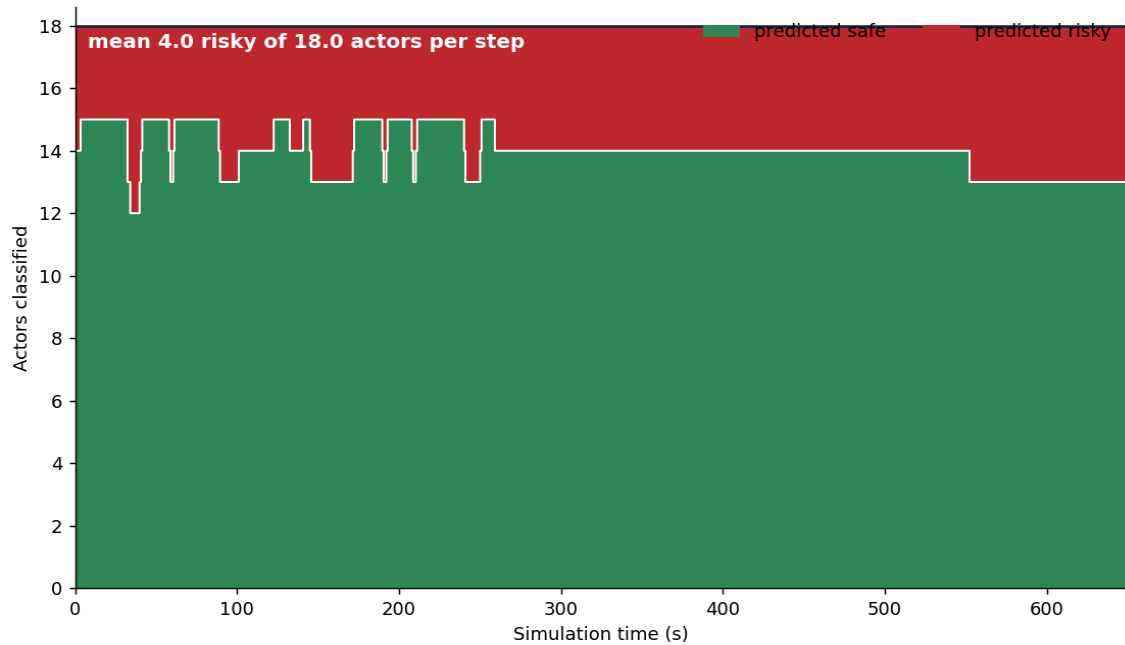


Figure E.1: Live demonstration risky share signal over the full run: the proportion of tracked actors classified risky at each instant, the time resolved companion to the per-actor summary of Figure 5.8.